Final Year Project

FixKar

Project Team:

Ms. Zainab Asif
2280134
Mr. Muhammad Osama
2280151

Project Supervisor:

Mr. Usama Khalid

January 26, 2026

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Software Engineering
in the
Faculty of Computing and Engineering Sciences
Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology University
(SZABIST University) Karachi Campus

Table of Contents

Project Proposal **1**
1. Introduction **2**
2. Objective **2**
3. Problem Description **2**
4. Target Industry **2**
5. Methodology **3**
6. Project Scope **3**
7. Feasibility Study **4**
8. Scientific Areas / Application Covered **4**
9. Emerging Technologies Used **5**
10. Correlation with the Industrial Practices and Trends **5**
11. Expertise of the Team Members **6**
12. Milestones **6**
13. Project Schedule **7**
14. Work Breakdown Structure **8**
15. Business Canvas Model **9**
Software Requirement Specification **10**
1. Introduction **11**
1.1 Purpose 11
1.2 Document Conventions 11
1.3 Intended Audience and Reading Suggestions 11
1.4 Product Scope 12
1.5 References 12
2. Overall Description **12**
2.1 Product Perspective 12
2.2 Product Functions 13
2.3 User Classes and Characteristics 13
2.4 Operating Environment 14
2.5 Design and Implementation Constraints 14
2.6 User Documentation 15
2.7 Assumptions and Dependencies 15
3. External Interface Requirements **15**
3.1 User Interfaces 15
3.2 Hardware Interfaces 16
3.3 Software Interfaces 16
3.4 Communications Interfaces 16
4. System Features **16**
4.1 User Account Management 16
4.2 Vehicle Management 22
4.3 Service Request Handling 25
4.4 Live Tracking 30
4.5 Real Time Chat 33
4.6 AI Assistance 35
4.7 Reviews and Feedback 36
4.8 Payment and Financial Management 38
4.9 Admin Management and Analytics 43
4.10 Multilingual Support & In-App Tutorials 46
4.11 Notifications 48
5. Other Nonfunctional Requirements **51**

5.1 Performance Requirements 51
5.2 Safety Requirements 51
5.3 Security Requirements 51
5.4 Software Quality Attributes 51
5.5 Business Rules 52
6. Other Requirements 52
6.1 Appendix A: Glossary 52
6.2 Appendix C: To Be Determined List 53
Software Design Specification 54
1. Introduction 55
1.1 Purpose of this document 55
1.2 Scope of the development project 55
1.3 Definitions, acronyms, and abbreviations 55
1.4 Overview of document 56
2. System architecture description 56
2.1 Section Overview 56
2.2 General Constraints 56
2.3 Data Design 57
2.4 Program Structure 58
2.5 Alternatives Considered 59
3. Detailed description of Components 60
3.1 Section Overview 60
3.2 Component in Detail 60
3.2.1 Client 60
3.2.2 Mechanic 62
3.2.3 AI_Chat 65
3.2.4 Service Request 66
3.2.5 Car 69
3.2.6 Chat 71
3.2.7 Message 72
3.2.8 Review 74
3.2.9 Bill 76
3.2.10 Notification 78
3.2.11 Service 80
3.2.12 Admin 82
4. User Interface Design 85
4.1 Section Overview 85
4.2 Interface Design Rules 86
4.3 GUI Components 86
4.4 Detailed Description 87
5. Reuse and Relationship to Other Products 103
6. Design Decisions and Tradeoffs 103
7. Pseudo for components 104
7.1 Auth Controller 104
7.2 Mechanic Controller 107
7.3 Client Controller 110
7.4 Car Controller 112
7.5 AI Chat Controller 115
7.6 Agendas 116
7.7 Request Controller 117
7.8 Message Controller 120
7.9 Inbox Controller 122
7.10 Admin Controller 123
7.11 Sockets 124
7.12 Images Via Cloudinary 126

8. Design Diagrams: 127
8.1 Class Diagram: 127
8.2 State Chart Diagram 128
8.2.1 Admin State Chart Diagram 128
8.2.2 Client State Chart Diagram 128
8.2.3 Mechanic State Chart Diagram 129
8.3 Sequence Diagrams 130
8.3.1 Client SignUp and Login 130
8.3.2 Mechanic SignUp and Login 131
8.3.3 Add a Car 132
8.3.4 View All Cars 133
8.3.5 Delete a Car 134
8.3.6 Forgot Password 135
8.3.7 AI Chat 136
8.3.8 Service Booking Flow 137
8.3.9 Chat Module 138
8.3.10 Live Track Module 139
8.3.11 Payment Module 140
8.3.12 Review and Rating Module 141
8.3.13 Multilingual Support Module 142
8.3.14 Notification Management 143
8.3.15 Create New Admin 144
8.3.16 Payment Management 145
8.3.17 User Management 146
8.3.18 Analytics Management 147
8.3.19 Approve/Reject Mechanics 148
8.3.20 Booking Management 149
8.4 Activity Diagrams 150
8.4.1 Client SignUp and Login 150
8.4.2 Mechanic SignUp and Login 151
8.4.3 Forgot Password 152
8.4.4 Add a Car 153
8.4.5 View All Cars 154
8.4.6 Delete a Car 155
8.4.7 AI Chat 156
8.4.8 Service Booking Flow 157
8.4.9 Chat Module 158
8.4.10 Payment Module 159
8.4.11 Live Tracking Module 160
8.4.12 Review and Rating Module 161
8.4.13 Multilingual Support Module 162
8.4.14 Notification Management 163
8.4.15 Create New Admin 164
8.4.16 Payment Management 165
8.4.17 User Management 166
8.4.18 Analytics Management 167
8.4.19 Approve/ Reject Mechanics 168
8.4.20 Booking Management 169
8.5 Use Case Diagrams 170
8.5.1 Admin Use Case Diagram 170
8.5.2 Client Use Case Diagram 171
8.5.3 Mechanic Use Case Diagram 172
8.6 Component Diagram 173
Test Cases 174

1. User Account Management 175
1.1 SignUp 175
1.2 Forgot Password 177
1.3 Change Password 180
1.4 Login 182
1.5 Admin Approval/Rejection 183
1.6 Profile Updation (User) 184
1.7 Profile Updation (Mechanic Vehicle Expertise) 186
2. Vehicle Management 187
2.1 View My Cars 187
2.2 Add a New Car 188
2.3 Edit Car 191
2.4 View Car Details 194
2.5 Delete Car 196
3. AI Chat 197
4. Service Request Handling 200
4.1 Request For Service 200
4.2 View Nearby Requests 202
4.3 Cancel the Request 204
5. Real Time Chat 205
6. Live Tracking 207
6.1 View Location 207
6.2 Live Track Mechanic 208

Table of Tables

Table 1: Sign Up - Use Case 18
Table 2: Sign In – Use Case 19
Table 3: Profile Updation – Use Case 20
Table 4: Forgot Password – Use Case 21
Table 5: User Account Management - Functional Requirements 22
Table 6: Add Vehicle – Use Case 23
Table 7: Update Vehicle – Use Case 24
Table 8: Delete Vehicle – Use Case 24
Table 9: Vehicle Management - Functional Requirements 25
Table 10: Request for Service – Use Case 26
Table 11: View Nearby Requests – Use Case 26
Table 12: Offer Instant Quote or Toggle for After-Inspection Prices – Use Case 27
Table 13: View the Quotes – Use Case 28
Table 14: Accept the Quote – Use Case 28
Table 15: Reject the Quote – Use Case 29
Table 16: Cancel Service Request – Use Case 29
Table 17: Service Request Handling - Functional Requirements 30
Table 18: Live Tracking Mechanic – Use Case 31
Table 19: View Client Location On Map – Use Case 32
Table 20: Live Tracking - Functional Requirements 32
Table 21: Real-Time Chat – Use Case 34
Table 22: Real-Time Chat - Functional Requirements 34
Table 23: AI Assistance – Use Case 36
Table 24: AI Assistance – Functional Requirements 36
Table 25: Client Feedback Submission – Use Case 37
Table 26: View Ratings and Reviews – Use Case 37

Table 27: Reviews and Feedback – Functional Requirements 38
Table 28: Enter Price Before Starting Service (After-Inspection Price) – Use Case 39
Table 29: Update Service Status – Use Case 40
Table 30: Generate Bill – Use Case 41
Table 31: Make Payments Manually – Use Case 42
Table 32: Make Payment Online – Use Case 42
Table 33: Payment and Financial Management - Functional Requirements 43
Table 34: Admin Management and Analytics – Use Case 45
Table 35: Admin Management and Analytics - Functional Requirements 46
Table 36: Multilingual Support – Use Case 47
Table 37: In-App Tutorials (OPT) – Use Case 47
Table 38: Multilingual Support & In-App Tutorials - Functional Requirements 48
Table 39: Real-Time Notification System – Use Case 50
Table 40: Notification - Functional Requirements 50
Table 41: Glossary – Fixkar 53
Table 42: Definitions - SDS 55
Table 43: Component - Client 62
Table 44: Component - Mechanic 65
Table 45: Component - AI_Chat 66
Table 46: Component – ServiveRequest 68
Table 47: Component – Car 70
Table 48: Component - Chat 72
Table 49: Component - Message 74
Table 50: Component - Review 76
Table 51: Component - Bill 78
Table 52: Component - Notification 80
Table 53: Component - Service 82
Table 54: Component - Admin 85
Table 55: SignUp Test Case 177
Table 56: Forgot Password Test Case 180
Table 57: Change Password Test Case 181
Table 58: Login Test Case 183
Table 59: Admin Approval/Rejection Test Case 184
Table 60: Profile Updation (User) Test Case 185
Table 61: Profile Updation (Mechanic Vehicle expertise) Test Case 186
Table 62: View My Cars Test Case 188
Table 63: Add New Car Test Case 191
Table 64: Edit Car Test Case 194
Table 65: View Car Details Test Case 195
Table 66: Delete Car Test Case 197
Table 67: AI Chat Test Case 199
Table 68: Request For Service Test Case 201
Table 69: View Nearby Requests Test Case 204
Table 70: Cancel Request Test Case 205
Table 71: Real Time Chat Test Case 207
Table 72: View Location Test Case 208
Table 73: Live Track Mechanic Test Case 210

Table of Figures

Figure 1: Project Schedule - FYP1 7
Figure 2: Project Schedule – FYP 2 8

Figure 3: Work Break Down Structure 9
Figure 4: Business Canvas Model 9
Figure 5: Product Perspective Diagram 13
Figure 6: ERD Diagram 57
Figure 7: Splash Screen 87
Figure 8: On-boarding Screen - 1 87
Figure 9: On-boarding Screen - 2 87
Figure 10: On-boarding Screen - 3 88
Figure 11: On-boarding Screen - 4 88
Figure 12: Choose Role Screen 88
Figure 13: SignUp Screen 89
Figure 14: OTP Screen 89
Figure 15: SignUp Detail Screen - Client 89
Figure 16: SignUp Detail Screen - Mechanic - 1 90
Figure 17: SignUp Detail Screen - Mechanic - 2 90
Figure 18: SignUp Detail Screen -Mechanic - 3 90
Figure 19: SignUp Detail Screen -Mechanic - 4 91
Figure 20: SignUp Detail Screen - Mechanic - 5 91
Figure 21: SignUp Detail Screen - Mechanic - 6 91
Figure 22: Login Screen 92
Figure 23: Profile Screen - Client - 1 92
Figure 24: Profile Screen - Client - 2 92
Figure 25: Profile Screen - Mechanic - 1 93
Figure 26: Profile Screen - Mechanic - 2 93
Figure 27: Profile Screen - Mechanic - 3 93
Figure 28: My Cars Screen 94
Figure 29: Add Car Screen 94
Figure 30: Edit Car Screen 94
Figure 31: AI Chat Screen - 1 95
Figure 32: AI Chat Screen - 2 95
Figure 33: Dashboard Screen - Client 96
Figure 34: Vehicle Type Screen 96
Figure 35: Search Location Screen 96
Figure 36: Service Detail Screen - 1 97
Figure 37: Service Detail Screen - 2 97
Figure 38: Active Request - Client - 1 97
Figure 39: Active Screen - Client - 2 98
Figure 40: Dashboard Screen - Mechanic 98
Figure 41: Request Screen 98
Figure 42: Request Detail Screen - 1 99
Figure 43: Request Detail Screen - 2 99
Figure 44: Chat Screen - Mechanic 99
Figure 45: Inbox Screen - Client 100
Figure 46: Chat Screen - Client 100
Figure 47: Active Screen - Client - 3 100
Figure 48: Active Screen - Mechanic - 1 101
Figure 49: Active Screen - Mechanic - 2 101
Figure 50: Live Tracking - Client 101
Figure 51: Live Tracking - Mechanic 102
Figure 52: Cancel Request Screen 102
Figure 53: Toasts 102
Figure 54: Class Diagram 127
Figure 55: Admin - State Chart Diagram 128
Figure 56: Client - State Chart Diagram 128

Figure 57: Mechanic - State Chart Diagram 129
Figure 58: Client SignUp and Login - Sequence Diagram 130
Figure 59: Mechanic SignUp and Login - Sequence Diagram 131
Figure 60: Add a Car - Sequence Diagram 132
Figure 61: View All Cars – Sequence Diagram 133
Figure 62: Delete a Car - Sequence Diagram 134
Figure 63: Forgot Password - Sequence Diagram 135
Figure 64: AI Chat - Sequence Diagram 136
Figure 65: Service Booking Flow - Sequence Diagram 137
Figure 66: Chat Module - Sequence Diagram 138
Figure 67: Live Track Module - Sequence Diagram 139
Figure 68: Payment Module - Sequence Diagram 140
Figure 69: Review and Rating Module - Sequence Diagram 141
Figure 70: Multilingual Support Module - Sequence Diagram 142
Figure 71: Notification Management - Sequence Diagram 143
Figure 72: Create New Admin - Sequence Diagram 144
Figure 73: Payment Management - Sequence Diagram 145
Figure 74: User Management - Sequence Diagram 146
Figure 75: Analytics Management - Sequence Diagram 147
Figure 76: Approve/Reject Mechanics - Sequence Diagram 148
Figure 77: Booking Management - Sequence Diagram 149
Figure 78: Client SignUp and Login - Activity Diagram 150
Figure 79: Mechanic SignUp and Login - Activity Diagram 151
Figure 80: Forgot Password - Activity Diagram 152
Figure 81: Add a Car - Activity Diagram 153
Figure 82: View All Cars - Activity Diagram 154
Figure 83: Delete a Car - Activity Diagram 155
Figure 84: AI Chat - Activity Diagram 156
Figure 85: Service Booking Flow - Activity Diagram 157
Figure 86: Chat Module - Activity Diagram 158
Figure 87: Payment Module - Activity Diagram 159
Figure 88: Live Tracking Module – Activity Diagram 160
Figure 89: Review and Rating Module – Activity Diagram 161
Figure 90: Multilingual Support Module - Activity Diagram 162
Figure 91: Notification Management - Activity Diagram 163
Figure 92: Create New Admin - Activity Diagram 164
Figure 93: Payment Management - Activity Diagram 165
Figure 94: User Management - Activity Diagram 166
Figure 95: Analytics Management - Activity Diagram 167
Figure 96: Approve/Reject Mechanics - Activity Diagram 168
Figure 97: Booking Management - Activity Diagram 169
Figure 98: Admin – Use Case Diagram 170
Figure 99: Client - Use Case Diagram 171
Figure 100: Mechanic - Use Case Diagram 172
Figure 101: Component Diagram 173

Revision History

Name	Date	Reason For Changes	Version

Project Proposal

1. Introduction

In emergency and routine vehicle breakdown situations, timely access to a skilled mechanic is often a challenge, especially when stranded or unaware of nearby services. FixKar is a mobile-based solution that connects users to nearby verified mechanics within a 1 km radius in emergencies. The app bridges the gap between users and reliable roadside assistance, bringing real-time help to fingertips with transparency, affordability, and AI support.

2. Objective

To design and develop a real-time mobile application that connects users with nearby verified mechanics for emergency services, incorporating AI-based vehicle assistance, featuring live tracking, user reviews, and admin-level monitoring through a web dashboard.

3. Problem Description

Vehicle breakdowns, whether on highways or within cities, often leave users helpless, especially during off-hours or in unfamiliar areas. Traditional towing or workshop discovery is slow, non-transparent, and lacks verified credibility of service providers. Additionally, routine vehicle care such as oil changes or general inspections lack easy booking options with at-home availability.
FixKar addresses:

- The absence of a verified, transparent, and real-time mechanic booking system.
- Lack of live location tracking during mechanic dispatch, causing uncertainty for users.
- Emergency roadside situations requiring nearby immediate help.
- No feedback loop or rating system to filter poor service quality.
- Communication and education gaps, especially for less-tech-savvy mechanics.
- Absence of intelligent AI guidance for vehicle-related queries.

This system brings affordability, accountability, and informed decision-making to the mechanic-user interaction through reviews, live tracking, geolocation, chat, and verified onboarding.

4. Target Industry

The target industry is the automobile assistance and emergency response sector.

5. Methodology

- The mobile and web application will be built using MERN Stack.
- Google Maps API will be used for real-time location tracking.
- WebSockets (Socket.IO) will enable real-time chat between users and mechanics and handle live booking status updates.
- Mechanic identity verification will be conducted via the Tasdeeq using CNIC information (manually).
- Payments will be processed using Stripe API, and the app will retain 10% commission per successful booking.
- AI-based vehicle support will be provided through OpenRouter (GPT), limited to vehicle-related queries.
- Notifications will be handled using FCM.
- Image uploads will be handled using Cloudinary.

6. Project Scope

FixKar will focus on connecting vehicle users with verified mechanics, including:

- Mechanic registration and verification by admin via CNIC using Tasdeeq Verification manually.
- Emergency bookings based on real-time location, with options for upfront or post-inspection pricing.
- Live GPS tracking of mechanics during active bookings.
- In-app chat between users and mechanics for better coordination.
- AI-based vehicle assistance using GPT (vehicle-related queries only).
- Multilingual UI (English & Urdu) and video tutorials for mechanics (opt).
- Ratings and reviews from users after service completion.
- Transparent billing, viewable by both user and mechanic.
- Mechanic pays 10% commission to the app via Stripe after successful service completion.
- Real-time notifications for clients, mechanics will be handled through the app across all modes.
- Admin oversight of users, mechanics, bookings, and revenue analytics.

For this project, the system will not cover multi-mechanic or collaborative bookings, advanced vehicle diagnostics beyond AI text assistance, offline app functionality, integrations with external services (such as insurance providers or towing companies). These features may be considered for future enhancements.

7. Feasibility Study

With above defined scope, we are hopeful that we'll complete the project within the time constraints defined.

Risks Involved:

- **Technical literacy gap:** Mitigated through in-app video tutorials and a bilingual interface (English & Urdu).
- **Fake or unverified mechanic registrations**: Addressed by using **Tasdeeq API manually** for CNIC-based verification, ensuring only credible mechanics are allowed on the platform.
- **AI misuse**: Controlled by limiting GPT-based responses strictly to vehicle-related queries using system-level instructions.
- **Mechanic non-payment of commission**: Managed by setting payment deadlines and automatic fines. Repeat violations may result in account suspension or permanent blacklisting.
- **User refusal after mechanic's inspection (post-inspection pricing model):** If a mechanic quotes the price after inspection and the user declines service, a standard Rs 250 **inspection visit fee** is enforced to compensate the mechanic's time and travel.
- **Mechanic no-show or overcharging**: Managed via review system, shared bill transparency, and fixed arrival charges if the service is declined post-inspection.

Resource Requirement:

- Development laptop with internet access
- Free-tier access to APIs (Google Maps, Stripe, Tasdeeq, OpenRouter)
- WebSocket hosting (using Socket.IO) for real-time communication.
- FCM for notifications.
- Cloud-based database and storage: MongoDB Atlas (data) and Cloudinary (media uploads).

8. Scientific Areas / Application Covered

This solution has high real-world value in the **automobile assistance** and **emergency response** sectors. It specifically targets:

- Roadside Assistance Industry
- Urban/Remote Mobility
- Gig Economy for Mechanics

Benefits:

- Reduces waiting time during emergencies
- Improves service reliability through identity verification and ratings

Creates income opportunities for local service providers

9. Emerging Technologies Used

Frontend:

- React Native (User and Mechanic Apps)
- ReactJS (Admin Panel)
- Bootstrap (UI Styling)

Backend:

- NodeJS and ExpressJS (API development)
- MongoDB (Database management)
- Socket.IO (Real time Chats)

APIs/Integrations:

- Google Maps (Location services)
- OpenRouter GPT API (Restricted chatbot)
- Stripe API (Payments)

Other Tools:

- GitHub
- Postman
- Cloudinary (for hosting videos and images)
- Firebase Services (FCM)
- AWS and Vercel (Hosting App and Web)

10. Correlation with the Industrial Practices and Trends

The proposed FixKar system aligns with current industrial practices in the automobile assistance and mobility service sector. The industry is rapidly moving towards on-demand, app-based service platforms that provide real-time access to nearby service providers, transparent pricing, and digital communication. FixKar follows this trend by enabling location-based mechanic discovery, live tracking, and in-app chat.

The inclusion of verified mechanics, user reviews, and digital payments reflects widely adopted industry standards that emphasize trust, accountability, and service quality. Furthermore, the use of AI-based vehicle assistance corresponds with emerging trends in intelligent support systems

aimed at improving user experience and decision-making. Overall, the proposed system adopts modern technological and operational practices, making it relevant, practical, and aligned with current industry trends.

11. Expertise of the Team Members

Both team members are well-equipped with the skills required for successful project execution. We have strong command over the MERN stack, real-time communication using WebSocket's, and API integrations. We are adaptive and quick learners, ready to explore and integrate any new technology required during development. This project aligns with our technical strengths and is of equal interest to both team members

12. Milestones

FYP-1:

1. **User Account Management:**
   Users shall be able to sign up, log in, recover their password using the "forgot password" feature, and update their profiles. Mechanic accounts shall require **admin approval via Tasdeeq verification(manually)** before activation.
2. **Vehicle Management:**
   Clients shall be able to add, update, or delete their vehicle details that can be reused while making future service requests.
3. **AI Assistance:**
   Clients shall have access to an AI assistant that responds only to vehicle-related queries.
4. **Service Request Handling:**
   Clients shall be able to request mechanics for assistance; nearby available mechanics can respond to these requests in real time and either offer an instant quote or provide a price after inspection.
5. **Live Tracking:**
   Clients shall be able to track the live location of the assigned mechanic on a map, moreover the mechanic shall be able to see client location on the map.
6. **Real Time Chat:**
   Clients and mechanics shall be able to communicate via real-time chat.

FYP-2:

7. **Reviews and Feedback:**
   Clients shall be able to submit feedback and reviews once the service is completed.
8. **Admin Management and Analytics:**
   Admin shall be able to manage users, mechanics, bookings, and financial operations, including payments and commission distribution, while also monitoring system analytics.
9. **Payments and Financial Management:**
   Clients shall be able to pay for services through an integrated payment system; both admin and mechanic shall be able to manage and split payments via the system.
10. **Notifications:**
    Real-time notifications for clients, mechanics will be handled through the app across all modes.
11. **Multilingual Support and In-App Tutorials:**
    The mechanic app shall offer multilingual support and include in-app tutorial videos for ease of use. (optional)

13. Project Schedule

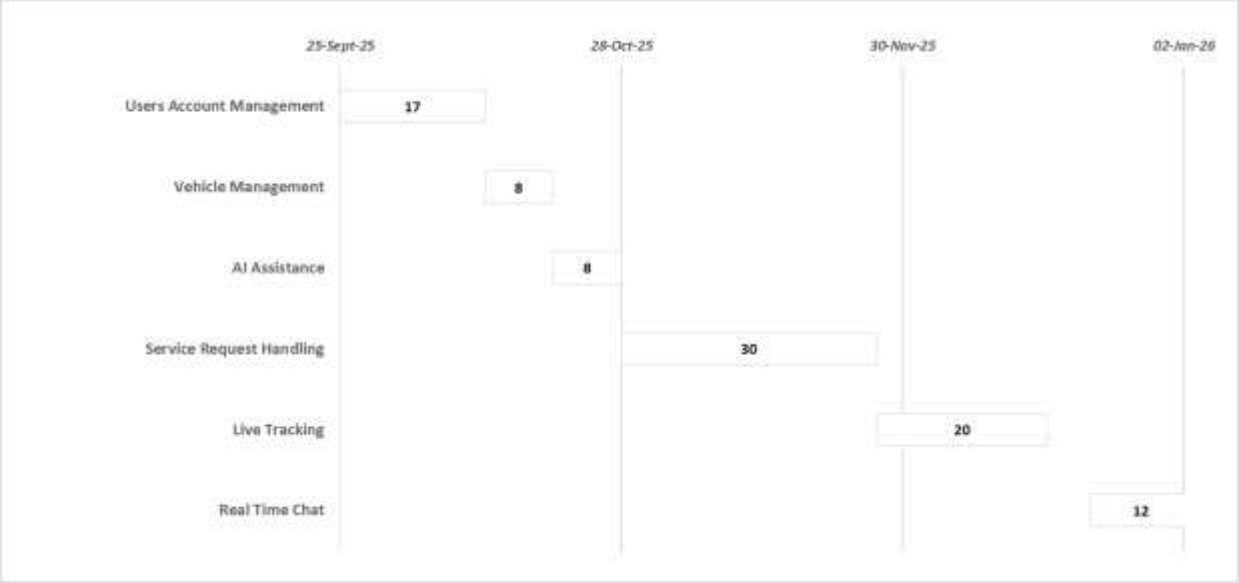


Figure 1: Project Schedule - FYP1

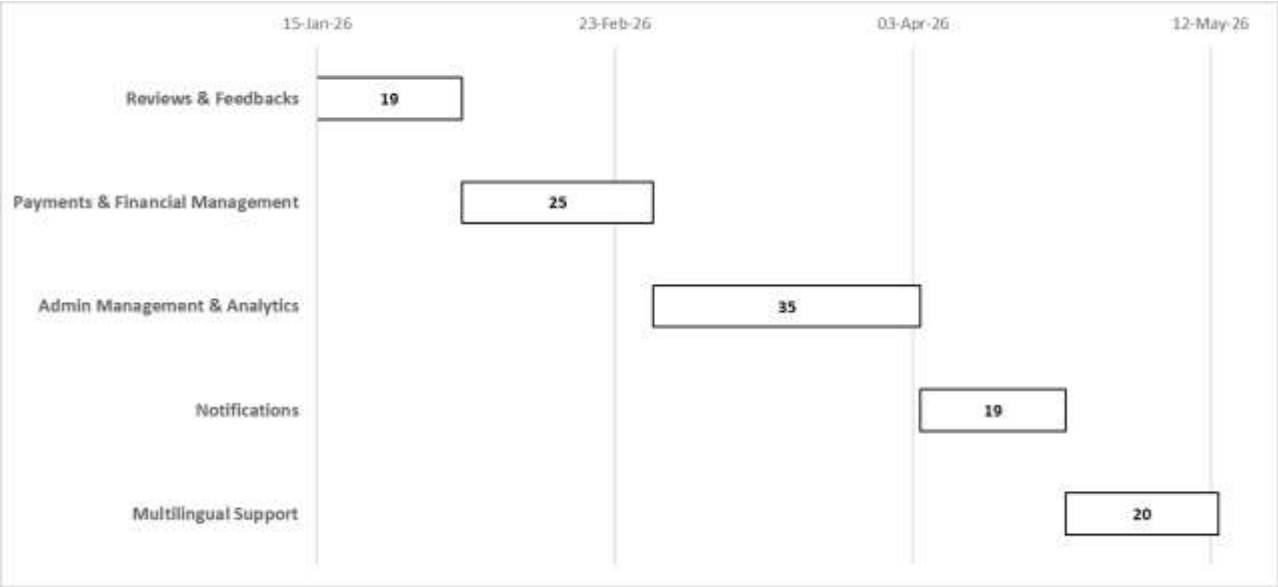
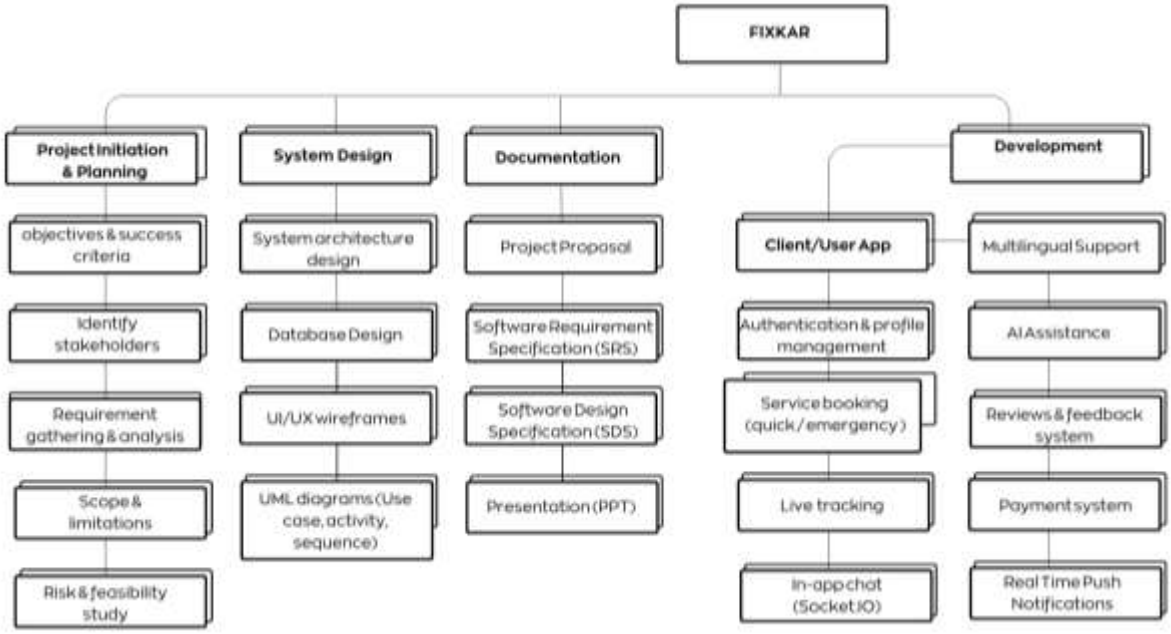


Figure 2: Project Schedule – FYP 2

14. Work Breakdown Structure



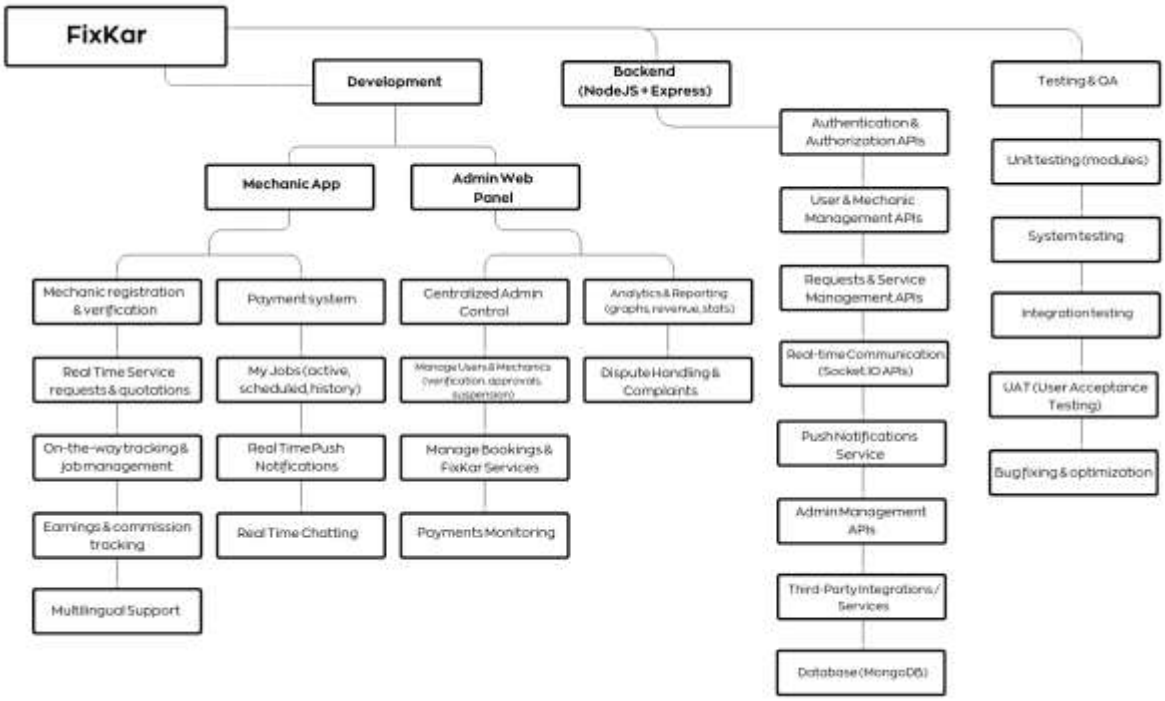


Figure 3: Work Break Down Structure

15. Business Canvas Model

Figure 4: Business Canvas Model

Software Requirement Specification

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for **FixKar**, a multi-platform roadside assistance and automobile service management system. This document specifies the requirements for the initial release (Version 1.0).
The system will consist of:

- A mobile application for **clients** to request on-demand mechanics, track live locations, communicate via chat, make secure payments, and access AI-powered vehicle assistance.
- A mobile application for **mechanics** to receive and respond to service requests in real time, provide quotes, update job status, and access in-app learning materials (tutorial videos, multilingual support).
- A web-based **admin** portal to manage user and mechanic registrations, verify mechanics via *Tasdeeq* (manually), manage bookings, oversee payments and commission distribution, enforce cancellation policies, and view system analytics (monthly bookings, mechanic count, booking status distribution, revenue trends).

1.2 Document Conventions

This document follows the IEEE 830-1998 standard for software requirements specification.

- Section Headings are numbered hierarchically.
- Font family: Times New Roman.
- Functional requirements are expressed using the keywords:
    - “Shall” denotes a mandatory requirement.
    - “Should” denotes a recommended but not mandatory requirement.
    - “May” denotes an optional feature.
- Bold text is used to emphasize key terms, roles, and system names.
- Italics are used for referencing external systems, APIs, or integration names.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following stakeholders:

- Project Supervisors & Evaluators
- Developers & Designers
- Quality Assurance Testers
- Project Managers

1.4 Product Scope

FixKar is designed to bridge the gap between stranded vehicle owners and verified **mechanics** by providing real-time roadside assistance within a 1-5 km radius.
Key goals include:

- Immediate access to verified mechanics in emergency situations via real-time location tracking.
- Transparent service process with instant or post-inspection price quotes.
- In-app communication between **client** and **mechanic.**
- Secure payment system for commission distribution.
- AI-powered vehicle troubleshooting for common issues.
- Ratings and reviews from users after service completion.
- Comprehensive **admin** control over user management, **mechanic** verification, analytics, and policy enforcement.

1.5 References

- "Google Maps Platform Documentation." https://developers.google.com/maps.
- "inDrive pioneering a sustainable 'fair' ride-hailing business." https://www.intellinews.com/indrive-pioneering-a-sustainable-fair-ride-hailing-business-377028/
- "How does Ride handling app works?" https://www.youtube.com/watch?v=POSMcu_ze8E

2. Overall Description

2.1 Product Perspective

FixKar is a new, self-contained, multi-platform product designed specifically for roadside assistance and automobile service management. It is not a follow-on member of an existing product family but rather an innovative integration of location-based services, on-demand gig economy **mechanics**, AI-based vehicle troubleshooting, and secure payment processing.

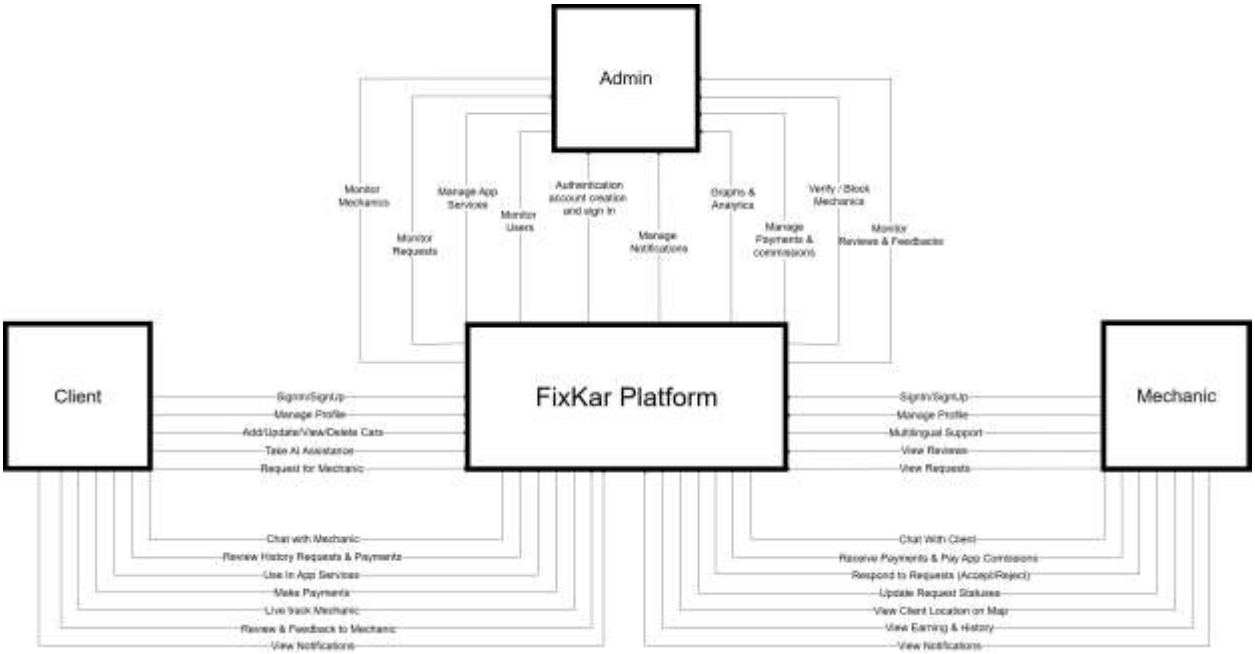


Figure 5: Product Perspective Diagram

2.2 Product Functions

1. User Account Management
2. Vehicle Management
3. Service Request Handling
4. Live Tracking
5. Real-Time Communication
6. AI Assistance
7. Feedback & Reviews
8. Payment & Financial Management
9. Admin Management & Analytics
10. Notifications
11. Multilingual Support & In-App Tutorials (opt)

2.3 User Classes and Characteristics

1. **Clients**
   - **Frequency of Use:** Occasional to frequent, depending on vehicle needs.
   - **Functions Used:** Search mechanics, request services, track mechanic location, chat, make payments, submit reviews.
   - **Technical Expertise:** Basic smartphone usage skills.

- **Security/Privilege Level:** Can access only their own booking, payment, and chat data.
- **Importance:** Primary

2. **Mechanics**
    - **Frequency of Use:** Frequent/daily during work hours.
    - **Functions Used:** Accept/reject service requests, update status, navigate to client, chat, receive payments, manage profile.
    - **Technical Expertise:** Basic to moderate mobile app skills.
    - **Security/Privilege Level:** Can access only assigned bookings, earnings, and profile details.
    - **Importance:** Primary

3. **Admins**
    - **Frequency of Use:** Daily for system oversight.
    - **Functions Used:** Manage user accounts, verify mechanics, view analytics, handle disputes, manage payments.
    - **Technical Expertise:** Moderate to high system management knowledge.
    - **Security/Privilege Level:** Full system access.
    - **Importance:** Primary

2.4 Operating Environment

1. Smartphones (Android)
2. Laptops/desktops (Windows/macOS/Linux)
3. Reliable internet connection required

2.5 Design and Implementation Constraints

1. **Time Required:** May 2026
2. **Tools & Technologies used:**
    - React Native (mobile), ReactJS (admin dashboard)
    - Node.js, Express.js
    - MongoDB
    - APIs & Services: *OpenRouter*, *Google Maps*, *Stripe*, *Socket.IO*
    - Cloudinary (for image storage)
    - VS Code, Android Studio

- Git/GitHub
- Postman

3. **Security:** Authentication Via JWT token and password hashing

2.6 User Documentation

A user manual will be submitted at project completion, along with a video tutorial to guide mechanics on app usage.

2.7 Assumptions and Dependencies

- Target Android devices meet minimum OS (Android 9+) and hardware requirements.
- The system is dependent on the server and backend services**.** In case of server failure, users may not be able to access the app or their data.
- Reliable internet connectivity is available for real-time features and API calls.
- Third-party services (*Google Maps, OpenRouter, Stripe, Cloudinary, Sockets*) remain functional.
- Assumes users are aware of basic Android/laptop usage and will follow provided manuals and tutorials.

3. External Interface Requirements

3.1 User Interfaces

- The system includes a mobile app for **clients** and **mechanics**, and a web portal for the admin.
- The interface follows a clean and simple design to ensure ease of use for all users.
- Theme colors: Primary: #705794 and Secondary: #EFBE2F
- Common UI components include buttons, input fields, cards, modals, shared navigation and etc.
- Navigation is conditionally rendered based on the user's role (**client**, **mechanic**, or **admin**).
- Error and success messages are displayed using toast notifications.
- Key screens include login, signup, dashboard, bookings, chats, and profile.
- Detailed layouts and visual designs will be provided later

3.2 Hardware Interfaces

- Android smartphones (Android 9+), 2GB+ RAM, GPS-enabled.
- Laptops/desktops (Windows/macOS/Linux).
- Internet required for real-time features.

3.3 Software Interfaces

- **Mobile:** React Native; Web: ReactJS; Backend: Node.js and Express.js
- **Database:** MongoDB.
- **APIs:** *Google Maps, Socket.IO, Stripe, Cloudinary, OpenRouter*.
- **Data format:** JSON

3.4 Communications Interfaces

- **Protocols:** HTTP usage with REST APIs (initially) and WebSocket for real-time updates.
- **Data transfer:** JSON

4. System Features

4.1 User Account Management

4.1.1 Description and Priority

Allows all users (**clients, mechanics, and administrators**) to register, log in, recover passwords, and update their profiles. **Mechanics** require verification before activation; **admins** are created by existing admins. This feature has high priority.

4.1.2 Stimulus/Response Sequences

Use Case Name: Sign Up
Description: User can sign up for an account on the platform.
Actors: Client, Mechanic and Admin
Goal: Create a new user account on the platform.
Pre-Conditions: User must have a valid and unique email address that isn't already registered. Only Admin can signup another Admin.

Basic Course of Events /Main Flow:

1. **User Action (Step1):**
   Starting from onboarding screen user comes on "SignUp" Screen
   System Response:
   The system prompts the user to enter a valid email address.

2. **User Action (Step2):**
   User enters a valid email
   System Response:
   The system sends an OTP on the given email

3. **User Action (Step3):**
   User enters a valid OTP within the given time
   System Response:
   The system will ask for essential details according to the role
   - If the user is a client, he will be asked for fullName, phoneNumber.
   - If the user is a mechanic, he will be asked for fullName, phoneNumber, cnicNumber, profileImage, cnicImages, vehcileType, location and pastWorkImages.
   - If user is an admin, he will be asked for fullName, phoneNumber, cnicNumber and profileImage.

4. **User Action (Step4):**
   User submits the details with valid data
   System Response:
   - The system verifies the provided details and activates the account (for Client and Admin). The client is then redirected to the login page with a success message.
   - For Admin: The email and password will be provided to the other admin.
   - For Mechanic: The details are forwarded to the Admin. The Admin manually verifies the account using *Tasdeeq*(manually) and other manual processes. Based on verification results, the mechanic account is activated.

Alternate Flow (Invalid Inputs): 1. **User Action (Step2):** User enters already registered or invalid email **System Response**: The system says email already registered and incase of wrong format, it will show "Invalid email format", but in case the format is correct but email is wrong OTP will not be sent. 2. **User Action (Step3):** User enters invalid OTP or expired OTP **System Response:** The system says OTP is wrong or expired 3. **User Action (Step4):** User submits the details with invalid data **System Response:** The system will show appropriate error messages such as : • "Please Enter a Valid phoneNumber or cnicNumber" • "Age should be above 25 (in case of mechanic)"
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: User is shown the "Login" Screen

Table 1: Sign Up - Use Case

Use Case Name: Sign In
Description: User can sign in for an account on the platform.
Actors: Client, Mechanic and Admin
Goal: Access the existing account
Pre-Conditions: User must have a valid and activated account.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** User is on "Login" Screen **System Response:** The system asks the user for the valid email and password

2. **User Action (Step2):** User enters valid login credentials and clicks on the Login button. **System Response:** The system verifies the credentials in the database, displays a success message and redirects the user to the dashboard.
Alternate Flow (Invalid Inputs): 1. **User Action (Step2):** User enters invalid login credentials or tries to login inactivated account **System Response:** The system shows an error message "Invalid email or password" or "Account not activated"
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: User is shown the "Dashboard"

Table 2: Sign In – Use Case

Use Case Name: Profile Updation
Description: Allows users to update their personal or professional information in their existing profile.
Actors: Client, Mechanic and Admin
Goal: Update user details to ensure accurate and up-to-date information on the platform.
Pre-Conditions: The user must be logged into the system.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** User navigates to the "Profile" section from the bottom navigation on dashboard **System Response:** The system displays the current profile information. 2. **User Action (Step2):** User updates relevant fields such as fullName, phoneNumber, address, password, profileImage, or any other details. **System Response:** The system validates the entered information (e.g proper format, mandatory fields not empty).

3. **User Action (Step3):** User clicks on the "Save Changes" button. **System Response:** The system updates the information in the database, shows a success message and shows updated profile information
Alternate Flow (Invalid Inputs): 1. **User Action (Step2):** User enters invalid or incomplete data for updation **System Response:** The system will show appropriate error messages such as : • "Please Enter a Valid phoneNumber" • "Age should be above 18 (for Mechanic it's 25)"
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: User can view the updated profile information.

Table 3: Profile Updation – Use Case

Use Case Name: Forgot Password
Description: Allows users to reset their password in case they forget it, using email verification.
Actors: Client, Mechanic
Goal: Help users regain access to their account securely.
Pre-Conditions: The user must have a registered and verified email in the system.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** User clicks on the "Forgot Password" option on the Login screen. **System Response:** The system displays a form asking for the registered email. 2. **User Action (Step2):** User enters the registered email and clicks on **"**Get OTP**."** **System Response:** The system sends OTP on the email

3. **User Action (Step3):** User enters a valid OTP within the given time **System Response:** The system verifies the OTP and asks for new password 4. **User Action (Step4):** User enters a new password and confirms it. **System Response:** The system updates the password in the database, displays a success message and redirects the user to the Login screen.
Alternate Flow (Invalid Inputs): 1. **User Action (Step2):** User enters an unregistered or incorrect email. **System Response:** The system shows an error message, "Email not found." 2. **User Action (Step3):** User enters an invalid or expired OTP **System Response:** The system shows an error message, "Invalid or expired code. Please try again."
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: User successfully resets their password

Table 4: Forgot Password – Use Case

4.1.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	Users shall be able to register themselves; mechanics accounts shall remain pending verification.	High	Done
Req 2	Admins shall be able to verify mechanics using Tasdeeq manually.	High	Done
Req 3	Users shall be able to log in with valid credentials.	High	Done
Req 4	Users shall be able to recover their password with OTP verification.	High	Done

Req 5	Users shall be able to update their profile information.	High	Done
Req 6	The system shall display success and error messages where needed	High	Done

Table 5: User Account Management - Functional Requirements

4.2 Vehicle Management

4.2.1 Description and Priority

Allows **clients** to add, update, or delete their vehicle details that can be reused while making future service requests. This feature has medium priority.

4.2.2 Stimulus/Response Sequences

Use Case Name: Add Vehicle
Description: The client can add a new vehicle to their account.
Actors: Client
Goal: Save vehicle information for future use.
Pre-Conditions: Client must be logged in to the system
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Client is on the dashboard and clicks on "Your Vehicles" and navigate to add a vehicle. **System Response:** The system displays a form asking for details such as make, model, year, engineType, color, numberPlate and etc. 2. **User Action (Step2):** The client submits the form with valid details **System Response:** System stores the vehicle and displays confirmation.
Alternate Flow (Invalid Inputs): 1. **User Action (Step2)** Client submits details with invalid data **System Response**: The system will show appropriate error messages such as: • "Transmission must be manual or automatic"

• "Number plate must be alphanumeric, in a proper format"
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: Vehicle is stored for reuse in future service requests.

Table 6: Add Vehicle – Use Case

Use Case Name: Update Vehicle
Description: The client can modify stored vehicle details.
Actors: Client
Goal: Keep vehicle details accurate and up-to-date.
Pre-Conditions: Vehicle must exist in user's profile.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Client is on the dashboard and clicks on "Your Vehicles" and select the vehicle to update. **System Response:** System displays editable vehicle details. 2. **User Action (Step2):** Client updates the desired information with valid data. **System Response:** System saves updated data and confirms success.
Alternate Flow (Invalid Inputs): 1. **User Action (Step2):** Client updates the desired information with invalid data **System Response**: The system will show appropriate error messages such as: • "Transmission must be manual or automatic" • "Number plate must be alphanumeric, in a proper format"
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"

Post-Condition: Vehicle info updated successfully.

Table 7: Update Vehicle – Use Case

Use Case Name: Delete Vehicle
Description: Allows client to remove a saved vehicle.
Actors: Client
Goal: Delete unwanted or outdated vehicle records.
Pre-Conditions: Vehicle must exist.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Client is on the dashboard and clicks on "Your Vehicles" and select the vehicle to delete. **System Response:** System gives an alert (Are you sure you want to delete this vehicle ?)for the confirmation 2. **User Action (Step2):** Client confirms deletion. **System Response:** System removes vehicle from profile and shows confirmation message.
Alternate Flow (Invalid Inputs): 1. **User Action (Step2):** Client cancels deletion **System Response**: The system will close the alert and make the vehicle preserve.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: Vehicle record deleted successfully.

Table 8: Delete Vehicle – Use Case

4.2.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The client shall be able to add new vehicles to their account.	Medium	Done
Req 2	The client shall be able to update existing	Medium	Done

	vehicle information.		
Req 3	The client shall be able to delete stored vehicles.	Medium	Done
Req 4	The system shall allow the user to select stored vehicles while creating a new service request.	High	Pending

Table 9: Vehicle Management - Functional Requirements

4.3 Service Request Handling

4.3.1 Description and Priority

Allows **clients** to request **mechanics** for assistance, and mechanics to respond in real time. **Mechanics** can offer instant quotes or provide prices after inspection. This feature has high priority.

4.3.2 Stimulus/Response Sequences

Use Case Name: Request for Service
Description: The client can create a new service request on the platform.
Actors: Client
Goal: Create a new service request on the platform.
Pre-Conditions: User must be logged in to the system
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Client is on the dashboard and clicks on “Make Emergency Request” **System Response:** The system displays a form asking for details such as Service Type (Car/Bike**),** Description, and Location. The user can also select their default car details already stored in the system. 2. **User Action (Step2):** The client submits the form with valid details **System Response:** The system validates the input and displays a list of available nearby mechanics within a 1 km to 5 km radius.

Alternate Flow (Invalid Inputs): 1. **User Action (Step2)** Client submits details with invalid data **System Response**: The system will show appropriate error messages such as: • "Please enter a description properly" • "Location not found"
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The client can view nearby available mechanics within the specified range in real time.

Table 10: Request for Service – Use Case

Use Case Name: View Nearby Requests
Description: Mechanics can view nearby service requests in real time.
Actors: Mechanic
Goal: View new nearby service requests on the platform.
Pre-Conditions: User must be logged in and online on the system
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Mechanic is on dashboard and clicks on "View Nearby Service Requests" **System Response:** The system displays all nearby service requests within range, including service details.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The mechanic can view nearby service requests in real time.

Table 11: View Nearby Requests – Use Case

Use Case Name: Offer Instant Quote or Toggle for After-Inspection Prices
Description: Mechanics can respond to service requests by offering an instant quote or opting to provide a price after inspection.
Actors: Mechanic
Goal: To respond to service requests by offering prices.
Pre-Conditions: User must be logged in and on the Nearby requests screen.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** The mechanic offers an instant quote or toggles the "After Inspection" option. **System Response:** • The system shares the quote with the client in real time and waits for the client's response. • The mechanic will be reserved for 5 min till then (like he will be able to see new requests but will not be able to accept any)
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The offered quote is shown to the client while the mechanic waits for acceptance.

Table 12: Offer Instant Quote or Toggle for After-Inspection Prices – Use Case

Use Case Name: View the Quotes
Description: The client can view the quotes offered by nearby mechanics.
Actors: Client
Goal: To view the quotes related to the service request.
Pre-Conditions: The request must not be cancelled.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** User is on Nearby Mechanics screen **System Response:** The system displays quotes with mechanic details in real time (visible for 5 minutes if offered by a mechanic).

Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like “System temporarily unavailable”
Post-Condition: The client can view quotes offered by different mechanics.

Table 13: View the Quotes – Use Case

Use Case Name: Accept the quote
Description: The client can accept a mechanic’s quote, confirming the service.
Actors: Client
Goal: To confirm the mechanic by accepting the quote.
Pre-Conditions: User must not cancel the request and offer must be available
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Client is on nearby mechanic screen and press “Accept” button **System Response:** • The system navigates the user to the Live Tracking screen. • The system sets the status of service request to on_the_way
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like “System temporarily unavailable”
Post-Condition: Client will be able to live track the mechanic

Table 14: Accept the Quote – Use Case

Use Case Name: Reject the quote
Description: The client can reject a mechanic’s quote.
Actors: Client
Goal: To reject a mechanic’s quote and free the mechanic for other requests.
Pre-Conditions: User must not cancel the request and offer must be available
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Client is on nearby mechanic screen and press “Reject” button

System Response: The system frees the mechanic and he might go to other requests available
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system frees the mechanic, allowing them to respond to other available requests.
Post-Condition: The client will no longer see that quote unless the mechanic offers again.

Table 15: Reject the Quote – Use Case

Use Case Name: Cancel Service Request
Description: Allows the client to cancel a service request before it is accepted or completed by the mechanic.
Actors: Client
Goal: To let the client withdraw a pending service request.
Pre-Conditions: The service request must exist and be in a cancellable state (e.g., pending or mechanic accepted but not yet confirmed by client).
Basic Course of Events /Main Flow: 1. **User Action (Step1):** The client is on "Nearby Mechanics" screen **System Response:** The system displays the details of the selected request along with the "Cancel Request" button. 2. **User Action (Step2):** The client clicks the "Cancel Request" button and confirms the cancellation. **System Response:** The system updates the request status to Cancelled, removes it from the active requests list and display a success message
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The service request status is updated to Cancelled.

Table 16: Cancel Service Request – Use Case

4.3.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	Clients shall be able to submit service requests specifying issue and location.	High	Pending
Req 2	The system shall notify nearby mechanics of new service requests.	High	Pending
Req 3	Mechanics shall be able to provide an instant quote or toggle for post-inspection pricing	High	Pending
Req 4	Clients shall be able to confirm or decline a quote.	High	Pending
Req 5	The system shall notify clients and mechanics of booking confirmations, cancellations in real-time.	High	Pending
Req 6	The client shall be able to cancel the request	High	Pending

Table 17: Service Request Handling - Functional Requirements

4.4 Live Tracking

4.4.1 Description and Priority

Allows the **client** to track the **mechanic's** live location in real time after a request has been accepted. This feature uses GPS and map integration to display the **mechanic's** current position and progress until arrival. This is a high-priority feature for transparency, reliability, and better communication between parties.

4.4.2 Stimulus/Response Sequences

Use Case Name: Live Tracking Mechanic
Description: Allows the client to track the mechanic's live location in real time after the mechanic accepts the service request. Tracking remains active until the mechanic marks the job as completed.
Actors: Client
Goal: Enable clients to view the mechanic's location in real time for transparency and convenience.
Pre-Conditions: Client must have accepted the mechanic's quote.

Basic Course of Events /Main Flow: 1. **User Action (Step1):** Client accepts the mechanic's quote. **System Response:** The system activates live tracking and displays the mechanic's location on the client's map interface. 2. **User Action (Step2):** Mechanic starts moving towards the client's location. **System Response:** The system updates the mechanic's position in near real time, showing an estimated distance and direction. 3. **User Action (Step3):** Mechanic updates status to Reached. **System Response:** The system stops live tracking and displays the final status.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: Live tracking stops automatically when the mechanic marks the status as Reached.

Table 18: Live Tracking Mechanic – Use Case

Use Case Name: View Client Location On Map
Description: Allows the mechanic to view the client's exact location on the map once a service request has been accepted. This ensures that the mechanic can navigate accurately to the client's location.
Actors: Mechanic
Goal: Enable the mechanic to locate the client's position on the map for accurate navigation and timely arrival.
Pre-Conditions: Client must have accepted the mechanic quote.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Client accepts the mechanic's quote. **System Response:** The system automatically fetch and displays the client's location on mechanic map interface.

2. **User Action (Step2):**
   The mechanic opens the map view for navigation.
   System Response:
   The system highlights the client's location with a pin and provides an estimated distance and direction.

3. **User Action (Step3):**
   Mechanic reached the client location
   System Response:
   The system automatically stops live tracking.

Exception Flow (System Errors):

1. If the system encounters a database or network error during submission:
   System Response:
   The system shows an error message like "System temporarily unavailable"

Post-Condition: The mechanic can view and navigate to the client's location accurately until he reaches the destination.

Table 19: View Client Location On Map – Use Case

4.4.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The system shall activate live tracking automatically once a mechanic accepts a client's service request.	High	Pending
Req 2	The system shall display the mechanic's current location on the client's map interface in near real time.	High	Pending
Req 3	The system shall display the client's current location on the mechanic's map interface in near real time.	High	Pending
Req 4	The system shall stop location tracking automatically once the mechanic updates the service status to "Reached."	High	Pending
Req 5	The system shall display an appropriate message if the mechanic's GPS or internet connection is disabled.	Medium	Pending

Table 20: Live Tracking - Functional Requirements

4.5 Real Time Chat

4.5.1 Description and Priority

Allows communication between **mechanics** and **clients** within the app. Before a booking is accepted, only the **mechanic** can initiate chat with a **client**; the client can respond but cannot start a conversation. After the booking is accepted (confirmed), both parties can initiate and continue chatting freely. This feature ensures smooth discussion of service details before and after confirmation, improving coordination and reducing misunderstandings. This is a high-priority feature for customer satisfaction and service efficiency.

4.5.2 Stimulus/Response Sequences

Use Case Name: Real-Time Chat
Description: Enables real-time communication between the client and mechanic. Before booking acceptance, only the mechanic can initiate the chat and the client can reply. After booking confirmation, both parties can chat freely until the service is completed or cancelled.
Actors: Client, Mechanic
Goal: To facilitate smooth communication between the mechanic and client before and after service confirmation.
Pre-Conditions: A service request must exist and booking status determines who can initiate chat.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Mechanic opens the chat interface and sends a message before booking confirmation. **System Response:** The client receives the message and can reply but cannot initiate if the mechanic hasn't started the chat. 2. **User Action (Step2):** After booking confirmation, either the mechanic or client sends a message. **System Response:** The message is delivered and displayed in real time with timestamps for both parties. 3. **User Action (Step3):** Both users continue chatting during the active request. **System Response:** Messages are stored in the conversation history and shown chronologically.

4. **User Action (Step4):** When the service request is completed or cancelled. **System Response:** The chat session ends and chats are deleted.
Alternate Flow (Invalid Inputs): 1. **User Action** User tries to send an empty message or when the chat feature is unavailable. **System Response**: The button will remain disabled on screen.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: • Both client and mechanic can view the conversation during the service session. • The chat is deleted once the service is completed or cancelled.

Table 21: Real-Time Chat – Use Case

4.5.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The system shall allow mechanics to initiate chat before booking acceptance; clients may only reply in this phase.	High	Pending
Req 2	The system shall allow both clients and mechanics to initiate chat after booking acceptance.	High	Pending
Req 3	The system shall deliver and display messages in real time with timestamps.	High	Pending
Req 4	The system shall delete the chat history immediately once the booking is completed or cancelled.	Medium	Pending

Table 22: Real-Time Chat - Functional Requirements

4.6 AI Assistance

4.6.1 Description and Priority

Provides **clients** with an AI-powered assistance tool that answers only vehicle-related questions, such as troubleshooting issues, maintenance tips, and repair guidance. The system will reject or redirect non-vehicle-related queries. This feature is a Medium priority for enhancing self-service support.

4.6.2 Stimulus/Response Sequences

Use Case Name: AI Assistance
Description: Provides clients with an AI-powered assistance feature that answers only vehicle-related questions such as troubleshooting, maintenance, and repair guidance. The system automatically rejects or redirects non-vehicle-related queries with predefined messages.
Actors: Client, AI System
Goal: To help clients receive quick, automated assistance for vehicle-related issues.
Pre-Conditions: The client must be logged into the system.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** The client is on Dashboard and presses “Chat with AI” button. **System Response:** The system shows a chat screen 2. **User Action (Step2):** The client asks a vehicle related question **System Response:** The AI processes the query, identifies it as valid, and returns a concise and relevant answer with reference to previous chats.
Alternate Flow (Invalid Inputs): 1. **User Action** The client enters a non-vehicle-related question. **System Response**: The AI responds with a predefined message such as “I can only assist with vehicle-related questions.”
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission:

System Response: The system shows an error message like "System temporarily unavailable"
Post-Condition: The chats will be deleted after a day

Table 23: AI Assistance – Use Case

4.6.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The system shall accept queries from the client and determine if they are vehicle-related.	Medium	Done
Req 2	The system shall return relevant responses for valid vehicle-related questions	Medium	Done
Req 3	The system shall reject non-vehicle queries with a predefined message.	Medium	Done

Table 24: AI Assistance – Functional Requirements

4.7 Reviews and Feedback

4.8.1 Description and Priority

Allows **clients** to provide feedback and ratings for **mechanics** after a service is completed. Feedback helps improve service quality, maintain accountability, and assist future **clients** in selecting **mechanics**. This is a high-priority feature for trust-building and quality assurance.

4.8.2 Stimulus/Response Sequences

Use Case Name: Client Feedback Submission
Description: Allows clients to provide ratings and text feedback for mechanics after a booking is completed or canceled. The feedback contributes to the mechanic's overall rating and helps improve service quality.
Actors: Client, System
Goal: To let clients share their experience after service completion.
Pre-Conditions: The booking status must be either "Completed".

Basic Course of Events /Main Flow: 1. **User Action (Step1):** Mechanic updates the status of service as completed and paid **System Response:** The system displays a "Leave Feedback" option. 2. **User Action (Step2):** The client selects a rating (e.g., 1–5 stars) and optionally enters text feedback. **System Response:** • The system saves the review and associates it with the mechanic's profile. • The system displays a success message: "Thank you for your feedback!"
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The feedback is stored and becomes visible to the mechanic and future clients.

Table 25: Client Feedback Submission – Use Case

Use Case Name: Mechanic View Ratings & Reviews
Description: Allows mechanics to view feedback, ratings, and comments given by clients after completed services. This helps them monitor their performance and identify improvement areas.
Actors: Mechanic, System
Goal: To allow mechanics to see their ratings and feedback from clients.
Pre-Conditions: At least one completed booking should exist with feedback.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** The mechanic navigates to the "My Reviews" section in their dashboard. **System Response:** The system retrieves and displays all reviews, including ratings and client comments.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The mechanic successfully views their accumulated feedback and ratings.

Table 26: View Ratings and Reviews – Use Case

4.8.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The system shall allow clients to submit ratings and text feedback after a booking ends.	Medium-High	Pending
Req 2	The system shall associate each review with the mechanic's profile.	Medium-High	Pending
Req 3	The system shall allow clients to view ratings and recent reviews of a mechanic before booking.	Medium-High	Pending
Req 4	The system shall allow mechanics to view the rating and feedback	Medium-High	Pending

Table 27: Reviews and Feedback – Functional Requirements

4.8 Payment and Financial Management

4.9.1 Description and Priority

Enables secure handling of payments between **clients**, **mechanics**, and the **FixKar** platform using *Stripe* as the payment gateway. Supports both direct online payments to **FixKar** and manual payments to **mechanics**, with automatic or manual deduction of a 10% commission. This feature ensures smooth financial transactions, proper commission management, and transparency for all parties. This is a high-priority feature for revenue handling and trust.

4.9.2 Stimulus/Response Sequences

Use Case Name: Enter Price Before Starting Service (After-Inspection Price)
Description: If the mechanic had selected the "After Inspection" option earlier, they must enter the service price before starting the work.
Actors: Mechanic
Goal: To enter and confirm the final service price before starting.
Pre-Conditions: The mechanic must have reached the client and inspected the vehicle.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** The mechanic updates their status to "Start_the_work." **System Response:** The system prompts the mechanic to enter the service price (if "After Inspection" was

selected). 2. **User Action (Step2):** The mechanic enters the price. **System Response:** The system will save the amount in the database under the request, the amount will be instantly visible to all users.
Alternate Flow: 1. **User Action (Step 2):** The mechanic selects the option that the client is willing to cancel the request. **System Response**: The system saves a fixed amount of Rs. 250 for travel allowance and get it instantly visible to all users.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The final amount of the service will be visible to both client and mechanic.

Table 28: Enter Price Before Starting Service (After-Inspection Price) – Use Case

Use Case Name: Update Service Status
Description: The mechanic can update the current service status (Reached, Working, Completed). The system synchronizes updates in real time on both the mechanic's and client's sides.
Actors: Mechanic
Goal: To keep the client and system updated about the current progress of the service in real time.
Pre-Conditions: The mechanic must have an accepted service request and client must not have cancelled the request.

Basic Course of Events /Main Flow: 1. **User Action (Step1):** The mechanic opens the "Active Request" screen and selects the ongoing service. **System Response:** The system displays the current status (e.g., "Pending", "Reached", "Working", "Completed"). 2. **User Action (Step2):** The mechanic taps "Reached" when they arrive at the client's location. **System Response:** The system updates the status in real time for both the client and the mechanic. If the service type is After-Inspection, the system prompts the mechanic to enter the final service price (handled by the Enter Price Before Starting Service use case). 3. **User Action (Step3):** After starting the repair, the mechanic updates the status to "Working." **System Response:** The client sees "Mechanic is currently working" in real time. 4. **User Action (Step4):** Once the repair is done, the mechanic updates the status to "Completed." **System Response:** The system records service completion and moves the job to the *Payment & Review* module.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: • Both client and mechanic can view the final updated status in real time. • The service request moves into the "Completed" state and becomes ready for billing.

Table 29: Update Service Status – Use Case

Use Case Name: Generate Bill
Description: After the mechanic marks the service as "Completed," the system automatically generates a detailed bill for the client, based on either the agreed instant quote or the after-inspection price.
Actors: Mechanic, Client, Admin
Goal: To create and display a transparent and accurate service bill for both parties.

Pre-Conditions: The service status must be marked as Completed and mechanic must have provided a price (either instant quote or after-inspection).
Basic Course of Events /Main Flow: 1. **System Action (Step1):** Once the mechanic updates status to "Completed," the system automatically prepares the bill details, including: Service Type, Mechanic Name, Client Name, Service Charges (as quoted or after-inspection price), Platform Commission (10%), Total Payable Amount 2. **System Action (Step2):** The system displays the generated bill to both client and mechanic in real time. 3. **User Action (Step3):** The client reviews the bill the make payment.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: Bill is generated and stored in the database.

Table 30: Generate Bill – Use Case

Use Case Name: Make Payment Manually
Description: After receiving the generated bill, the client completes payment manually
Actors: Mechanic, Client, Admin
Goal: To confirm that the service payment has been made successfully.
Pre-Conditions: The bill must be generated
Basic Course of Events /Main Flow: 1. **User Action (Step1):** The client reviews the bill and proceeds to make the payment manually. 2. **User Action (Step2):** The mechanic marks the payment as "Received" in the app. 3. **System Action (Step3):** The system updates the request status as 'Completed & Paid' and adds the app's 10% commission entry in the mechanic's wallet for admin tracking

Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The service request is marked as Completed and Paid**.**

Table 31: Make Payments Manually – Use Case

Use Case Name: Make Payment Online
Description: After receiving the generated bill, the client completes payment while transferring
Actors: Mechanic, Client, Admin
Goal: To confirm that the service payment has been made successfully.
Pre-Conditions: The bill must be generated
Basic Course of Events /Main Flow: 1. **User Action (Step1):** The client reviews the bill and proceeds to make the payment online. 2. **User Action (Step2):** The payment is received to admin, and admin will transfer the mechanic's share to him online and marks the payment as received. 3. **System Action (Step3):** The system updates the request status as 'Completed & Paid'.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: The service request is marked as Completed and Paid**.**

Table 32: Make Payment Online – Use Case

4.9.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The mechanic and admin shall be able to update the service statuses.	High	Pending
Req 2	Mechanics shall be able to enter post-	High	Pending

	inspection price.		
Req 3	The system shall generate the bill for the service.	High	Pending
Req 4	The system shall integrate Stripe to allow clients to make secure online payments directly to FixKar.	High	Pending
Req 5	The admin shall deduct 10% commission from online payments before transferring funds to mechanics.	High	Pending
Req 6	The system shall allow mechanics to manually transfer 10% commission to FixKar in case of direct client payment.	High	Pending
Req 7	The system shall provide payment confirmations to both clients and mechanics.	High	Pending

Table 33: Payment and Financial Management - Functional Requirements

4.9 Admin Management and Analytics

4.10.1 Description and Priority

Provides the **admin** with a centralized web-based dashboard to manage users, bookings, reviews, and payments, as well as monitor platform activity through analytics. This feature ensures operational efficiency, fraud prevention, and performance tracking. It is a high-priority feature as it is critical for platform governance.

4.10.2 Stimulus/Response Sequences

Use Case Name: Admin Management and Analytics
Description: This use case allows the admin to manage users (clients and mechanics), bookings, payments, reviews, and overall platform performance through analytics. The system provides a centralized web dashboard enabling the admin to verify mechanic identities, handle payments, monitor user activity, and maintain service quality by managing reviews.
Actors: Admin, System
Goal: To enable the admin to efficiently manage platform operations, monitor analytics, and ensure quality and trust by managing users, bookings, payments, and reviews.
Pre-Conditions: • The admin must be logged into the web dashboard.

- All relevant data (users, bookings, payments, reviews) must be stored in the system database.

Basic Course of Events /Main Flow:

Scenario 1: User Management

1. **User Action (Step 1):**
   The admin opens the User Management section from the dashboard.
   System Response:
   The system displays a list of all registered users (clients and mechanics) with profile details and account status.

2. **User Action (Step 2):**
   The admin reviews pending mechanic verification requests.
   System Response:
   The admin can approve, reject, or deactivate accounts based on verification results or violations.

Scenario 2: Booking Management

3. **User Action (Step 3):**
   The admin navigates to the Booking Management module.
   System Response:
   The system displays all bookings with filters (status, date, or user).

4. **User Action (Step 4):**
   The admin views or updates booking details, such as changing status or cancelling a booking.
   System Response:
   The system updates booking status and notifies related users accordingly.

Scenario 3: Payment Management

5. **User Action (Step 5):**
   The admin accesses the Payment Management section.
   System Response:
   The system displays transaction records, commission breakdowns, and pending payouts.

6. **User Action (Step 6):**
   The admin reviews and verifies payments or resolves disputes.
   System Response:
   The payment record is updated, and any resolved disputes are logged in history.

Scenario 4: Review Management

7. **User Action (Step 7):**
   The admin opens the Review Management section.

System Response:
The system displays all client-submitted reviews and ratings for mechanics.

8. **User Action (Step 8):**
   The admin identifies negative, spam, or inappropriate reviews.
   System Response:
   The admin can delete such reviews or temporarily deactivate the related user account (client or mechanic) if repeated poor feedback or misconduct is detected.

Scenario 5: Analytics and Reports

9. **User Action (Step 9):**
   The admin opens the Analytics Dashboard.
   System Response:
   The system displays statistics on bookings, user growth, revenue, and mechanic performance.

10. **User Action (Step 10):**
    The admin filters analytics by date range or user type.
    System Response:
    The system updates chart and allows exporting reports in PDF format.

Scenario 6: Notification Management

11. **User Action (Step 11):**
    The admin opens the Notification Management section.
    System Response:
    The system displays options to create, view, and manage notifications for users and mechanics.

12. **User Action (Step 12):**
    The admin composes a new notification message (e.g., announcements, updates, or alerts) and selects the target audience (all users, specific mechanics, or specific clients).
    System Response:
    The system sends the notification to the selected recipients and records it in the notification history for reference.

Exception Flow (System Errors):

1. If the system encounters a database or network error during submission:

   System Response:
   The system shows an error message like “System temporarily unavailable”

Post-Condition: All admin actions (user edits, review deletions, or deactivations) are stored in the system log.

Table 34: Admin Management and Analytics – Use Case

4.10.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The system shall allow the admin to manage user accounts (approve/reject, edit, deactivate).	High	Pending
Req 2	The system shall allow the admin to view and manage bookings.	High	Pending
Req 3	The system shall allow the admin to view and manage payment transactions.	High	Pending
Req 4	The system shall provide analytics on revenue, bookings, and user activity	High	Pending
Req 5	The system shall allow the admin to view and manage reviews.	High	Pending
Req 6	The system shall allow admin to view and manage notifications.	High	Pending

Table 35: Admin Management and Analytics - Functional Requirements

4.10 Multilingual Support & In-App Tutorials

4.11.1 Description and Priority

Provides multilingual support (Urdu and English) for **mechanics** and in-app tutorials on how to use the application. Tutorials will guide users through registration, booking, chat, live tracking, and payment processes. This is an optional feature aimed at improving accessibility and onboarding.

4.11.2 Stimulus/Response Sequences

Use Case Name: Multilingual Support
Description: Allows mechanics to select their preferred app language (Urdu or English) to ensure better understanding and accessibility.
Actors: Mechanic, System
Goal: To provide a localized experience in the mechanic's preferred language.
Pre-Conditions: Language resources (text, labels, etc.) must exist in both Urdu and English.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Mechanic opens the app for the first time.

System Response: Prompts mechanic to select a preferred language. 2. **User Action (Step2):** Mechanic selects a language. **System Response:** App interface switches to the chosen language.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: App displays all content in the selected language. (Except which comes from database)

Table 36: Multilingual Support – Use Case

Use Case Name: In-App Tutorials (OPT)
Description: Provides mechanics with visual or video-based tutorials to guide them through app features like registration, booking, live tracking, chat, and payment.
Actors: Mechanic, System
Goal: To educate mechanics on app usage for smooth onboarding and self-help.
Pre-Conditions: Tutorials must be preloaded or accessible via internet connection.
Basic Course of Events /Main Flow: 1. **User Action (Step1):** Mechanic opens the "Tutorials" section from the app menu. **System Response:** Displays a list of tutorial topics (registration, booking, etc.). 2. **User Action (Step2):** Mechanic selects a topic. **System Response:** Shows a short visual guide or video explaining the process.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: Mechanic successfully learns how to use key app features.

Table 37: In-App Tutorials (OPT) – Use Case

4.11.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The system may allow mechanics to choose between Urdu and English language interfaces.	Medium-High	Pending
Req 2	The system may provide in-app tutorials for registration, booking, chat, live tracking, and payments.	Medium-High	Pending

Table 38: Multilingual Support & In-App Tutorials - Functional Requirements

4.11 Notifications

4.12.1 Description and Priority

Real-time notifications for **clients** and **mechanics** will be handled through the app across all relevant interactions. Notifications ensure timely communication and improved coordination for service requests, bookings, chat updates, and status changes. This is a high-priority feature for user engagement and operational efficiency.

4.12.2 Stimulus/Response Sequences

Use Case Name: Real-Time Notification System
Description: This use case describes how the system sends real-time notifications to clients and mechanics during different stages of the service lifecycle, including new service requests, mechanic responses, client confirmations, chat messages, status updates, completions, and cancellations.
Actors: Client, Mechanic, Admin, System (Notification Service)
Goal: To ensure that both clients and mechanics receive instant updates and alerts relevant to their service interactions. (in all three modes foreground, background, kill)
Pre-Conditions: The system must have access to an active internet connection and notification permissions.
Basic Course of Events /Main Flow: **Scenario 1: Nearby Service Request Notification** 1. **User Action (Step1):** The client submits a new service request with issue details and location. **System Response:** The system identifies mechanics within the defined radius (e.g., 1 km) and sends a real-time notification to available mechanics about the new request.

Scenario 2: Mechanic Acceptance Notification

2. **User Action (Step2):**
   A mechanic accepts the client's service request.
   System Response:
   The system sends a real-time notification to the client containing mechanic details, quoted price, and current request status.

Scenario 3: Client Acceptance Notification

3. **User Action (Step3):**
   The client accepts the mechanic's offer or quote.
   System Response:
   The system notifies the mechanic that the booking has been confirmed and displays client contact details if applicable.

Scenario 4: Chat Notifications

4. **User Action (Step4):**
   Either user sends a message through the in-app chat.
   System Response:
   The recipient receives a real-time notification containing a message preview and timestamp.

Scenario 5: Status Update Notifications

5. **User Action (Step 5):**
   The mechanic updates the job status (e.g., "Started", "On Arrival", "Reached").
   System Response:
   The client receives a notification reflecting the updated status.

Scenario 6: Service Completion Notification

6. **User Action (Step 6):**
   The mechanic marks the service as "Completed."
   System Response:
   The system sends a notification to both parties confirming service completion and prompting the client to leave a review.

Scenario 7: Cancellation Notification

7. **User Action (Step 7):**
   Either the client or mechanic cancels a booking.
   System Response:
   The system sends a cancellation notification to both parties with cancellation reason and booking details.

Scenario 8: General Notification 8. **User Action (Step 8):** Any system event occurs that requires user awareness, such as admin announcements, new offers, or reminders. **System Response:** The system sends a general notification to the relevant users (client, mechanic, or admin) containing the event details.
Exception Flow (System Errors): 1. If the system encounters a database or network error during submission: **System Response:** The system shows an error message like "System temporarily unavailable"
Post-Condition: Notifications are delivered to relevant users in real time.

Table 39: Real-Time Notification System – Use Case

4.12.3 Functional Requirements

Req.ID	Description	Priority	Status
Req 1	The system shall send real-time notifications to mechanics within a specified radius when a new service request is submitted.	High	Pending
Req 2	The system shall notify the client when a mechanic accepts or responds to a service request.	High	Pending
Req 3	The system shall notify the mechanic when a client confirms a booking.	High	Pending
Req 4	The system shall send chat notifications in real time with message previews and timestamps.	High	Pending
Req 5	The system shall notify the client when the mechanic updates job status (Started, On Arrival, Reached).	High	Pending
Req 6	The system shall notify both parties upon job completion or cancellation.	High	Pending
Req 7	The system shall send general notifications to the relevant users	High	Pending

Table 40: Notification - Functional Requirements

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system shall load the home screen within 3 seconds under normal network conditions.
- Nearby **mechanics** search results shall be displayed within 5 seconds after location retrieval.
- The system shall handle up to 500 concurrent user requests without performance degradation.
- Real-time location updates for mechanics shall be refreshed every 5 seconds.

5.2 Safety Requirements

- The app shall provide safety instructions for roadside assistance scenarios.
- No **mechanic** location or client personal details shall be shared without consent.
- Emergency contact information shall be visible to both **client** and **mechanic** during an active booking.
- The system shall prevent booking acceptance by **mechanics** without verified identity via *Tasdeeq Verification*.

5.3 Security Requirements

- All communication between **client**, **mechanic**, and server will use HTTP protocol.
- User authentication shall be implemented using JWT tokens support.
- Payment transactions shall be processed securely using *Stripe PCI-DSS* compliant APIs.
- User passwords shall be stored using salted hashing.
- Access control shall restrict admin functionalities to authorized admin accounts only.

5.4 Software Quality Attributes

- **Availability**: The system shall maintain 99% uptime excluding planned maintenance.
- **Usability**: User interfaces shall follow standard mobile design principles for easy navigation and operation.
- **Reliability**: The system shall recover automatically from server crashes within 1 minute.

- **Maintainability**: The system shall be developed with modular architecture to allow updates without downtime.
- **Portability**: The app shall run on Android devices.
- **Testability**: All modules shall be unit-tested with a minimum **80% code coverage**.

5.5 Business Rules

- Only verified **mechanics** can receive and accept booking requests.
- A 10% commission from each completed job shall be transferred to the **FixKar** account (via automatic deduction or manual transfer).
- Feedback and reviews can only be submitted by **clients** after the booking is marked as completed.
- Cancellations by **mechanics** after job acceptance shall result in penalties as per **FixKar** policy.
- If a **client** cancels after the **mechanic's** on-site inspection, a fixed inspection fee shall still be paid to the **mechanic.**
- AI Assistance will only respond to vehicle-related queries from **clients.**

6. Other Requirements

6.1 Appendix A: Glossary

Term / Acronym	Definition
FixKar	The proposed system providing real-time roadside assistance, on-demand mechanic booking, and AI-based vehicle help.
Tasdeeq Verification	A third-party verification service for CNIC-based identity verification of mechanics.
Quote	The estimated service price provided by a mechanic either instantly or after inspecting the vehicle.
OpenRouter API	A third-party API used to power AI vehicle assistance in the FixKar mobile app.
Cloudinary	A cloud-based service used for secure image and media storage in the system.
OTP (One-Time Password)	A temporary code sent to the user's email or phone for password recovery and account verification.
Multilingual	A feature allowing users (especially mechanics) to select between Urdu and English for the app interface.

Real Time	Real-time refers to the immediate processing and response of data or events as they occur, with little to no delay between input and system output.
Vehicle Restricted AI	Vehicle-Restricted AI refers to an artificial intelligence system designed exclusively for vehicle-related tasks or contexts, with its functionality limited to automotive operations.
APK	APK stands for Android Package Kit. It is the file format used by the Android operating system for distributing and installing mobile applications.
JSON	JSON is a lightweight data-interchange format used to store and transmit data between a server and a client.
PCI DSS	To protect cardholder data and reduce credit card fraud through better control over data storage, processing, and transmission.
Inspection	When the mechanic checks the vehicle to diagnose the problem and estimate the repair cost before starting the service.
GPS	It's a satellite-based navigation system that lets devices determine their exact location (latitude, longitude, and altitude) anywhere on Earth.

Table 41: Glossary – Fixkar

6.2 Appendix C: To Be Determined List

- Final decision on backend deployment platform is to be determined.
- Clarification of in-house service flow, including scheduling, mechanic assignment, and booking time slots is to be determined.
- The payment flow final make is to be determined

Software Design Specification

1. Introduction

1.1 Purpose of this document

The purpose of this document is to provide a detailed specification of the software design for the **FixKar** system. It serves as a blueprint for developers, testers, and future maintainers to understand the structure, components, and interactions within the system. This document ensures that all stakeholders have a common understanding of the system design and facilitates consistent implementation in alignment with the requirements defined in the SRS.

1.2 Scope of the development project

FixKar is designed to bridge the gap between stranded vehicle owners and verified **mechanics** by providing real-time roadside assistance within a 1-5 km radius.
Key goals include:

- Immediate access to verified mechanics in emergency situations via real-time location tracking.
- Transparent service process with instant or post-inspection price quotes.
- In-app communication between **client** and **mechanic.**
- Secure payment system for commission distribution.
- AI-powered vehicle troubleshooting for common issues.
- Ratings and reviews from users after service completion.
- Comprehensive **admin** control over user management, **mechanic** verification, analytics, and policy enforcement.

1.3 Definitions, acronyms, and abbreviations

Term / Acronym	Definition
CRUD	Create, Read, Update, Delete operations.
UI	User Interface
UX	User Experience
API	Application Programming Interface, used for integrating external services.
MERN	Technology stack (MongoDB, Express.js, React/React Native, Node.js).

Table 42: Definitions - SDS

1.4 Overview of document

This document provides a comprehensive specification of the software design for FixKar. It is organized as follows:

1. **Introduction**: Purpose, scope, definitions, references, and document overview.
2. **System Architecture Description**: High-level design, data structures, program structure, and alternatives considered.
3. **Detailed Description of Components**: Module-level descriptions, interfaces, and responsibilities.
4. **User Interface Design**: Design rules, GUI components, and detailed UI description.
5. **Reuse and Relationship to Other Products**: How existing frameworks, APIs, and components were reused or adapted.
6. **Design Decisions and Tradeoffs**: Key design choices, tradeoffs, and abandoned ideas.

2. System architecture description

2.1 Section Overview

This section describes the overall architecture of the **FixKar**, including design rationale, data structures, program structure, and alternatives considered. It provides a high-level understanding of how system components interact and the constraints affecting design decisions.

2.2 General Constraints

- **Hardware Requirements:** The mobile app must run on Android devices (9+); minimum RAM 2GB.
- Software Requirements: The system is built using React Native (Mobile), ReactJS (Admin), Node.js backend; for database its MongoDB cloud.
- **Network Requirements:** Internet connectivity is required.
- **Performance Requirements:** Requests must respond within 3 seconds.
- **Interface Requirements:** The system must integrate seamlessly with external APIs and maintain a responsive app and web design.
- **Security Requirements:** The system must enforce strong security measures, such as secure user authentication using JWT.

2.3 Data Design

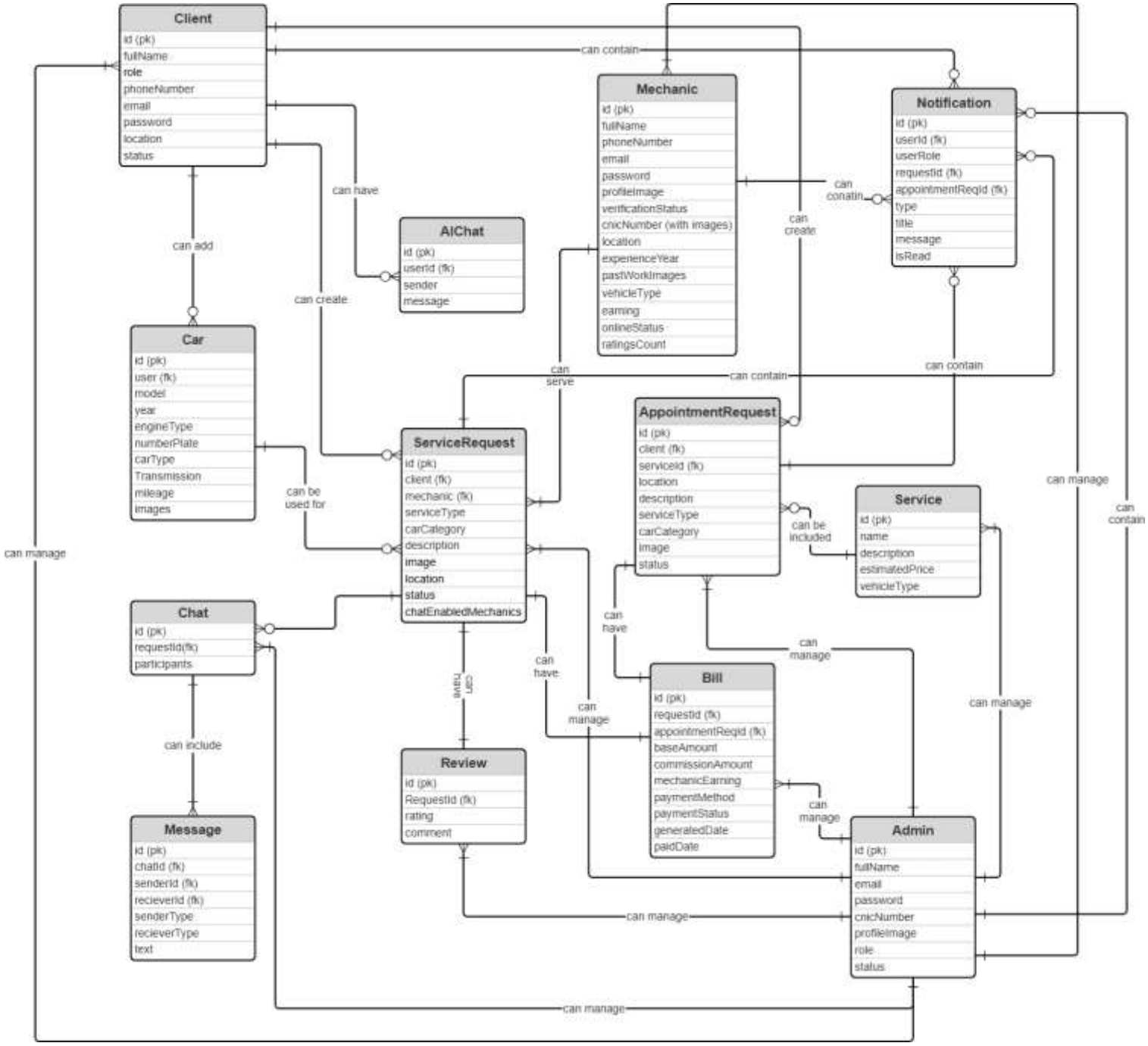


Figure 6: ERD Diagram

2.4 Program Structure

1. **User Module:**
   - **Signup / Login:**
     - Users can register as Client or Mechanics via OTP verification.
     - Login functionality with role-based access control.
   - **Account Settings**
     - Users can view profile, update profile, change password, and reset password.

2. **Vehicle Module:**
   - **CRUD Cars:**
     - Clients can view, add, update and delete their cars for faster access later.

3. **AI Module:**
   - **AI Assistance:**
     - Client can chat with AI and take suggestion related to vehicle related questions only.

4. **Service Request Module:**
   - **Request Service:**
     - Clients can request for quick service.
     - Clients can view nearby Mechanics
     - Clients can cancel the request
   - **View Requests:**
     - Mechanics can View nearby Requests
     - Mechanics can accept nearby Request, one at a time

5. **Chat Module:**
   - **Real time Chat:**
     - Mechanics can initiate the chat before and after the request is accepted, but client can only reply until request is accepted by mechanic.
     - Both shall be able to communicate in real time.

6. **Live Tracking Module:**
   - **Track Mechanic:**
     - Client can track Mechanic's location in real time.
     - System shows distance and estimated arrival time using Google Maps.

- **View Location:**
  - Mechanic can view the location and directions while navigating to Google Map.

7. **Payment Module:**
   - **Revenue Management:**
     - System calculates service charges and adds the commission to mechanic's wallet automatically, either physically or using Stripe**.**

8. **Review Module:**
   - **Drop a Review:**
     - Client can drop a review or rating after the service is completed.

9. **Notification Module:**
   - **Push Notifications:**
     - All users shall receive notifications about acceptance, requests, updates, cancellations and messages.

10. **Multilingual Support Module:**
    - **Toggle language:**
      - Mechanics can change the language of all content between Urdu and English.

11. **Admin Module:**
    - **Management of all modules and View Analytics:**
      - Admin will be able to manage all modules mentioned above and view the analytics and graphs annually or monthly.

2.5 Alternatives Considered

Several architectural approaches were considered for the FixKar system:

1. **Monolithic Architecture:**
   - **Layers:** All Frontend and Backend in single project.
   - **Pros:** Simple setup
   - **Cons:** Hard to maintain, less scalable, difficult for multiple developers.

2. **Layered Architecture (Chosen):**
   - **Layers:** Presentation (UI), Business Logic, Data Access, External Services (APIs).

- **Pros:** Clear separation of concerns, easier maintenance, parallel development, simplifies external service integration.
- **Cons:** Slightly more complex than monolithic.

3. **Microservices Architecture:**
   - **Layers:** Each module as an independent service.
   - **Pros:** Highly scalable.
   - **Cons:** Overkill for this project, complex to deploy and maintain.

The Layered Architecture was chosen because it provides a well-organized, maintainable, and scalable structure, supports parallel development, and allows easy integration with external services, while keeping the system manageable for the FYP timeline.

3. Detailed description of Components

3.1 Section Overview

This section describes the major software components of the FixKar system. Each component is represented as a class in the class diagram and is responsible for handling a specific functionality such as user management, service requests, payments, communication, and administration.

3.2 Component in Detail

3.2.1 Client

Identification	The unique name for the component is "**Client**", located in the system's user interface layer for client account management and service requests.
Type	**Module:** Handles client account creation, authentication, vehicle management, and service request functionality.
Purpose	Enables clients to register for an account, manage their profile, add/update vehicles, request mechanic services, track service requests, and communicate via chat.
Function	**Handles:** • User registration with email validation and OTP verification. • Login/logout authentication with password security. • Profile management (name, email, phone, address). • Vehicle management (add, update, delete multiple vehicles).

	• Service request creation with vehicle and location details. • Real-time chat communication with mechanics and AI assistant. **Processes:** • Input validation for all mandatory fields. • Password encryption and secure authentication. • Vehicle information storage and retrieval. • Service request submission and status tracking. **Outputs:** • Error messages for invalid credentials, duplicate emails, or incomplete data. • Success messages for registration, login, vehicle updates, and service requests. • Service request confirmation and tracking information.
Subordinates	**Consists of subcomponents:** • **Registration Handler:** Manages new client account creation with email verification. • **Authentication Manager:** Handles login/logout and password reset functionality. • **Profile Manager:** Updates client information and manages account details. • **Vehicle Manager:** Adds, updates, and deletes client vehicles. • **Service Request Handler:** Creates and tracks service requests. • **Chat Interface:** Provides real-time messaging with mechanics and AI.
Dependencies	**Relies on:** • Email service for OTP verification. • Database for storing client information and vehicle data. • AI_Chat component for intelligent assistance. • ServiceRequest component for creating service requests. • Chat and Message components for communication. • Car component for vehicle management.

Interfaces	**External Interfaces:** • REST API endpoints for registration, login, profile updates, vehicle management, and service requests. • WebSocket connection for real-time chat. **Internal Interfaces:** • register(): Creates new client account. • login(): Authenticates client credentials. • forgotPassword(): Initiates password reset. • changePassword(): Updates client password. • updateProfile(): Modifies client information. • deleteClient(): Removes client account. • getAllClient(): Retrieves client list (admin function). • getClientById(): Fetches specific client details. All error messages and codes are defined in the system specification. Screen formats and user interface components are documented separately.
Resources	• Database tables for client accounts and session management. • Email service API for verification. • File storage for profile images. • Authentication tokens and session storage. • WebSocket connections for real-time features.
Processing	Functions handle client registration with email verification, secure authentication with password hashing, profile and vehicle CRUD operations, service request creation and tracking, and real-time chat communication. Validation ensures data integrity across all operations.
Data	Stores client ID (primary key), full name, role (client), email, password (encrypted), phone number, address, profile image, and status. Maintains relationships with vehicles, service requests, and chat messages. Data is normalized and stored across related tables.

Table 43: Component - Client

3.2.2 Mechanic

Identification	The unique name for the component is "**Mechanic**", located in the system's service provider layer for mechanic account management and service delivery.
Type	**Module:** Handles mechanic account creation, profile management, service

	acceptance and client communication.
Purpose	Enables mechanics to register, manage their profile with verification details, view and accept service requests, track earnings, update location, and communicate with clients.
Function	**Handles:** • Mechanic registration with verification requirements (CNIC, profile image). • Profile management including past work images and vehicle type specialization. • Service request viewing and acceptance. • Location tracking and updates. • Earnings tracking and online status management. **Processes:** • Verification of mechanic credentials (CNIC, profile, past work). • Validation of specialization and service capabilities. • Real-time location updates for service dispatch. • Earning calculations and payment tracking. Password encryption and secure authentication. **Outputs:** • Error messages for incomplete verification or invalid data. • Success messages for profile updates, service acceptance, and appointment confirmations. • Notification of new service requests.
Subordinates	**Consists of subcomponents:** • **Verification Manager:** Handles CNIC and credential verification. • **Profile Manager:** Manages mechanic information and specializations. • **Service Handler:** Processes service request acceptance and completion. • **Location Tracker:** Updates and manages real-time mechanic location. • **Earnings Calculator:** Tracks income and payment history.
Dependencies	**Relies on:** • Database for storing mechanic profiles and verification data.

	• ServiceRequest component for viewing and accepting requests. • Notification component for alerts. • File storage for CNIC images, profile images, and past work images. • Location services for GPS tracking.
Interfaces	**External Interfaces:** • REST API for registration, profile management, service operations. • GPS/location API for real-time tracking. • WebSocket for instant notification delivery. WebSocket connection for real-time chat. **Internal Interfaces:** • `register()`: Creates mechanic account with verification. • `login()`: Authenticates mechanic credentials. • `forgotPassword()`: Password recovery process. • `changePassword()`: Updates mechanic password. • `getAllMechanics()`: Retrieves all mechanics (admin). • `getMechanicById()`: Fetches specific mechanic details. • `updateMechanic()`: Modifies mechanic profile. • `deleteMechanic()`: Removes mechanic account. • `getAllUnverifiedMechanics()`: Lists pending verifications (admin). • `updateMechanicLocation()`: Updates GPS coordinates.login(): Authenticates client credentials. Error codes and validation messages are standardized across the system.
Resources	• Database tables for mechanic profiles, verifications, and earnings. • File storage for images (CNIC, profile, past work). • Location tracking services and GPS APIs. • Payment processing system integration. • Real-time notification infrastructure.
Processing	Manages mechanic registration with multi-step verification including CNIC upload and validation. Processes service request acceptance with location-based matching. Calculates earnings based on completed services. Maintains real-time availability and location status for optimal service dispatch.

Data	Stores mechanic ID (primary key), full name, phone number, email, password (encrypted), CNIC number with images, profile image, verification status (boolean), location (GPS coordinates), past work images, vehicle type specialization, earning amount, online status, and rating. Maintains relationships with service requests and reviews.

Table 44: Component - Mechanic

3.2.3 AI_Chat

Identification	The unique name for the component is "**AI_Chat**", located in the system's intelligent assistance layer providing automated support to users.
Type	**Module:** Handles AI-powered chat interactions, natural language processing, and automated assistance for users.
Purpose	Provides intelligent conversational assistance to users, answers queries about services, helps with navigation, offers recommendations, and escalates to human support when necessary.
Function	**Handles:** • Natural language understanding of user queries. • Context-aware responses based on conversation history. • Service information and pricing queries. **Processes:** • Message analysis using NLP algorithms. • Response generation based on context. • Escalation triggers for complex queries. **Outputs:** • Conversational responses to user queries. • Service recommendations and information. • Error handling messages for unsupported queries. • Escalation notifications to human support.
Subordinates	**Consists of subcomponents:** • **Intent Classifier:** Determines user intent from messages. • **Response Generator:** Creates contextually appropriate responses. • **Context Manager:** Maintains conversation history and state. • **Escalation Handler:** Routes complex queries to human support.

Dependencies	**Relies on:** • GPT-4o key. • Database for conversation history and user context. • Knowledge base for service information. • User authentication for personalized responses.
Interfaces	**External Interfaces:** • GPT-4o API for real-time chat communication. • REST API for retrieving service and user information. **Internal Interfaces:** • `askAI()`: Processes user query and generates response. • `getUserId()`: Retrieves associated user for personalized responses. GPT model endpoints and training data formats are documented separately. All conversation flows and response templates are maintained in the knowledge base.
Resources	• GPT model servers for processing. • Knowledge base database with service information. • Conversation history storage. • API rate limiting and queue management. • Analytics infrastructure for performance monitoring.
Processing	Receives user messages, sent message to GPT model, retrieves relevant information from knowledge base or system components, generates contextually appropriate response considering conversation history, and delivers response while logging interaction for continuous improvement. Escalates to human support for complex or sensitive queries.
Data	Stores chat ID (primary key), associated user ID (foreign key), sender type (user/AI), and message content. Maintains conversation context including previous messages, identified intents, extracted entities, and user preferences for personalized interactions.

Table 45: Component - AI_Chat

3.2.4 Service Request

Identification	The unique name for the component is "**ServiceRequest**", located in the system's core business logic layer managing service request lifecycle.
Type	**Module:** Handles creation, management, tracking, and completion of service requests between clients and mechanics.

Purpose	Manages the complete service request workflow from creation by clients, assignment to mechanics, status tracking, completion, and billing. Central component coordinating client needs with mechanic availability.
Function	**Handles:** • Service request creation with vehicle and location details. • Request assignment to available mechanics based on location and specialization. • Status tracking through request lifecycle (pending, accepted, in-progress, completed). • Vehicle information management for each request. • Request modification and cancellation. **Processes:** • Validation of request details and requirements. • Mechanic matching algorithm based on proximity and expertise. • Status workflow management and notifications. • Request history and audit trail maintenance. **Outputs:** • Confirmation messages for request creation and updates. • Status notifications to clients and mechanics. • Error messages for invalid requests or unavailable services. • Request summaries and details for tracking.
Subordinates	**Consists of subcomponents:** • **Request Creator:** Handles new request submission and validation. • **Mechanic Matcher:** Assigns appropriate mechanics to requests. • **Status Manager:** Tracks and updates request status. • **Request Validator:** Ensures data integrity and completeness. • **History Tracker:** Maintains audit trail of all changes.
Dependencies	**Relies on:** • Client component for request origination. • Mechanic component for service fulfillment. • Car component for vehicle information. • Location services for proximity matching. • Notification component for status updates.

	• Bill component for payment processing. • Review component for service feedback.
Interfaces	**External Interfaces:** • REST API for request CRUD operations. • WebSocket for real-time status updates. • Location API for mechanic proximity calculations. **Internal Interfaces:** • `quickRequest()`: Creates urgent service request. • `getNearByRequest()`: Finds requests near mechanic location. • `getClientActiveStatus()`: Checks client's active requests. • `getClientActiveRequest()`: Retrieves current client requests. • `getMechanicActiveRequest()`: Retrieves assigned mechanic requests. Status codes and workflow states are defined in the system specification. API rate limits and validation rules are documented in the technical architecture.
Resources	• Database tables for requests, status history, and assignments. • Location calculation services and mapping APIs. • Message queue for asynchronous notifications. • File storage for vehicle images and documentation. • Caching layer for active request lookups.
Processing	Processes service request creation with validation of all required fields, executes mechanic matching algorithm considering location, vehicle type expertise, availability, and ratings, manages status transitions with proper authorization checks, maintains complete audit trail of all changes, coordinates with related components for billing, and reviews, and ensures data consistency across related entities.
Data	Stores request ID (primary key), client ID (foreign key), mechanic ID (foreign key when assigned), vehicle number, engine type, number plate, vehicle details (object), vehicle images, location (GPS coordinates), and status (pending/accepted/in-progress/completed/cancelled). Maintains relationships with Client, Mechanic, Car Bill, and Review components.

Table 46: Component – ServiveRequest

3.2.5 Car

Identification	The unique name for the component is "**Car**", located in the system's data management layer for vehicle information storage and retrieval.
Type	**Module:** Handles vehicle registration, information management, and association with client accounts.
Purpose	Manages comprehensive vehicle information for clients including registration details, specifications, service history, and maintenance records. Supports multiple vehicles per client account.
Function	**Handles:** • Vehicle registration with complete specifications. • Storage of vehicle details (model, year, engine, transmission). • Vehicle image management. • Service history tracking per vehicle. • Multiple vehicle management per client. **Processes:** • Validation of vehicle registration numbers and details. • Duplicate vehicle detection. • Vehicle information updates and modifications. • Association with service requests and history. **Outputs:** • Confirmation messages for vehicle addition and updates. • Error messages for invalid data or duplicate registrations. • Vehicle information retrieval for service requests. • Service history reports per vehicle.
Subordinates	**Consists of subcomponents:** • **Vehicle Registry:** Manages vehicle registration and unique identification. • **Specification Manager:** Handles vehicle technical details and specifications. • **Image Manager:** Stores and retrieves vehicle images. • **History Tracker:** Maintains service history per vehicle. • **Validation Engine:** Ensures data accuracy and completeness.
Dependencies	**Relies on:** • Database for vehicle information storage. • File storage for vehicle images.

	• Client component for ownership association. • ServiceRequest component for service history. • External vehicle registration validation services (if applicable).
Interfaces	**External Interfaces:** • REST API for vehicle CRUD operations. • File upload API for vehicle images. **Internal Interfaces:** • `quickaddCar()`: Registers new vehicle for client. • `getCarsByUser()`: Retrieves all vehicles for a client. • `getSingleCar()`: Fetches specific vehicle details. • `updateCar()`: Modifies vehicle information. • `deleteCar()`: Removes vehicle from system. Data validation rules and field constraints are defined in the database schema. Vehicle specification standards are documented in the data dictionary.
Resources	• Database tables for vehicle information and specifications. • File storage system for vehicle images. • Vehicle registration validation services. • Data backup and recovery systems. • Search and indexing infrastructure for quick lookups.
Processing	Processes vehicle registration with validation of required fields, checks for duplicate registrations based on number plate and registration number, stores vehicle images with optimization and compression, maintains referential integrity with client accounts, tracks complete service history linked to each vehicle, supports efficient queries for service request matching, and ensures data consistency across related components.
Data	Stores car ID (primary key), user/client ID (foreign key), vehicle model, manufacturing year, number plate, registration number, engine type, transmission type, mileage, vehicle images, and status. Maintains one-to-many relationship with Client (one client can have multiple vehicles) and relationships with ServiceRequest for service history. Data is normalized to avoid redundancy.

Table 47: Component – Car

3.2.6 Chat

Identification	The unique name for the component is "**Chat**", located in the system's communication layer managing conversation sessions between users.
Type	**Module:** Handles chat session creation, management, and coordination between clients and mechanics.
Purpose	Manages chat sessions including conversation initialization, participant management, message routing, and conversation history. Supports real-time communication between clients and mechanics.
Function	**Handles:** • Chat session creation between clients and mechanics. • Participant management and permissions. • Message routing to appropriate recipients. • Conversation history and archiving. **Processes:** • Session initialization with proper authorization. • Real-time message delivery and acknowledgment. • Conversation state management. • Archive and retrieval of historical chats. **Outputs:** • Success messages for chat creation and updates. • Error messages for unauthorized access or invalid sessions. • Delivery and read receipts for messages. • Chat history and conversation logs.
Subordinates	**Consists of subcomponents:** • **Session Manager:** Creates and manages chat sessions. • **Message Router:** Directs messages to correct recipients. • **Participant Handler:** Manages chat participants and permissions. • **History Manager:** Stores and retrieves conversation history. • **Notification Dispatcher:** Sends real-time message notifications.
Dependencies	**Relies on:** • WebSocket infrastructure for real-time communication. • Message component for individual message handling. • Client and Mechanic components for participant information. • Database for conversation persistence. • Notification component for message alerts.

Interfaces	**External Interfaces:** • REST WebSocket API for real-time bidirectional communication. • REST API for chat history retrieval and management. **Internal Interfaces:** • `createChat()`: Initiates new conversation session. • `deleteChat()`: Archives or removes conversation. WebSocket event schemas and message formats are documented in the API specification. Connection handling and reconnection logic are detailed in the technical documentation.
Resources	• WebSocket server infrastructure for real-time connections. • Database for chat sessions and conversation history. • Message queue for reliable message delivery.
Processing	Creates chat sessions with proper participant authorization, establishes WebSocket connections for real-time communication, routes messages to appropriate recipients with delivery confirmation, maintains conversation context and participant state, stores complete message history for retrieval, handles connection failures with automatic reconnection, and ensures message ordering and consistency across participants.
Data	Stores chat ID (primary key), request ID (foreign key linking to service context), and participants (object containing client ID, mechanic ID, and AI participation status). Maintains one-to-many relationship with Message component. Tracks conversation metadata including creation time, last activity, and participant status.

Table 48: Component - Chat

3.2.7 Message

Identification	The unique name for the component is "**Message**", located in the system's communication layer handling individual message transmission and storage.
Type	**Module:** Handles individual message creation, delivery, storage, and retrieval within chat conversations.
Purpose	Manages individual messages including text content, image attachments, metadata, delivery status, and read receipts. Supports rich messaging features within client-mechanic.
Function	**Handles:** • Message creation with text and image support.

	• Message delivery with status tracking. • Read receipt management. • Message history retrieval. • Sender and receiver identification. **Processes:** • Message validation and sanitization. • Image attachment processing and storage. • Delivery confirmation and read tracking. • Message ordering and timestamp management. **Outputs:** • Delivery confirmations and read receipts. • Error messages for failed delivery or invalid content. • Message history ordered by timestamp. • Notification triggers for new messages.
Subordinates	**Consists of subcomponents:** • **Message Creator:** Handles message composition and validation. • **Delivery Manager:** Ensures reliable message delivery. • **Attachment Handler:** Processes and stores image attachments. • **Receipt Tracker:** Manages delivery and read receipts. • **History Manager:** Retrieves and orders message history.
Dependencies	**Relies on:** • Chat component for conversation context. • Database for message persistence. • File storage for image attachments. • WebSocket infrastructure for real-time delivery. • Notification component for message alerts.
Interfaces	**External Interfaces:** • REST API for message retrieval and management. • WebSocket events for real-time message delivery. • File upload API for image attachments. **Internal Interfaces:** • `getMessages()`: Retrieves conversation messages. • `sendMessage()`: Creates and delivers new message. • `deleteMessage()`: Removes message (with permissions). Message size limits, supported image formats, and content policies are

	defined in the system specification. Rate limiting and anti-spam measures are documented in the security guidelines.
Resources	• Database for message storage and indexing. • File storage system for image attachments. • WebSocket connections for real-time delivery. • Message queue for reliable asynchronous processing.
Processing	Processes message creation with content validation and sanitization, handles image attachments with compression and storage, delivers messages via WebSocket with retry logic for failures, tracks delivery status and read receipts, maintains chronological ordering with accurate timestamps, supports message retrieval with pagination, applies content moderation for safety, and ensures message integrity and persistence.
Data	Stores message ID (primary key), chat ID (foreign key), sender ID (foreign key), receiver ID (foreign key), sender type (client/mechanic/AI), receiver type (client/mechanic/AI), message text content, image attachments (URLs/paths), timestamp (creation time), and images (array of image references). Maintains many-to-one relationship with Chat component and links to user components for sender/receiver details.

Table 49: Component - Message

3.2.8 Review

Identification	The unique name for the component is "**Review**", located in the system's quality assurance layer managing service feedback and ratings.
Type	**Module:** Handles review creation, rating management, feedback collection, and quality metrics for mechanics and services.
Purpose	Enables clients to provide feedback on completed services, rate mechanics, leave comments, and contribute to quality metrics. Supports mechanic reputation system and service improvement initiatives.
Function	**Handles:** • Review submission after service completion. • Numeric rating (1-5 scale) validation and storage. • Written feedback collection and moderation. • Review editing and deletion (within timeframe). • Rating aggregation for mechanic profiles. **Processes:** • Validation that review is for completed service only. • Rating calculation and mechanic score updates.

	• Review verification (ensuring actual service completion). **Outputs:** • Success confirmations for review submission. • Error messages for invalid reviews or unauthorized attempts. • Aggregated rating displays on mechanic profiles. • Review summaries and feedback reports.
Subordinates	**Consists of subcomponents:** • **Review Creator:** Handles new review submission and validation. • **Rating Calculator:** Computes aggregate ratings for mechanics. • **Review Validator:** Ensures review authenticity and authorization. • **Analytics Engine:** Generates insights from review data.
Dependencies	**Relies on:** • ServiceRequest component to verify service completion. • Mechanic component for rating updates. • Client component for review authorship. • Database for review storage. • Notification component for review alerts.
Interfaces	**External Interfaces:** • REST API for review CRUD operations. • Content moderation API for safety checks. **Internal Interfaces:** • `getReviewsByRequestId()`: Retrieves reviews for specific service. • `addReview()`: Creates new review after service completion. • `deleteReview()`: Removes review (with authorization). • `getReviewsOfMechanic()`: Fetches all reviews for a mechanic. • `updateReview()`: Modifies existing review (within edit window). Review guidelines, rating criteria, and moderation policies are documented in the user terms of service. Statistical methods for rating aggregation are detailed in the technical specification.
Resources	• Database for review storage and aggregation. • Content moderation service integration. • Analytics infrastructure for insights generation. • Notification system for review alerts.
Processing	Validates that service is completed before allowing review, authenticates reviewer as actual service recipient, processes rating input (1-5 scale) with

	validation, calculates updated aggregate rating for mechanic, stores review with timestamp and service context, triggers notification to mechanic, updates mechanic profile with new rating, and supports review editing within defined timeframe while maintaining audit trail.
Data	Stores review ID (primary key), request ID (foreign key linking to service), numeric rating (1-5), and comment text. Maintains relationships with ServiceRequest, Client (reviewer), and Mechanic (recipient). Includes metadata such as creation timestamp, last modification time, and moderation status. Aggregated ratings are calculated and stored in Mechanic component.

Table 50: Component - Review

3.2.9 Bill

Identification	The unique name for the component is "**Bill**", located in the system's financial transaction layer managing payment processing and billing.
Type	**Module:** Handles bill generation, payment processing, transaction tracking, and financial record keeping for completed services.
Purpose	Manages complete billing lifecycle including bill creation after service completion, payment method selection, transaction processing, payment status tracking, commission calculation, and financial reporting.
Function	**Handles:** • Automatic bill generation upon service completion. • Base amount calculation from service details. • Commission amount calculation for platform fees. • Multiple payment method support. • Payment status tracking (pending, paid, failed). • Transaction history and receipts. **Processes:** • Service cost calculation based on job complexity. • Platform commission computation. • Payment gateway integration and processing. • Payment verification and confirmation. • Receipt generation and delivery. **Outputs:** • Bill summaries with itemized charges. • Payment confirmation messages. • Error messages for failed transactions.

	• Digital receipts and invoices. • Financial reports and transaction logs.
Subordinates	**Consists of subcomponents:** • **Bill Generator:** Creates bills from completed service requests. • **Payment Processor:** Handles payment gateway integration. • **Commission Calculator:** Computes platform fees. • **Transaction Manager:** Tracks payment status and history. • **Receipt Generator:** Creates digital invoices and receipts.
Dependencies	**Relies on:** • ServiceRequest component for service completion triggers. • Payment gateway APIs (Stripe). • Database for transaction records. • Notification component for payment alerts.
Interfaces	**External Interfaces:** • REST API for Payment gateway APIs for transaction processing. • REST API for bill management and payment operations. **Internal Interfaces:** • `addBill()`: Creates new bill for completed service. • `DeleteBill()`: Removes bill record (admin function). • `UpdateBill()`: Modifies bill details (within constraints). • `getBillsByRequestId()`: Retrieves bills for specific service request. Payment processing standards, supported payment methods, transaction security protocols, and refund policies are documented in the financial operations manual.
Resources	• Database for bill and transaction storage. • Payment gateway integration infrastructure. • Secure payment token storage and encryption. • Receipt generation. • Financial reporting and analytics tools. • Audit logging for compliance and dispute resolution.
Processing	Generates bills automatically upon service completion with base amount and commission calculation, integrates with payment gateways for secure transaction processing, validates payment methods and user authorization, processes payments with retry logic for failures, updates payment status in

	real-time (pending → processing → paid/failed), generates digital receipts upon successful payment, distributes funds between mechanic earnings and platform commission, maintains complete transaction audit trail, handles refunds and disputes according to policy, and ensures PCI compliance for all payment data handling.
Data	Stores bill ID (primary key), request ID (foreign key), base amount (service charge), commission amount (platform fee), total amount (base + commission), payment method (card/cash/digital wallet), payment status (pending/paid/failed/refunded), candidate date (creation date), and paid date (payment completion timestamp). Maintains relationships with ServiceRequest component. Includes transaction reference IDs from payment gateways and complete payment history.

Table 51: Component - Bill

3.2.10 Notification

Identification	The unique name for the component is "**Notification**", located in the system's communication layer managing all user notifications and alerts.
Type	**Module:** Handles notification creation, delivery, tracking, and management across multiple channels (in-app, push, email).
Purpose	Manages all system notifications including service updates, payment confirmations, review alerts, and administrative messages. Ensures users stay informed about important events and status changes.
Function	**Handles:** • Multi-channel notification delivery (in-app, push, email). • Notification type categorization and prioritization. • Read/unread status tracking. • Notification preferences and filtering. • Delivery confirmation and retry logic. • Notification history and archiving. **Processes:** • Notification generation from system events. • Channel selection based on user preferences and urgency. • Template-based message formatting. • Delivery scheduling and batching. • Read receipt tracking and analytics. **Outputs:** • In-app notifications with action buttons.

	• Push notifications to mobile devices. • Email notifications for important updates. • Notification badges and counters. • Delivery status reports.
Subordinates	**Consists of subcomponents:** • **Notification Generator:** Creates notifications from system events. • **Channel Router:** Determines optimal delivery channels. • **Template Engine:** Formats messages using predefined templates. • **Delivery Manager:** Handles multi-channel notification dispatch. • **Tracking System:** Monitors read status and user engagement. • **Preference Manager:** Respects user notification settings.
Dependencies	**Relies on:** • All system components as notification triggers. • Push notification services (Firebase). • Email service for email notifications. • Database for notification storage and tracking. • User preferences from Client/Mechanic components. • Template storage for message formatting.
Interfaces	**External Interfaces:** • Push notification service APIs (Firebase Cloud Messaging). • Email service API for email delivery. • REST API for notification retrieval and management. • WebSocket for real-time in-app notifications. **Internal Interfaces:** • `sendNotification()`: Creates and sends notification to user. • `markAsRead()`: Updates notification read status. • `getNotificationsByUser()`: Retrieves user's notifications. • `getUnreadNotificationsByUser()`: Fetches unread notifications. • `getUnreadNotificationsByUser()`: Enhanced unread retrieval with filtering. • `deleteNotification()`: Removes notification from user's list. Notification types, priority levels, delivery rules, and template formats are defined in the messaging specification.
Resources	• Database for notification storage and history. • Push notification service infrastructure.

	• Email delivery service and templates. • Message queue for reliable delivery. • Template storage and rendering engine. • Analytics for notification effectiveness tracking.
Processing	Receives notification triggers from various system events, determines notification type and priority level, selects appropriate delivery channels based on urgency and user preferences, formats messages using predefined templates with dynamic content, delivers notifications through multiple channels with retry logic for failures, tracks delivery status and read receipts, respects user preferences for notification frequency and types, batches non-urgent notifications to avoid overwhelming users, maintains notification history for user reference, supports notification actions (eg quick replies, deep links), and provides analytics on notification effectiveness and user engagement.
Data	Stores notification ID (primary key), user ID (foreign key to recipient), request ID (foreign key for context), notification type (service_update/payment/review/admin), message content, read status (boolean), creation timestamp, read timestamp, delivery status (sent/delivered/failed), and delivery channel (in-app/push/email). Maintains relationships with all user types and related entities. Includes metadata for deep linking and action buttons.

Table 52: Component - Notification

3.2.11 Service

Identification	The unique name for the component is "**Service**", located in the system's configuration layer managing available service offerings and pricing.
Type	**Module:** Handles service catalog management, service definitions, pricing information, and service availability configuration.
Purpose	Manages the catalog of available services including service names, descriptions, base pricing, vehicle type compatibility, estimated duration, and service categories. Supports service discovery and selection for clients and mechanics.
Function	**Handles:** • Service catalog maintenance and updates. • Service descriptions and specifications. • Base pricing and cost structure management. • Vehicle type compatibility definitions.

	• Service categorization and tagging. • Service availability and active status. **Processes:** • Service creation and modification (admin function). • Service search and filtering. • Compatibility validation with vehicle types. • Service popularity and usage analytics. **Outputs:** • Service listings for client selection. • Pricing information and estimates. • Error messages for unavailable or incompatible services. • Service recommendations based on vehicle type. • Usage reports and analytics.
Subordinates	**Consists of subcomponents:** • **Catalog Manager:** Maintains service definitions and metadata. • **Pricing Engine:** Handles cost calculations and updates. • **Compatibility Checker:** Validates service-vehicle compatibility. • **Search Engine:** Enables service discovery and filtering. • **Analytics Tracker:** Monitors service usage and popularity.
Dependencies	**Relies on:** • Admin component for service management operations. • Database for service catalog storage. • Car component for vehicle compatibility checks. • Pricing configuration and business rules engine.
Interfaces	**External Interfaces:** • REST API for service catalog retrieval. • Admin API for service management operations. **Internal Interfaces:** • `addService()`: Creates new service offering (admin). • `updateService()`: Modifies service details (admin). • `deleteService()`: Removes service from catalog (admin). • `getAllService()`: Retrieves complete service catalog. • `getServiceById()`: Fetches specific service details. Service categorization taxonomy, pricing models, and compatibility matrices are documented in the business configuration guide. Service level

	agreements and quality standards are defined in the operations manual.
Resources	• Database for service catalog and metadata. • Configuration management for pricing rules. • Search indexing for service discovery. • Image storage for service illustrations.
Processing	Manages service catalog with comprehensive service definitions, maintains pricing structure with support for dynamic pricing and promotions, validates vehicle compatibility during service selection, enables efficient service search and filtering by multiple criteria, tracks service popularity and usage patterns, supports service bundling and package deals, provides cost estimates based on vehicle type and service complexity, maintains service availability status and seasonal offerings, ensures data consistency across related components, and supports localization for service names and descriptions in multiple languages.
Data	Stores service ID (primary key), service name, detailed description, base pricing (number), estimated duration, vehicle type compatibility (array or object), service category tags, image references, active status (boolean), creation date, and last update timestamp. Maintains relationships with ServiceRequest for usage history. Includes metadata such as popularity scores, average ratings, and completion statistics. Pricing may include tiered structures based on vehicle type or service complexity.

Table 53: Component - Service

3.2.12 Admin

Identification	The unique name for the component is "**Admin**", located in the system's administrative layer providing comprehensive system management and oversight capabilities.
Type	**Module:** Handles system administration, user management, content moderation, analytics, reporting, and configuration management for the entire platform.
Purpose	Provides administrative controls for system management including user account oversight, mechanic verification, service configuration, dispute resolution, analytics monitoring, report generation, and platform configuration. Central control point for system governance.
Function	**Handles:** • Administrator authentication and authorization. • User management (clients and mechanics) with full CRUD

	operations. • Mechanic verification and approval workflows. • Service catalog management and pricing updates. • Dispute resolution and support ticket handling. • Financial reporting and transaction oversight. • System analytics and performance monitoring. • Content moderation and safety enforcement. • Platform configuration and settings management. **Processes:** • Secure admin authentication with elevated privileges. • Multi-level approval workflows for sensitive operations. • Data validation and integrity checks. • Audit logging of all administrative actions. • Report generation and data export. • System health monitoring and alerts. **Outputs:** • User management confirmations and status updates. • Mechanic verification approvals/rejections. • Financial reports and transaction summaries. • System analytics dashboards and metrics. • Audit logs and compliance reports. • Error alerts and system health notifications. • Configuration change confirmations.
Subordinates	**Consists of subcomponents:** • **User Manager:** Handles client and mechanic account oversight. • **Verification Handler:** Processes mechanic verification requests. • **Service Manager:** Maintains service catalog and pricing. • **Report Generator:** Creates analytics and financial reports. • **Dispute Resolver:** Manages conflicts and support issues. • **Configuration Manager:** Controls platform settings and parameters. • **Audit Logger:** Tracks all administrative actions. • **Analytics Dashboard:** Provides real-time system metrics.

Dependencies	**Relies on:** • All system components for management oversight. • Database for comprehensive data access. • Authentication service for secure admin access. • Reporting and analytics infrastructure. • Email service for administrative communications. • Notification component for system alerts. • File storage for reports and backups.
Interfaces	**External Interfaces:** • Secure admin portal with role-based access control. • REST API with admin authentication and authorization. • Reporting API for data export and analytics. **Internal Interfaces:** • `register()`: Creates admin account (super-admin only). • `login()`: Authenticates admin credentials. • `forgotPassword()`: Admin password recovery. • `changePassword()`: Updates admin password. • `updateProfile()`: Modifies admin information. • `manageUsers()`: Full user account management. • `manageMechanics()`: Mechanic verification and oversight. • `manageRequests()`: Service request monitoring. • `manageBills()`: Financial transaction management. • `manageReviews()`: Review moderation. • `manageChats()`: Communication monitoring. • `manageNotifications()`: Notification system control. • `manageService()`: Service catalog administration. • `viewReports()`: Access analytics and reports. Admin permission levels, role definitions, audit requirements, and security protocols are documented in the administrative security policy. Data access controls and privacy compliance measures are detailed in the governance framework.
Resources	• Dedicated admin database with access controls. • Analytics and reporting infrastructure. • Audit logging system with tamper-proof storage. • Secure authentication with multi-factor support.

	• Data backup and recovery systems. • Monitoring and alerting infrastructure. • Export and import tools for data management. • Compliance reporting tools.
Processing	Authenticates administrators with enhanced security measures including MFA, authorizes actions based on role-based access control (RBAC), processes user management operations with appropriate safeguards, handles mechanic verification workflow with document validation, manages service catalog with pricing controls, resolves disputes through structured workflows, generates comprehensive reports on demand and scheduled basis, monitors system health with real-time alerts for issues, moderates content for policy compliance and safety, maintains complete audit trail of all administrative actions, enforces data retention and privacy policies, supports data export for compliance and analysis, manages system configuration with change tracking, and coordinates platform-wide updates and maintenance.
Data	Stores admin ID (primary key), full name, email, encrypted password, phone number, profile image, role (super-admin/admin/moderator), status (active/inactive), creation timestamp, and last login time. Maintains audit log linking admin actions to specific operations with timestamps. Includes permission sets defining access levels for different admin roles. All sensitive operations are logged with admin attribution for compliance and accountability.

Table 54: Component - Admin

4. User Interface Design

4.1 Section Overview

This section provides an overview of the user interface design of the project, describing the key principles, components, and layout strategies used to ensure a seamless experience across different user roles.

It covers the following aspects:

- **Interface Design Rules:** Explains the design principles followed to create an intuitive, consistent, and responsive interface for all users.

- **GUI Components:** Lists the core and custom components, third-party libraries, and APIs leveraged for building the mobile and web interfaces.
- **Detailed Description:** Offers a breakdown of the interface layout and functionality for each user role, supported by screen snapshots in the appendix to visualize the design and interactions.

4.2 Interface Design Rules

The interface design follows best practices to ensure a user-friendly and intuitive experience:

- **Mobile-First Design:** Ensures responsive layouts across different devices and screen sizes.
- **Simplicity:** Clean layouts with essential information for ease of use.
- **User-Centric Approach:** Focus on accessibility and usability for all roles (Clients, Mechanics, Admins).
- **Consistency:** Uniform design patterns, colours, and fonts across all screens for familiarity.
- **Feedback & Interaction:** Visual cues for button presses, loading states, and errors to confirm user actions.

4.3 GUI Components

The project utilizes the following components and APIs for development:

- **GUI Components:** React Native core and custom components with use of third-party libraries for responsiveness and modern UI.
- **APIs:** Open AI API for AI based vehicle assistance and Google maps API for live tracking purpose.
- **Development and Framework:** MERN Stack for Web and Mobile Application.
- **Database:** MongoDB Atlas for easy and free storage and retrieval.

4.4 Detailed Description

Figure 7: Splash Screen

Figure 8: On-boarding Screen - 1

Figure 9: On-boarding Screen - 2

Figure 10: On-boarding Screen - 3

Figure 11: On-boarding Screen - 4

Figure 12: Choose Role Screen

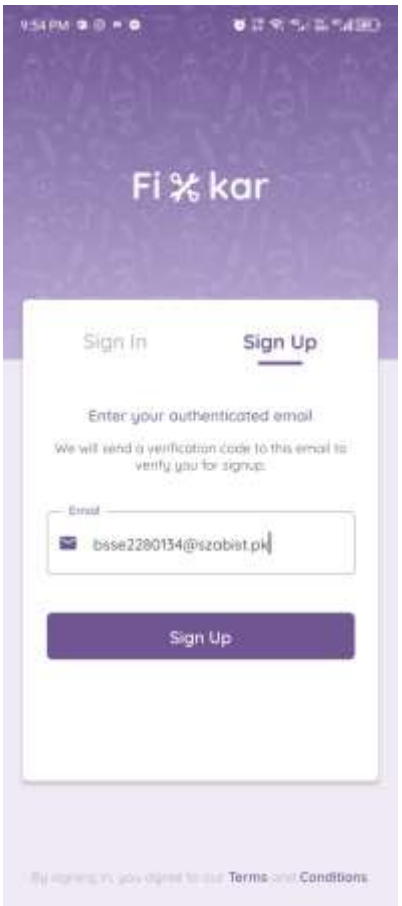


Figure 13: SignUp Screen

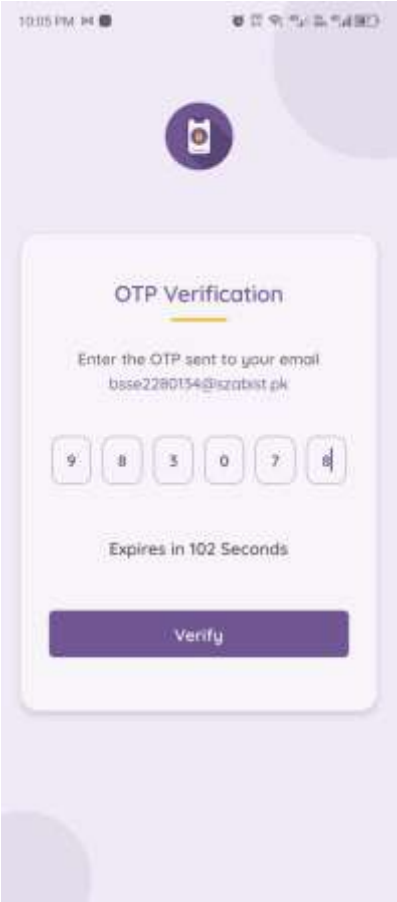


Figure 14: OTP Screen

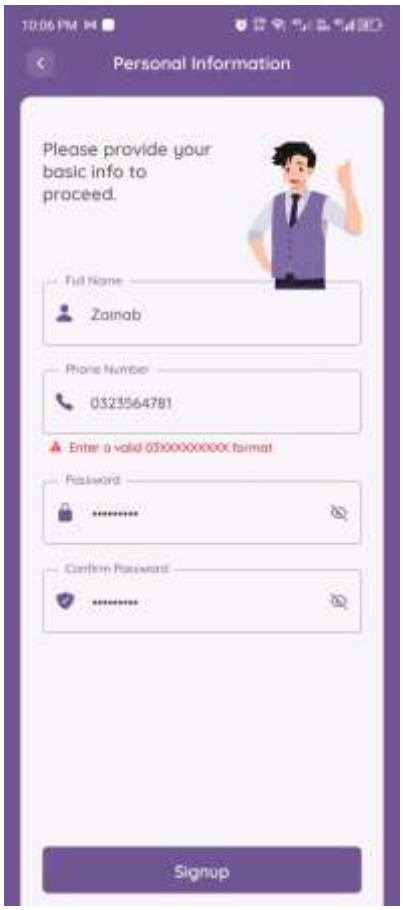


Figure 15: SignUp Detail Screen - Client

Figure 16: SignUp Detail Screen - Mechanic - 1

Figure 17: SignUp Detail Screen - Mechanic - 2

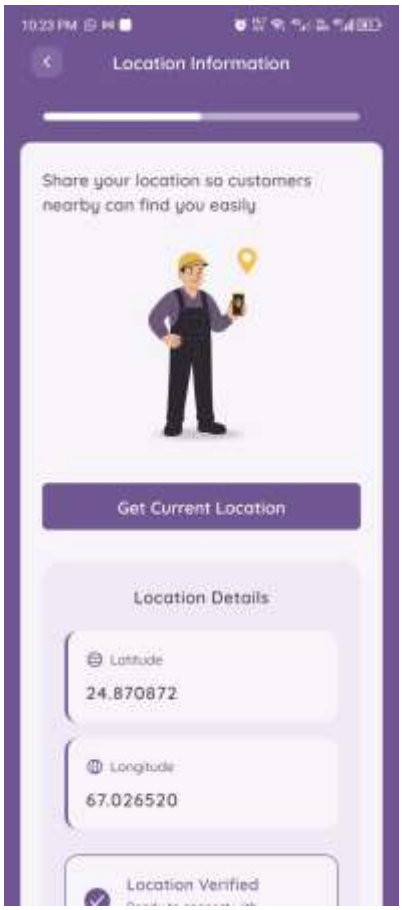


Figure 18: SignUp Detail Screen -Mechanic - 3

Figure 19: SignUp Detail Screen -Mechanic - 4

Figure 20: SignUp Detail Screen - Mechanic - 5

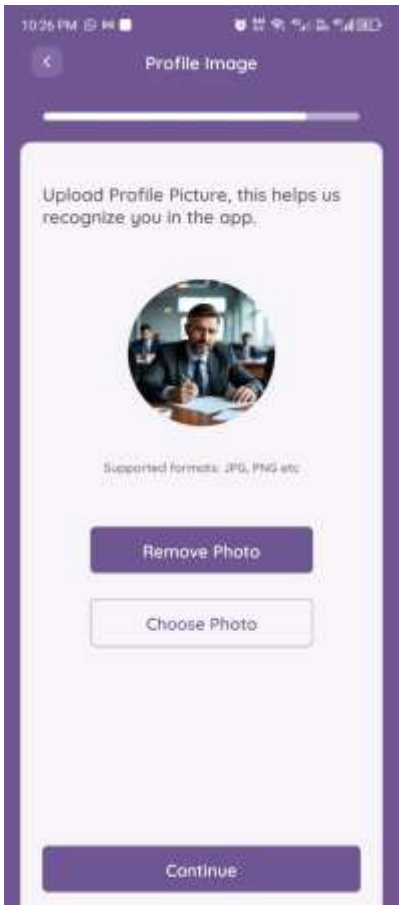


Figure 21: SignUp Detail Screen - Mechanic - 6

Figure 22: Login Screen

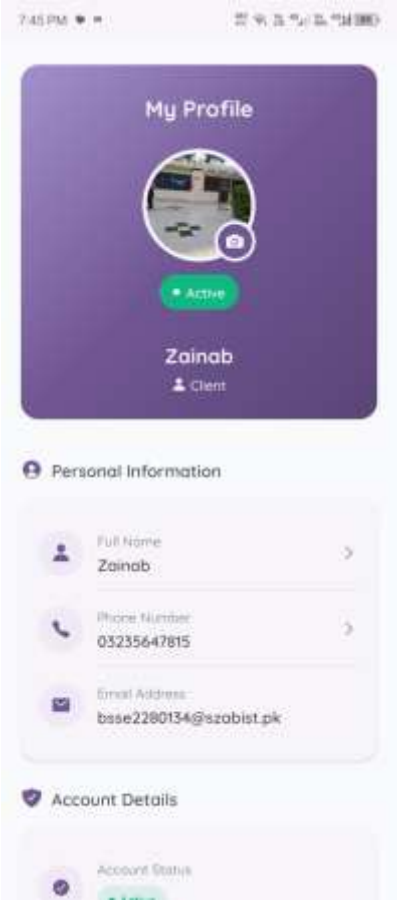


Figure 23: Profile Screen - Client - 1

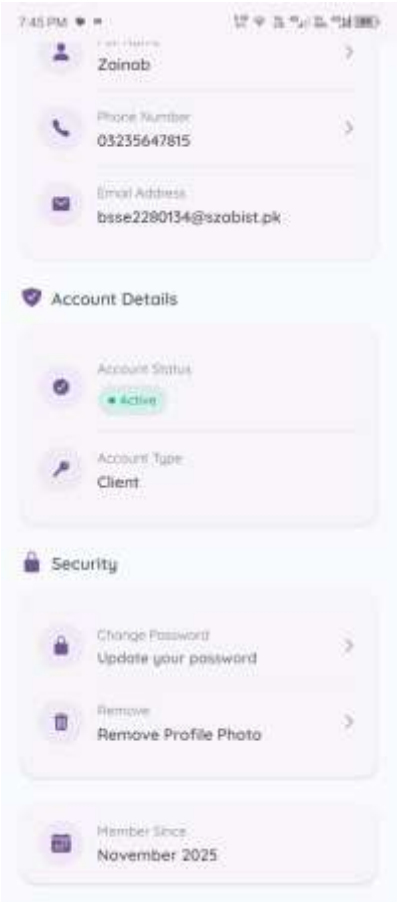


Figure 24: Profile Screen - Client - 2

Figure 25: Profile Screen - Mechanic - 1

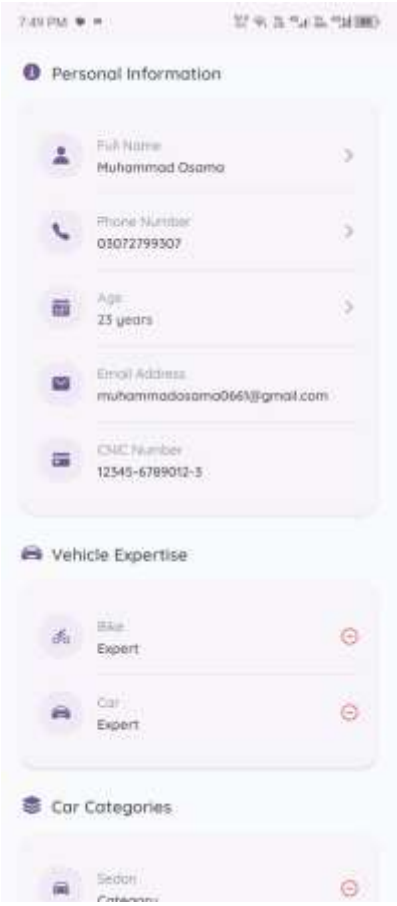


Figure 26: Profile Screen - Mechanic - 2

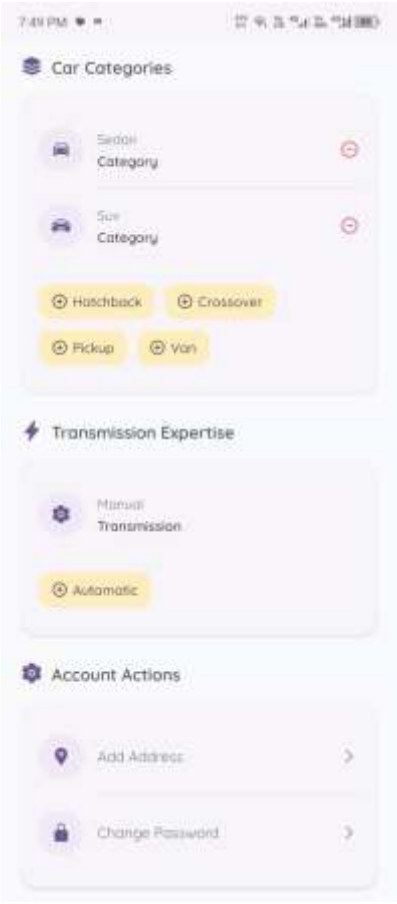


Figure 27: Profile Screen - Mechanic - 3

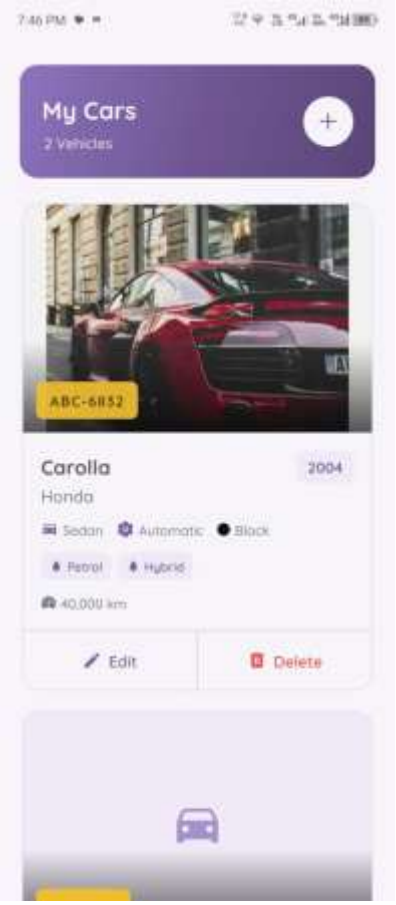


Figure 28: My Cars Screen

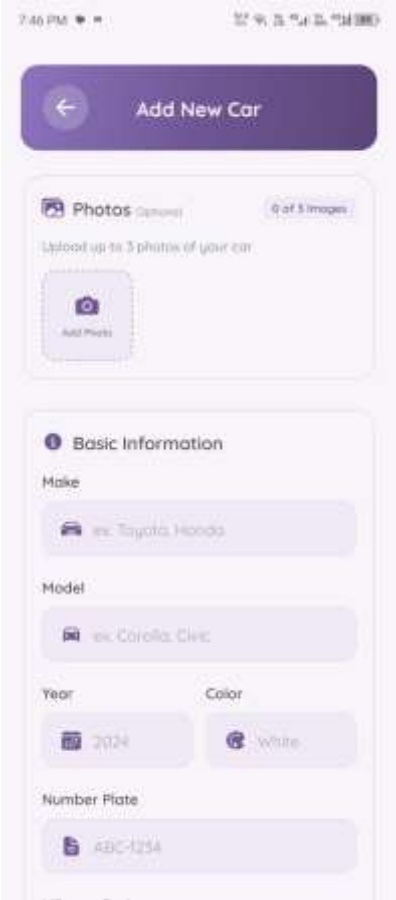


Figure 29: Add Car Screen

Figure 30: Edit Car Screen

Figure 31: AI Chat Screen - 1

Figure 32: AI Chat Screen - 2

Figure 33: Dashboard Screen - Client

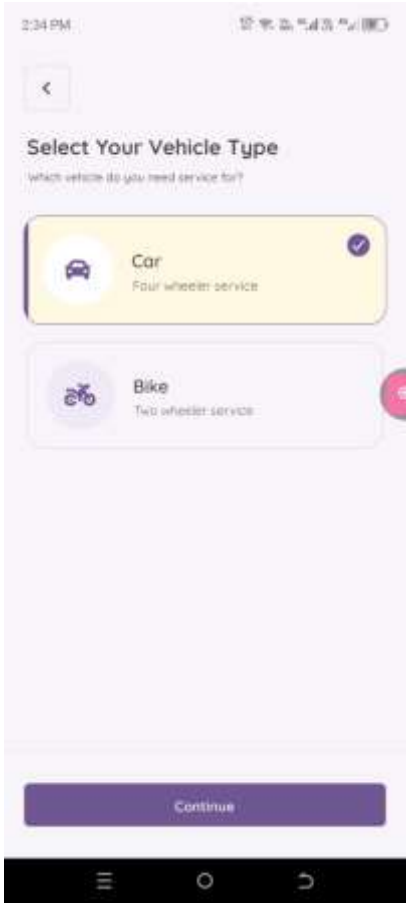


Figure 34: Vehicle Type Screen

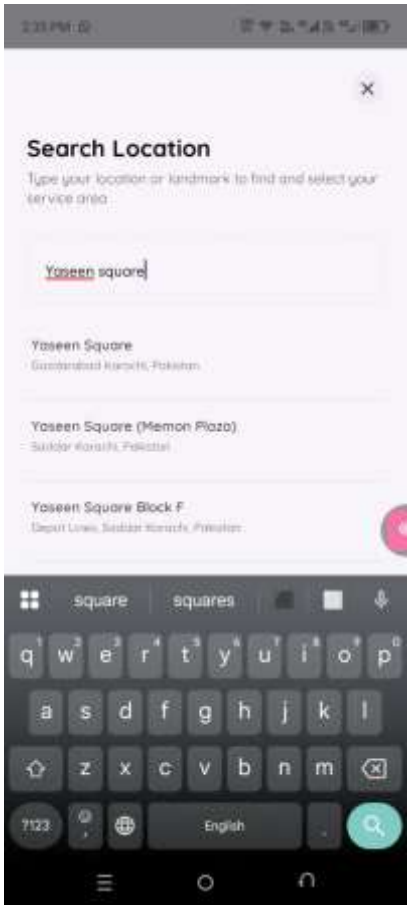


Figure 35: Search Location Screen

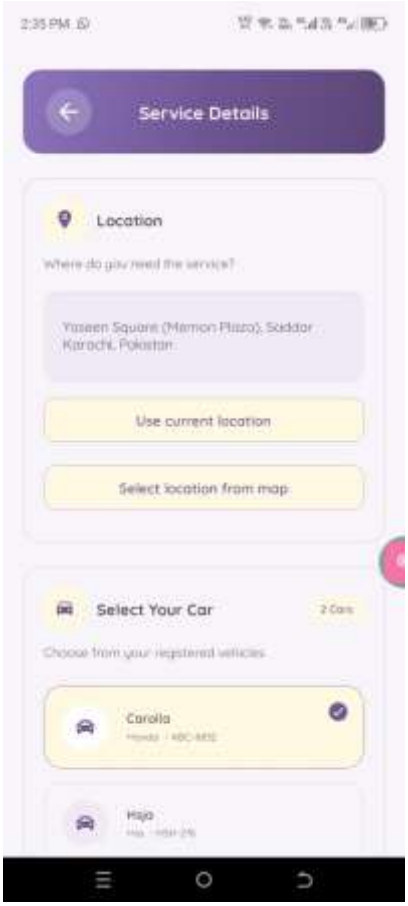


Figure 36: Service Detail Screen - 1

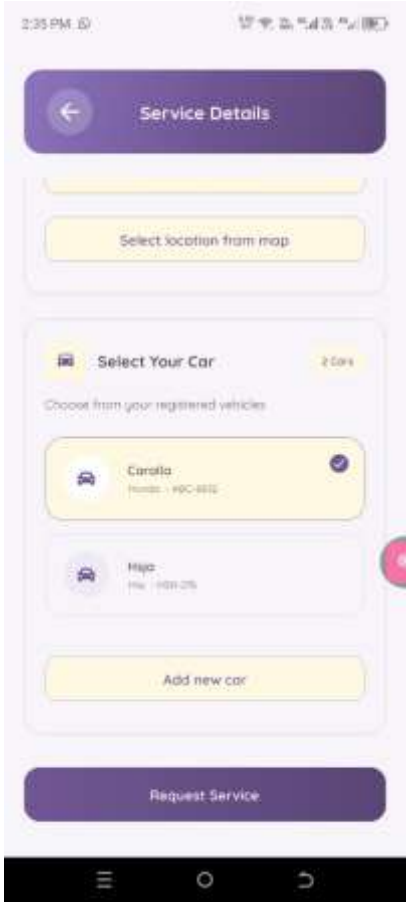


Figure 37: Service Detail Screen - 2

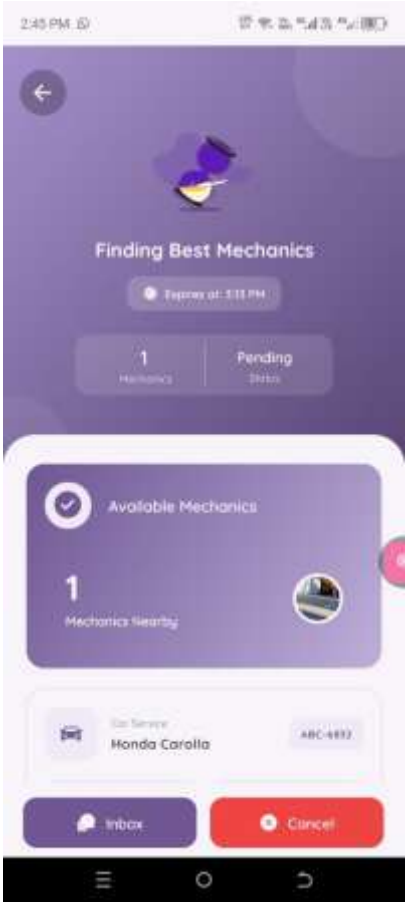


Figure 38: Active Request - Client - 1

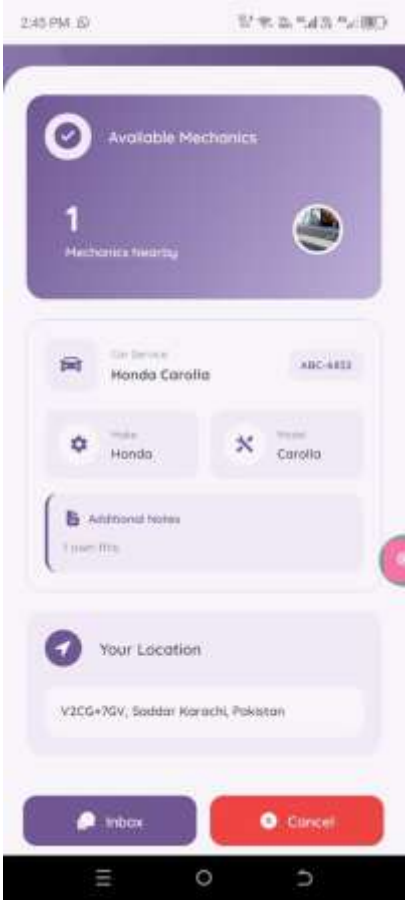


Figure 39: Active Screen - Client - 2

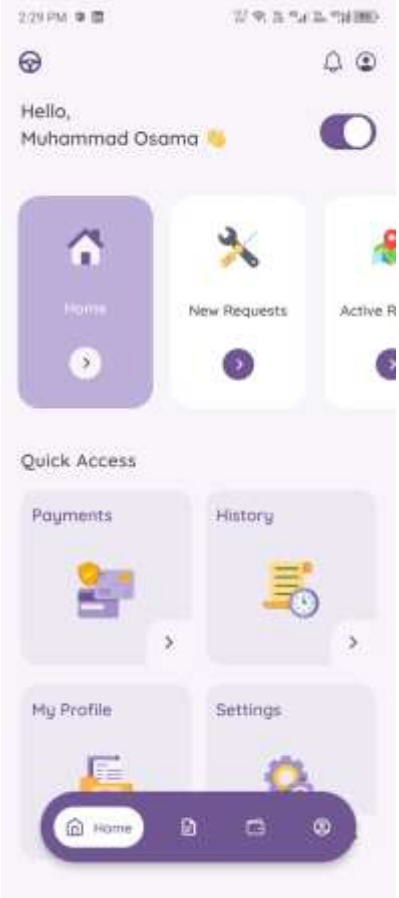


Figure 40: Dashboard Screen - Mechanic

Figure 41: Request Screen

Figure 42: Request Detail Screen - 1

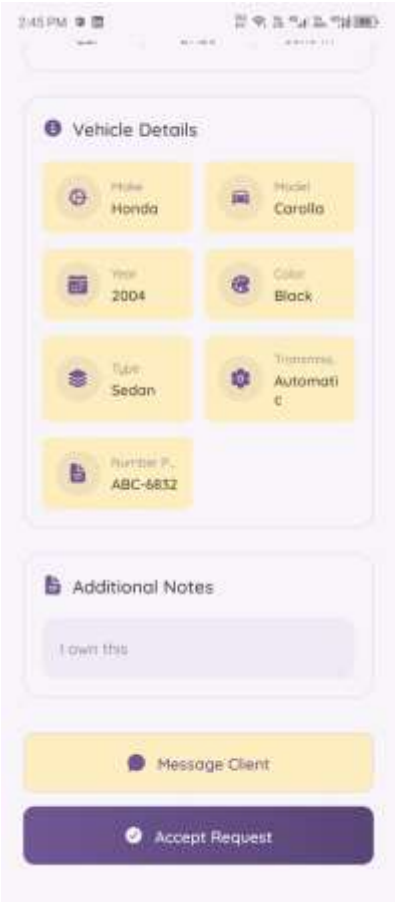


Figure 43: Request Detail Screen - 2

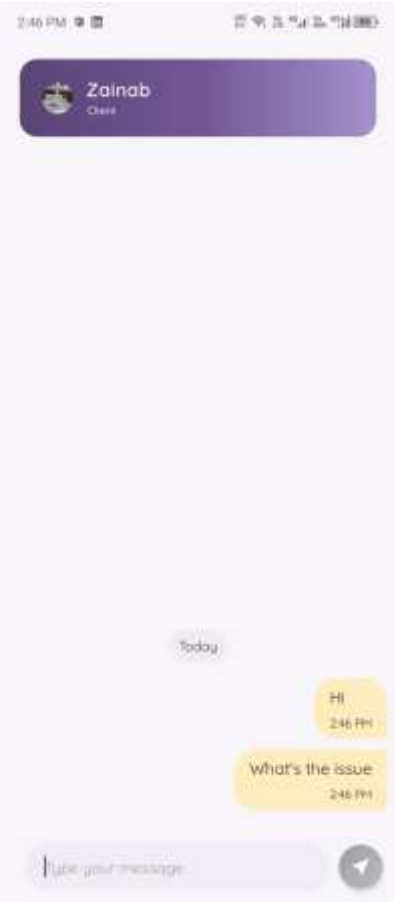


Figure 44: Chat Screen - Mechanic

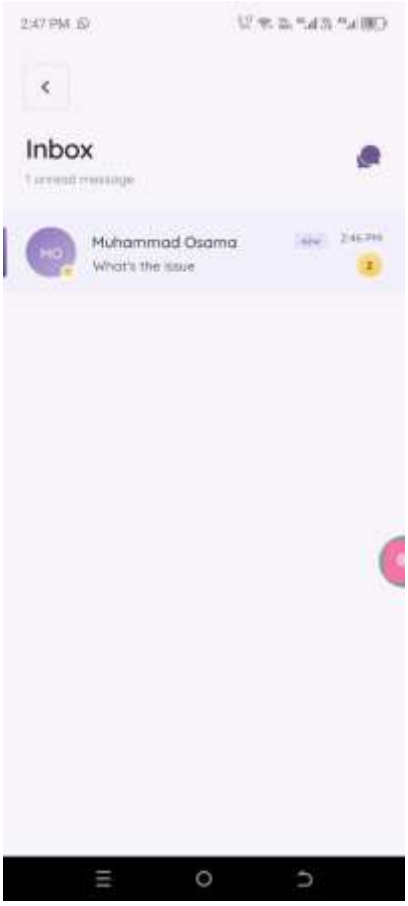


Figure 45: Inbox Screen - Client

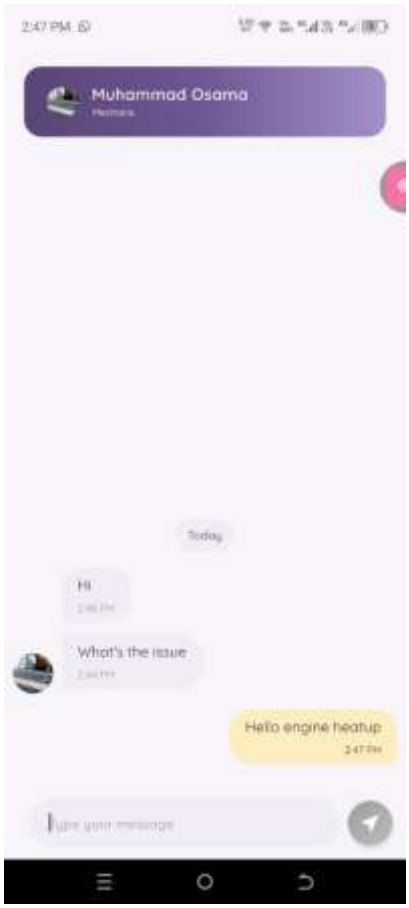


Figure 46: Chat Screen - Client

Figure 47: Active Screen - Client - 3

Figure 48: Active Screen - Mechanic - 1

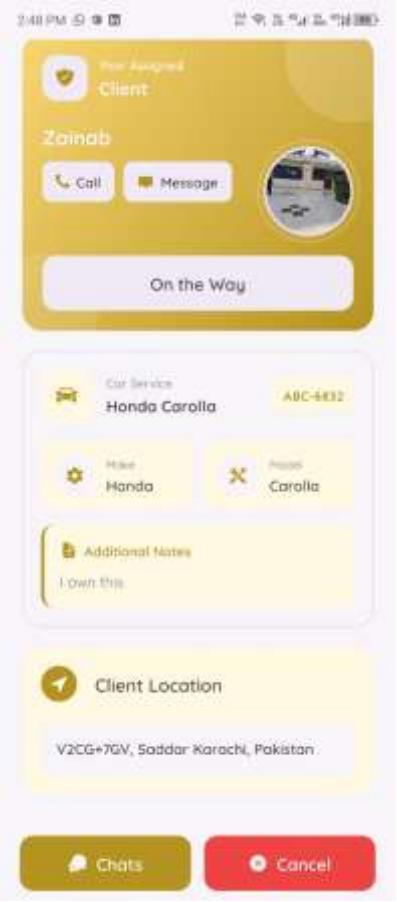


Figure 49: Active Screen - Mechanic - 2

Figure 50: Live Tracking - Client

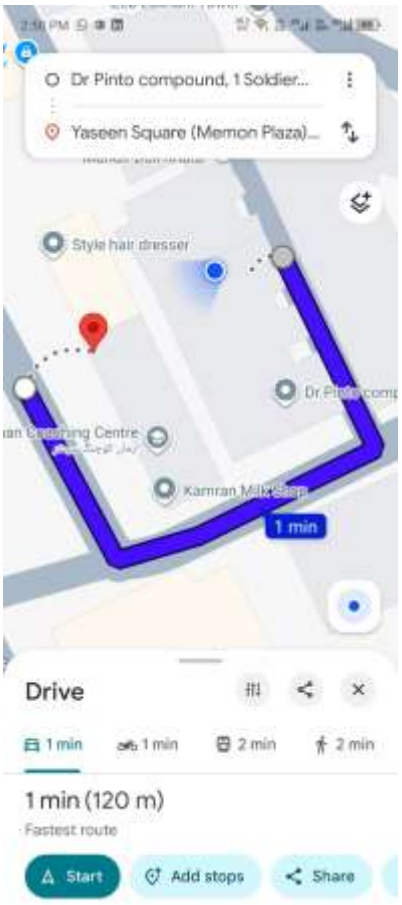


Figure 51: Live Tracking - Mechanic

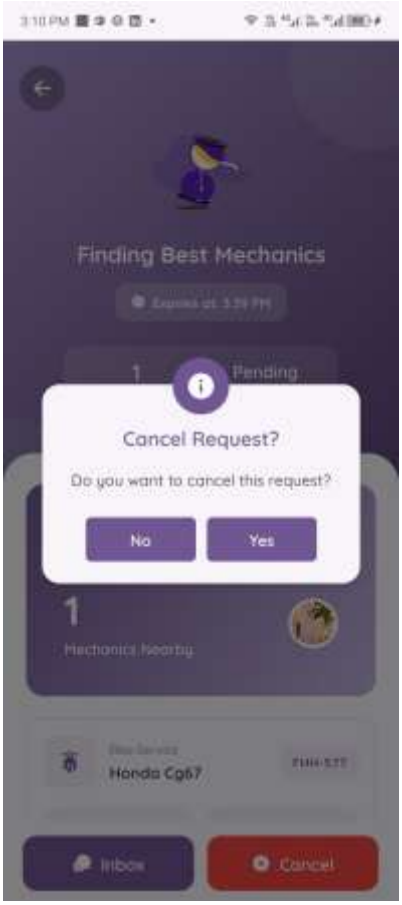


Figure 52: Cancel Request Screen

Figure 53: Toasts

5. Reuse and Relationship to Other Products

- **Role in Design:** Reuse is applied through frameworks like React Native and MERN, and reusable custom components to maintain a consistent design.
- **Role in Implementation:** APIs like OpenAI and Google Maps, Redux hooks, and MongoDB Atlas are reused across screens to simplify development and reduce redundancy.
- **Non-Reuse Motivation:** Some components were custom built due to compatibility issues, specific project requirements, or performance optimization.

6. Design Decisions and Tradeoffs

- **Decisions:**
  - **Cross-Platform Development**: Choose React Native for mobile and MERN stack for web to allow a single codebase for multiple platforms.
  - **Use of OpenAI API**: Integrated AI assistance for vehicle troubleshooting to reduce manual effort and improve user experience.
  - **Google Maps API**: Used for live tracking of mechanics instead of building a custom mapping solution to save development time.
- **Tradeoff:**
  - **Custom vs. Third-Party Components**: Some features (eg, status updates, vehicle info cards) were custom-built for performance and UI consistency, even though third-party libraries could have been used.
- **Abandoned Ideas:**
  - Instead of building a custom chat from scratch, the project uses the Gifted Chat library to implement messaging, allowing easy integration with AI responses and Redux state management.
  - Offline mode was initially considered but dropped due to time and complexity constraints.

7. Pseudo for components

7.1 Auth Controller

Login:

```
FUNCTION Login(request, response)

  READ email, password FROM request body

  IF email OR password is missing
     RETURN error "Email and password are required"

  SEARCH Mechanic database USING email
  SET userType = "mechanic"

  IF user not found
     SEARCH Client database USING email
     SET userType = "client"

  IF user still not found
     RETURN error "Invalid email or password"

  COMPARE input password WITH stored hashed password

  IF password does not match
     RETURN error "Invalid email or password"

  IF userType is mechanic
     IF user is not verified OR not approved
        RETURN error "Mechanic account not approved"

  IF userType is client
     IF user status is blocked
        RETURN error "Account is blocked"

  GENERATE JWT token WITH user id, email, role, and userType

  PREPARE user response data

  RETURN success response WITH token and user data

END FUNCTION
```

Send OTP:

```
FUNCTION SendOTP(request, response)

  READ email FROM request body

  IF email is missing
```

```
    RETURN error "Email is required"

  SEARCH Client, Mechanic, and Admin databases USING email

  IF user not found
    RETURN error "Email not registered"

  GENERATE 6-digit OTP
  SET OTP expiry time = current time + 2 minutes

  SAVE OTP and expiry time in user record

  SEND OTP to user email

  RETURN success message "OTP sent"

END FUNCTION
```

Verify OTP:

```
FUNCTION VerifyOTP(request, response)

  READ email, otp FROM request body

  IF email OR otp is missing
    RETURN error "Email and OTP are required"

  SEARCH Client, Mechanic, and Admin databases USING email

  IF user not found
    RETURN error "Email not registered"

  IF OTP or expiry not found
    RETURN error "No OTP found"

  IF stored OTP does not match input OTP
    RETURN error "Invalid OTP"

  IF OTP expiry time has passed
    RETURN error "OTP expired"

  CLEAR OTP and expiry from database

  GENERATE reset token valid for 10 minutes

  RETURN success response WITH reset token

END FUNCTION
```

Forgot Change Password:

```
FUNCTION ForgotChangePassword(request, response)

  READ userId and role FROM authenticated request
```

```
  READ newPassword FROM request body

  IF newPassword is missing
     RETURN error "New password is required"

  SELECT database model BASED ON user role

  IF role is invalid
     RETURN error "Invalid user role"

  FIND user BY userId

  IF user not found
     RETURN error "User not found"

  HASH new password

  UPDATE user's password in database

  RETURN success message "Password changed successfully"

END FUNCTION
```

Authentication Middleware:

```
FUNCTION ensureAuthenticated(req, res, next)
  GET authHeader from req.headers['authorization']
  IF authHeader missing
    RETURN 401 "Token is required"

  EXTRACT token from authHeader (remove 'Bearer ' prefix if exists)
  IF token missing
    RETURN 401 "Token is malformed"

  TRY
    DECODE token using JWT_SECRET
    SET req.user = decoded token
    CALL next()
  CATCH error
    RETURN 403 "Invalid or expired token"
END FUNCTION
```

Node Mailer:

```
IMPORT nodemailer library

CREATE email transporter USING Gmail service
    SET sender email from environment variables
    SET sender password from environment variables

FUNCTION sendMail(receiverEmail, emailSubject, emailContentHTML)

    CREATE mailOptions
        FROM sender email
```

```
      TO receiver email
      SUBJECT emailSubject
      BODY emailContentHTML

   SEND email using transporter
END FUNCTION

EXPORT sendMail function
```

7.2 Mechanic Controller

Register:

```
FUNCTION RegisterMechanic(request, response)

  READ mechanic details FROM request body

  CHECK Client and Mechanic databases USING email
  IF email already exists
     RETURN error "Email already registered"

  HASH password

  IF profile image is provided
     UPLOAD image to cloud
     STORE profile image URL

  IF CNIC front image is provided
     UPLOAD image
     STORE URL

  IF CNIC back image is provided
     UPLOAD image
     STORE URL

  IF past work images are provided
     FOR each image
        UPLOAD image
        SAVE image URL in list

  CREATE new mechanic object WITH all data

  SAVE mechanic in database

  RETURN success message "Account created, waiting for verification"

END FUNCTION
```

Get All Mechanics:

```
FUNCTION GetAllMechanics(request, response)

  FETCH all mechanics FROM database
```

```
  IF no mechanics found
     RETURN error "Data not found"

  RETURN success response WITH mechanics list

END FUNCTION
```

Get Mechanic By Id:

```
FUNCTION GetMechanicById(request, response)

  READ mechanic ID FROM request parameters

  FIND mechanic BY ID

  IF mechanic not found
     RETURN error "Mechanic not found"

  RETURN success response WITH mechanic data

END FUNCTION
```

Update Mechanic:

```
FUNCTION UpdateMechanic(request, response)

  READ mechanic ID FROM request parameters

  IF user is not admin AND user ID ≠ mechanic ID
     RETURN error "Forbidden"

  FIND mechanic BY ID

  IF mechanic not found
     RETURN error "Mechanic not found"

  FOR each field in request body
     IF field exists
        UPDATE mechanic field

  IF email is changed
     CHECK for duplicate email
     IF email already exists
        RETURN error

  IF new profile image provided
     DELETE old image from cloud
     UPLOAD new image
     SAVE new URL

  IF CNIC or past work images provided
     UPLOAD images and update URLs
```

```
  SAVE updated mechanic

  REMOVE password from response

  RETURN success message "Mechanic updated"

END FUNCTION
```

Delete Mechanic:

```
FUNCTION DeleteMechanic(request, response)

  READ mechanic ID FROM request parameters

  DELETE mechanic FROM database

  IF mechanic not found
     RETURN error "Mechanic not found"

  RETURN success message "Mechanic deleted"

END FUNCTION
```

Get All Unverified Mechanics:

```
FUNCTION GetAllUnverifiedMechanics(request, response)

  FIND mechanics WHERE verificationStatus = "pending"

  IF no mechanics found
     RETURN error "No unverified mechanics"

  RETURN success response WITH mechanics list

END FUNCTION
```

Update Mechanic Location:

```
FUNCTION UpdateMechanicLocation(request, response)

  READ latitude and longitude FROM request body

  IF latitude OR longitude missing
     RETURN error "Coordinates required"

  UPDATE mechanic location IN database USING GeoJSON

  IF mechanic not found
     RETURN error "Mechanic not found"

  EMIT location update event via socket to connected clients

  RETURN success message "Location updated"
```

```
END FUNCTION
```

7.3 Client Controller

Register:

```
FUNCTION ClientRegister(request, response)

  READ fullName, email, phoneNumber, password FROM request body

  IF any required field is missing
     RETURN error "All fields are required"

  CHECK Client and Mechanic databases USING email
  IF email already exists
     RETURN error "Email already registered"

  HASH password

  CREATE new Client object WITH provided details

  SAVE client to database

  RETURN success message "Account created successfully"

END FUNCTION
```

Get All Clients:

```
FUNCTION GetAllClients(request, response)

  FETCH all clients FROM database

  IF no clients found
     RETURN error "Data not found"

  RETURN success response WITH client list

END FUNCTION
```

Get Client By Id:

```
FUNCTION GetClientById(request, response)

  READ client ID FROM request parameters

  IF ID is missing
     RETURN error "Client ID required"

  FIND client BY ID

  IF client not found
     RETURN error "Client not found"
```

```
  RETURN success response WITH client data

END FUNCTION
```

Update Client:

```
FUNCTION UpdateClient(request, response)

  READ client ID FROM request parameters

  IF user is not admin AND user ID ≠ client ID
     RETURN error "Forbidden"

  FIND client BY ID

  IF client not found
     RETURN error "Client not found"

  IF email is changed
     CHECK Client and Mechanic databases
     IF email already exists
        RETURN error "Email already in use"

  UPDATE client fields (name, phone, location)

  IF user is admin AND status provided
     UPDATE client status

  IF profile image provided
     DELETE old image from cloud (if exists)
     UPLOAD new image
     SAVE image URL

  SAVE updated client

  REMOVE password from response

  RETURN success message "Client updated successfully"

END FUNCTION
```

Delete Client:

```
FUNCTION DeleteClient(request, response)

  READ client ID FROM request parameters

  FIND client BY ID

  IF client not found
     RETURN error "Client not found"

  DELETE client FROM database
```

```
  RETURN success message "Client deleted successfully"

END FUNCTION
```

Remove Client Profile Image:

```
FUNCTION RemoveClientImage(request, response)

  READ client ID FROM request parameters

  IF user is not admin AND user ID ≠ client ID
     RETURN error "Forbidden"

  FIND client BY ID

  IF client not found
     RETURN error "Client not found"

  IF profile image does not exist
     RETURN error "No image to remove"

  DELETE image from cloud storage

  SET profile image fields to null

  SAVE client

  REMOVE password from response

  RETURN success message "Profile image removed"

END FUNCTION
```

7.4 Car Controller

Add Car:

```
FUNCTION AddCar(request, response)

  READ userId, make, model, year, engineType, color, numberPlate,
       carType, transmission, mileage, additionalNotes FROM request body

  IF any required field (userId, make, model, year, carType, numberPlate) is missing
     RETURN error "Required fields missing"

  FIND client BY userId
  IF client not found
     RETURN error "Client not found"

  CHECK Car collection FOR duplicate numberPlate
  IF duplicate exists
     RETURN error "Number plate already exists"
```

```
  INITIALIZE empty uploadedImages array
  IF files provided
     IF more than 3 files
        RETURN error "Maximum 3 images allowed"

     FOR each file
        UPLOAD to cloud storage
        STORE {url, public_id} IN uploadedImages

  CREATE new Car object WITH provided details and uploadedImages

  SAVE new Car to database

  RETURN success response WITH new Car

END FUNCTION
```

Get Cars By User:

```
FUNCTION GetCarsByUser(request, response)

  READ user ID from request parameters

  VALIDATE user ID
  IF invalid
     RETURN error "Invalid user ID"

  FIND cars WHERE car.user = user ID
     POPULATE user info (id, fullName, email, role)

  IF no cars found
     RETURN success response WITH empty cars array

  FORMAT result:
     user info from first car
     cars array with relevant fields

  RETURN success response WITH result

END FUNCTION
```

Get Single Car:

```
FUNCTION GetSingleCar(request, response)

  READ car ID from request parameters

  VALIDATE car ID
  IF invalid
     RETURN error "Invalid car ID"

  FIND car BY ID
     POPULATE user info
```

```
  IF car not found
     RETURN error "Car not found"

  IF user is not admin AND car.user != current user ID
     RETURN error "Not authorized"

  RETURN success response WITH car data

END FUNCTION
```

Update Car:

```
FUNCTION UpdateCar(request, response)

  READ car ID from request parameters
  READ updates FROM request body

  FIND car BY ID
  IF car not found
     RETURN error "Car not found"

  IF user is not admin AND car.user != current user ID
     RETURN error "Not authorized"

  IF numberPlate is updated AND different from current
     CHECK duplicate in Car collection
     IF duplicate exists
        RETURN error "Number plate already exists"

  COPY current images to updatedImages

  IF deletedIndex provided
     DELETE image at that index from cloud
     REMOVE image from updatedImages

  IF new files provided
     IF imageIndex provided
        DELETE old image at that index from cloud
        UPLOAD new image at same index
     ELSE
        UPLOAD all new images and append to updatedImages

  SET updates.images = updatedImages

  UPDATE car in database with updates

  RETURN success response WITH updated car

END FUNCTION
```

Delete Car:

```
FUNCTION DeleteCar(request, response)
```

```
  READ car ID from request parameters

  FIND car BY ID
  IF car not found
     RETURN error "Car not found"

  IF user is not admin AND car.user != current user ID
     RETURN error "Not authorized"

  FOR each image in car.images
     DELETE image from cloud storage

  DELETE car from database

  RETURN success response "Car deleted successfully"

END FUNCTION
```

7.5 AI Chat Controller

Ask AI:

```
FUNCTION askAI(request, response)

  READ prompt, userId FROM request body

  IF prompt OR userId is missing
     RETURN error "prompt and userId required"

  DEFINE SYSTEM_INSTRUCTION as FixKar AI instructions
     - Friendly automotive assistant
     - Handles cars, bikes, trucks, buses, electric/hybrid vehicles
     - Maintain polite, clear, professional tone
     - Respond in user's language (English, Urdu, or mixed)
     - Ask for missing info if query is incomplete
     - Only respond to vehicle-related queries
     - Use last 30 minutes of conversation history for context

  FIND user BY userId
  IF user not found
     RETURN error "User not found"

  SAVE user's message to AIChat collection as {userId, sender: "user", message: prompt}

  DEFINE thirtyMinutesAgo as current time minus 30 minutes

  RETRIEVE user's chat history from AIChat WHERE createdAt >= thirtyMinutesAgo
     SORT by createdAt ascending

  FORMAT history into messages array:
```

Get User Chats:

```
FUNCTION getUserChats(request, response)

  READ userId FROM request parameters

  FIND user BY userId
     EXCLUDE sensitive fields (password, OTP, OTP expiry, profileImagePublicId, location)

  IF user not found
     RETURN error "User not found"

  RETRIEVE all chats for userId from AIChat
     SORT by createdAt descending

  RETURN success response WITH user info and chats

END FUNCTION
```

7.6 Agendas

Initialize Agendas:

```
FUNCTION initializeAgenda()
   START Agenda job processor
   LOG "Job processing started"

   GET all pending requests from DB where:
      status = "pending" AND
      isExpired = false AND
      expiredAt > now

   FOR EACH pending request
      SCHEDULE "expire-request" job with requestId at request.expiredAt
      LOG "Scheduled pending request"
   END FOR
END FUNCTION
```

Define Expire Request Job:

```
AGENDA JOB "expire-request"(requestId)
   FIND request by requestId
   IF request not found OR request.status != "pending" OR request.isExpired = true
      RETURN

   MARK request as expired:
      status = "expired"
      isExpired = true
      expiredAt = now
      deletedAt = now
   SAVE request

   NOTIFY client via socket:
```

```
      emit "requestExpired" with request info

   CLEAN UP related chats and inbox:
      FIND chats for requestId
      DELETE inbox entries for those chats
      NOTIFY mechanics and clients to refresh unread count

   NOTIFY nearby mechanics:
      FIND verified, online mechanics near request location
      EMIT "requestRemoved" to mechanics

   CANCEL the Agenda job
END JOB
```

Schedule Expiry For A Request:

```
FUNCTION scheduleRequestExpiry(requestId, expiredAt)
   IF expiredAt < now
      LOG warning: cannot schedule past date
      RETURN

   CREATE Agenda job "expire-request" with requestId
   UNIQUE by requestId
   SCHEDULE job at expiredAt
   SAVE job
   LOG "Scheduled expiry for request"
END FUNCTION
```

Cancel Scheduled Expiry:

```
FUNCTION cancelRequestExpiry(requestId)
   CANCEL Agenda jobs WHERE:
      name = "expire-request" AND
      data.requestId = requestId
   LOG number of jobs cancelled
END FUNCTION
```

Shutdown Agenda Gracefully:

```
FUNCTION shutdownAgenda()
   STOP Agenda job processor
   LOG "Shutdown complete"
END FUNCTION
```

7.7 Request Controller

Quick Request:

```
FUNCTION quickRequest(req, res)
  READ clientId, serviceType, location, vehicleDetails FROM req.body

  IF any required field missing
    RETURN error "Required fields are missing"
```

```
  CHECK if client has any active request (pending, accepted, on_the_way, reached, in_progress)
    IF yes
      RETURN error "You already have an active request"

  UPLOAD vehicle images if provided (max 3)

  CREATE new Request with:
    - clientId, serviceType, vehicleDetails
    - location (lat/lng)
    - status = "pending"
    - vehicleImages
    - expiredAt = 30 minutes from now

  SAVE request
  POPULATE client details

  IF vehicleDetails includes carId
    ATTACH car info to request

  SCHEDULE request expiry after 30 minutes

  FIND nearby mechanics within 5km
    - filter by onlineStatus, isVerified
    - filter vehicle type based on serviceType
    - filter car categories if serviceType = "car"

  CALCULATE distance of each mechanic from request location

  IF no nearby mechanics
    RETURN response "No mechanics available nearby"

  NOTIFY nearby mechanics via Socket.IO about new request

  RETURN success with request details and nearby mechanics info
END FUNCTION
```

Get NearBy Requests:

```
FUNCTION getNearByRequests(req, res)
  READ mechanicId FROM req.params

  FIND mechanic by ID
  IF not found OR offline
    RETURN error

  GET mechanic location (lat/lng)

  FIND pending requests within 5km of mechanic
    - separate bike and car requests
    - for cars, match car type and mechanic's carCategories

  ATTACH car details to car requests
```

```
  CALCULATE distance of each request from mechanic

  RETURN success with list of nearby requests
END FUNCTION
```

Distance Function:

```
FUNCTION getDistance(lat1, lon1, lat2, lon2)
   SET R = 6371 * 1000

   SET dLat = (lat2 - lat1) * PI / 180
   SET dLon = (lon2 - lon1) * PI / 180

   SET a = SIN(dLat / 2)^2
         + COS(lat1 * PI / 180) * COS(lat2 * PI / 180) * SIN(dLon / 2)^2

   SET c = 2 * ATAN2(SQRT(a), SQRT(1 - a))

   RETURN R * c
END FUNCTION
```

Update Request Status:

```
FUNCTION updateRequestStatus(req, res)
  READ id, status, mechanicId FROM req.body
  GET userId and role FROM authenticated user

  VALIDATE status against allowed list

  FIND request by ID
  IF not found
    RETURN error

  IF role = "client"
    ALLOW only "cancelled"
    CHECK if request is completed or expired
    UPDATE request status to cancelled
    NOTIFY assigned mechanic and nearby mechanics via Socket.IO
    RETURN success

  IF role = "mechanic"
    IF status = "accepted"
      CHECK mechanicId is provided
      CHECK mechanic has no other active requests
      ASSIGN request to mechanic and update status to "accepted"
      CANCEL expiry schedule
      NOTIFY client and mechanic via Socket.IO
      NOTIFY other nearby mechanics to remove this request
      RETURN success

    ELSE
      CHECK mechanic is assigned to this request
      VALIDATE status transition based on allowed workflow:
        pending → accepted
```

```
      accepted → on_the_way / cancelled
      on_the_way → reached
      reached → in_progress
      in_progress → completed
    UPDATE request status
    NOTIFY client and mechanic via Socket.IO
    RETURN success

  RETURN error if role is invalid
END FUNCTION
```

Get Client Active Request:

```
FUNCTION getClientActiveRequest(req, res)
  READ clientId FROM req.params

  FIND active request for client (pending, accepted, on_the_way, reached, in_progress)
    POPULATE client and mechanic details

  IF serviceType = car
    GET car details
    FILTER nearby mechanics by carType and mechanic carCategories
  ELSE
    FILTER nearby mechanics by vehicleType

  RETURN active request details, car info, nearby mechanics
END FUNCTION
```

Get Mechanic Active Request:

```
FUNCTION getMechanicActiveRequest(req, res)
  READ mechanicId FROM req.params

  FIND active request assigned to mechanic (accepted, on_the_way, reached, in_progress)
    POPULATE client and mechanic info

  IF serviceType = car
    GET car details

  RETURN active request details with car info
END FUNCTION
```

7.8 Message Controller

Create Chat:

```
FUNCTION createChat(req, res)
  READ requestId, senderId, receiverId, senderType, receiverType FROM req.body

  SEARCH Chat where requestId matches AND participants include both senderId and receiverId

  IF chat exists
    RETURN existing chatId
```

```
  ELSE
    CREATE new Chat with:
      - requestId
      - participants: sender and receiver (with types)

    UPDATE Request to add senderId to chatEnabledMechanics

    EMIT Socket.IO event "bookingUpdated" to notify about chat-enabled mechanics

    RETURN newly created chatId
END FUNCTION
```

Get Messages:

```
FUNCTION getMessages(req, res)
  READ chatId FROM req.params

  FIND all messages where chatId matches
    POPULATE sender and receiver details (fullName, profileImage, role)
    SORT messages by createdAt descending

  IF no messages found
    RETURN "No messages found"

  RETURN messages
END FUNCTION
```

Send Message:

```
FUNCTION sendMessage(req, res)
  READ chatId FROM req.params
  READ senderId, receiverId, text, images, senderType, receiverType FROM req.body

  VALIDATE required fields

  FIND chat by chatId
  IF chat not found
    RETURN error "Chat not found"

  CREATE Message with:
    - chatId
    - senderId, receiverId
    - senderType, receiverType
    - text or images

  POPULATE message sender and receiver details

  UPDATE Inbox for receiver:
    - lastMessage = text OR "Image/Attachment"
    - lastMessageAt = message timestamp
    - increment unreadCount by 1

  UPDATE Inbox for sender:
```

```
    - lastMessage = text OR "Image/Attachment"
    - lastMessageAt = message timestamp
    - unreadCount = 0

  EMIT Socket.IO events:
    - "newMessage" to chat room (chatId)
    - "newMessage" to receiver inbox
    - "refreshUnreadCount" to receiver

  RETURN created message
END FUNCTION
```

7.9 Inbox Controller

Get Inbox:

```
FUNCTION getInbox(userId)
  FIND Inbox where userId matches
    POPULATE otherUserId (name, image, role)
    POPULATE requestId where status NOT in ["completed", "expired", "cancelled"]
    SORT by lastMessageAt DESC

  FILTER inbox where requestId exists
  FORMAT inbox items to include:
    - chatId, request, otherUser details
    - lastMessage, unreadCount, timestamps

  RETURN formattedInbox
END FUNCTION
```

Mark Chat As Read:

```
FUNCTION markChatAsRead(userId, chatId)
  UPDATE Inbox document where userId AND chatId match
    SET unreadCount = 0

  RETURN success message
END FUNCTION
```

Get Unread Count:

```
FUNCTION getUnreadCount(userId)
  IF userId not provided
    RETURN error "User Id is required"

  COUNT Inbox documents where userId matches AND unreadCount > 0

  RETURN unreadChatsCount
END FUNCTION
```

7.10 Admin Controller

Register:

```
FUNCTION adminRegister(data, file)
  IF any required field missing
    RETURN error "All fields required"

  CHECK for existing email, phone, CNIC
    IF exists, RETURN error

  HASH password
  IF file exists, upload profile image

  SAVE new Admin
  RETURN success message
END FUNCTION
```

Get All Admins:

```
FUNCTION getAllAdmins()
  FIND all Admins
  IF none found
    RETURN error
  RETURN admins
END FUNCTION
```

Get Admin By Id:

```
FUNCTION getAdminById(id)
  FIND Admin by id
  IF not found
    RETURN error
  RETURN admin data
END FUNCTION
```

Update Admin:

```
FUNCTION updateAdmin(id, data, file)
  FIND admin by id
  IF not found RETURN error

  CHECK for email/phone/CNIC duplicates
  UPDATE fields
  IF file exists, upload profile image
  SAVE admin
  RETURN success
END FUNCTION
```

Delete Admin:

```
FUNCTION deleteAdmin(id)
  FIND admin by id
```

```
  IF not found RETURN error

  DELETE admin
  RETURN success message
END FUNCTION
```

7.11 Sockets

Configurations:

```
INITIALIZE io as NULL

FUNCTION initSocket(server, app)
   CREATE new Socket.IO server with CORS enabled
   ASSIGN io instance

   STORE io instance in app for global access

   ON new socket connection
      PRINT socket connected with socket ID

      INITIALIZE typingInfo
         chatId = null
         userId = null
         isTyping = false

      ON joinClientRoom(clientId)
         ADD socket to room clientId
         PRINT client joined room

      ON joinRoom(mechanicId)
         ADD socket to room mechanicId
         PRINT mechanic joined room

      ON joinMechanicRoom(mechanicId)
         ADD socket to room "mechanic_mechanicId"
         PRINT joined mechanic room

      ON leaveMechanicRoom(mechanicId)
         REMOVE socket from "mechanic_mechanicId"
         PRINT left mechanic room

      CALL handleMechanicSockets(io, socket)

      ON joinChatRoom(chatId)
         ADD socket to room chatId
         PRINT joined chat room

      ON sendMessage(roomId, message)
         SEND "newMessage" event to roomId

      ON leaveChatRoom(chatId)
         IF chatId exists
```

```
            REMOVE socket from chatId
            PRINT left chat room

      ON joinInbox(userId)
         ADD socket to room "inbox_userId"
         PRINT joined inbox

      ON leaveInbox(userId)
         IF userId exists
            REMOVE socket from "inbox_userId"
            PRINT left inbox

      ON typing(chatId, userId)
         UPDATE typingInfo (chatId, userId, isTyping = true)
         NOTIFY other users in chat room that user is typing

      ON stopTyping(chatId, userId)
         SET typingInfo.isTyping = false
         NOTIFY other users in chat room that user stopped typing

      ON disconnect
         PRINT socket disconnected
         IF user was typing
            NOTIFY chat room that user stopped typing
   END ON
END FUNCTION
```

Mechanic Sockets:

```
IMPORT Mechanic model
IMPORT Request model
IMPORT Car model

FUNCTION handleMechanicSockets(io, socket)

   ON event "mechanicStatusChange" with (mechanicId, onlineStatus)

      TRY
         FIND mechanic by mechanicId
         IF mechanic does not exist
            EXIT function

         GET current time (now)
         EXTRACT longitude and latitude from mechanic location

         FIND all active requests WHERE
            status = "pending"
            isExpired = false
            expiredAt > now
            location is within 5km radius of mechanic
         POPULATE client details

         INITIALIZE empty list validRequests
```

```
        FOR EACH request in activeRequests

          IF request serviceType is "bike"
            IF mechanic supports bike
              ADD request to validRequests

          IF request serviceType is "car" AND carId exists
            FIND car by carId
            IF car not found
              CONTINUE loop

            CHECK if mechanic supports car
            CHECK if mechanic supports car category

            IF both conditions are true
              ADD request to validRequests

        FOR EACH request in validRequests
          EMIT "nearbyMechanicUpdated" event TO client room
            SEND mechanicId
            SEND onlineStatus
            SEND mechanicData

      CATCH error
        LOG mechanic update error

END FUNCTION
```

7.12 Images Via Cloudinary

Upload and Delete Images:

```
IMPORT cloudinary configuration

FUNCTION streamUpload(fileBuffer)
  RETURN new Promise

      CREATE upload stream with Cloudinary
        SET upload folder = "uploads"
        ENABLE original filename usage
        ENABLE unique filename generation

      IF upload error occurs
        REJECT promise with error
      ELSE
        RESOLVE promise with upload result

      SEND file buffer to Cloudinary stream
END FUNCTION
```

8. Design Diagrams:

8.1 Class Diagram:

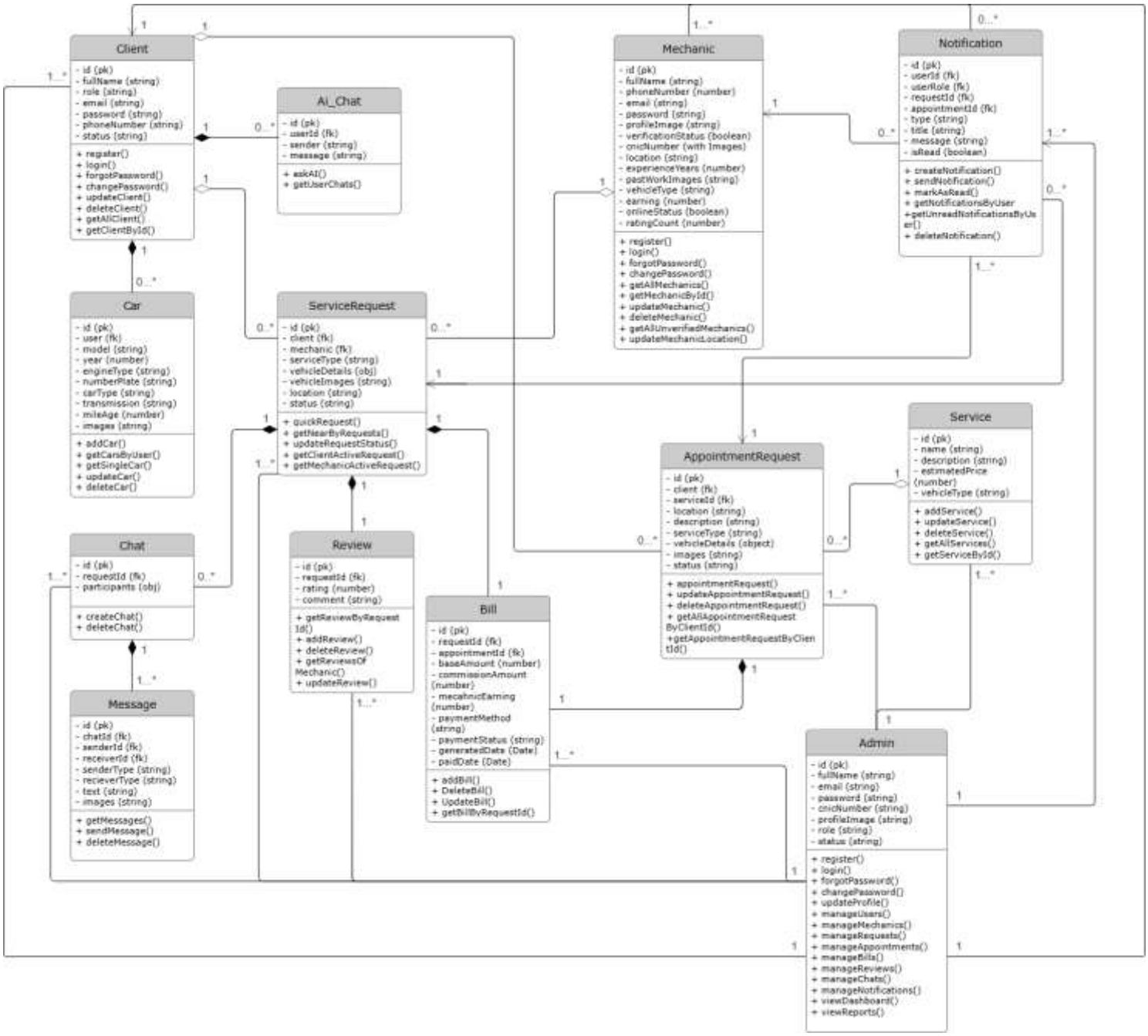


Figure 54: Class Diagram

8.2 State Chart Diagram

8.2.1 Admin State Chart Diagram

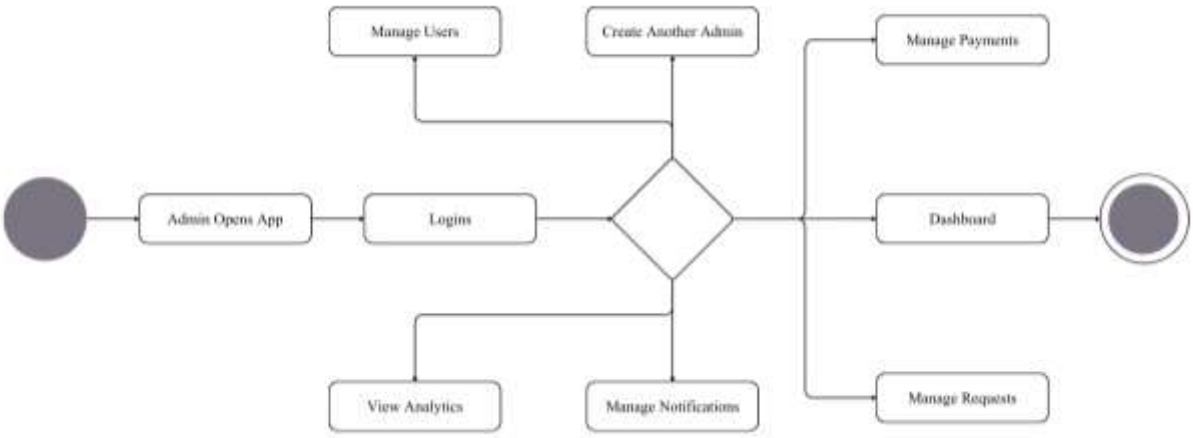


Figure 55: Admin - State Chart Diagram

8.2.2 Client State Chart Diagram

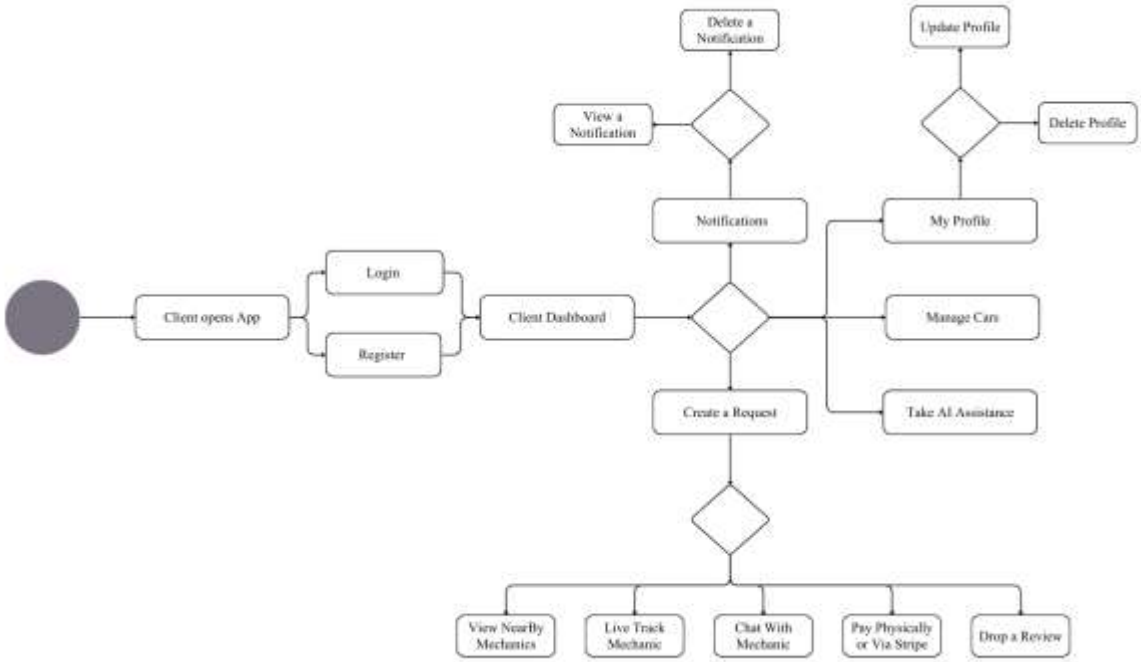


Figure 56: Client - State Chart Diagram

8.2.3 Mechanic State Chart Diagram

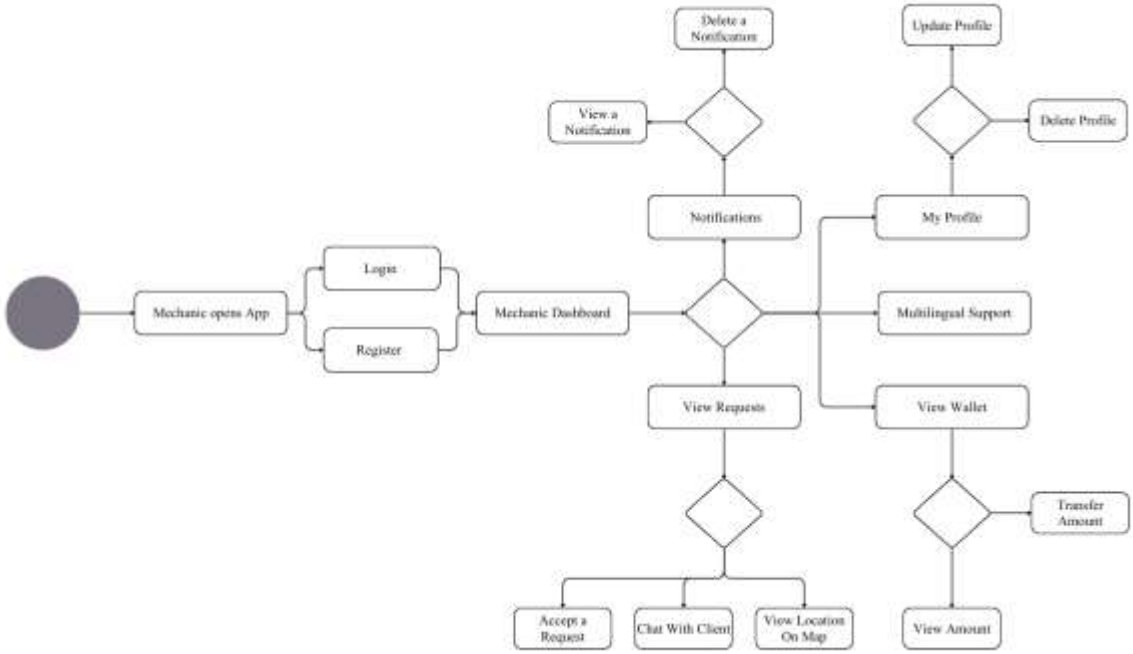


Figure 57: Mechanic - State Chart Diagram

8.3 Sequence Diagrams

8.3.1 Client SignUp and Login

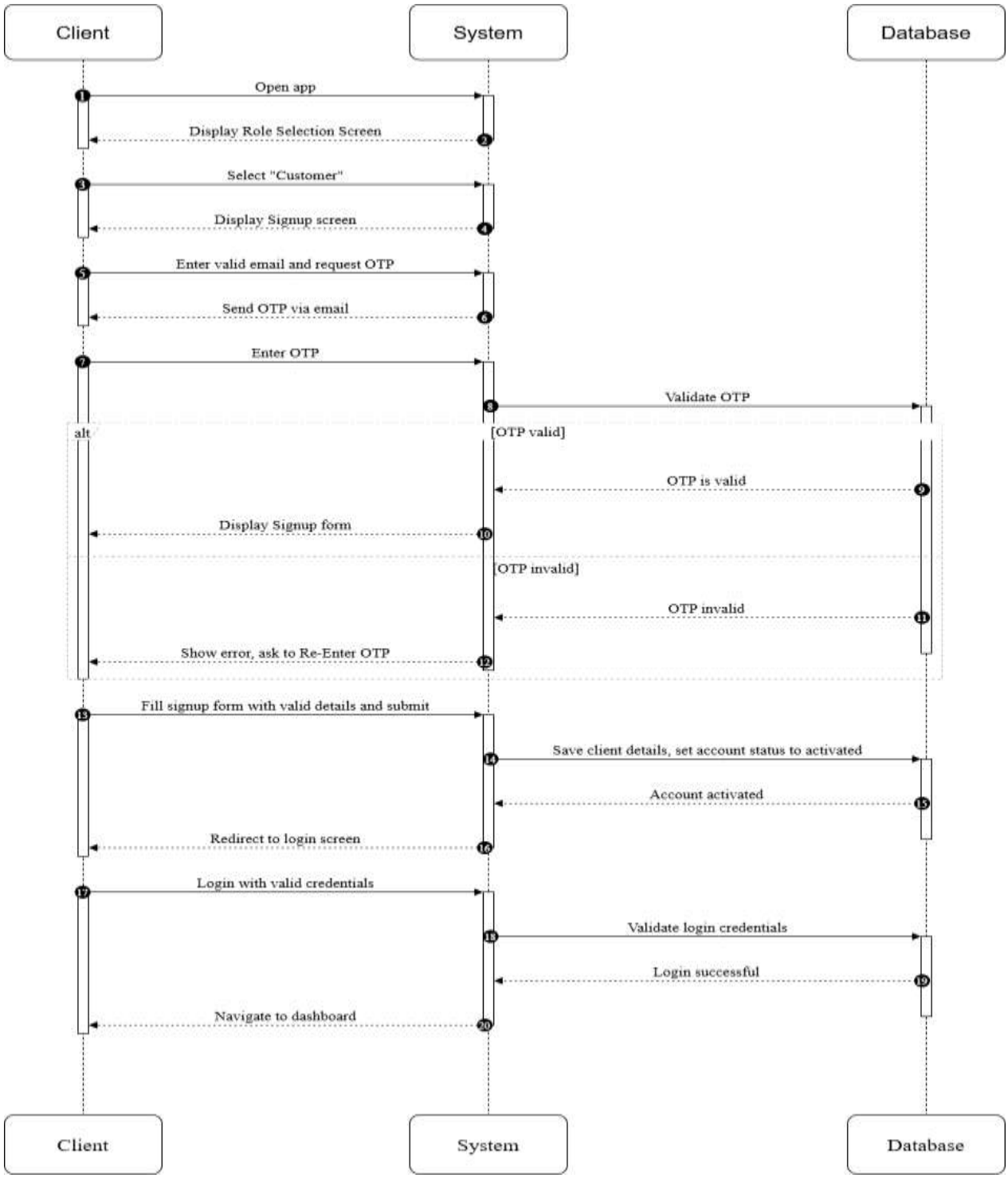


Figure 58: Client SignUp and Login - Sequence Diagram

8.3.2 Mechanic SignUp and Login

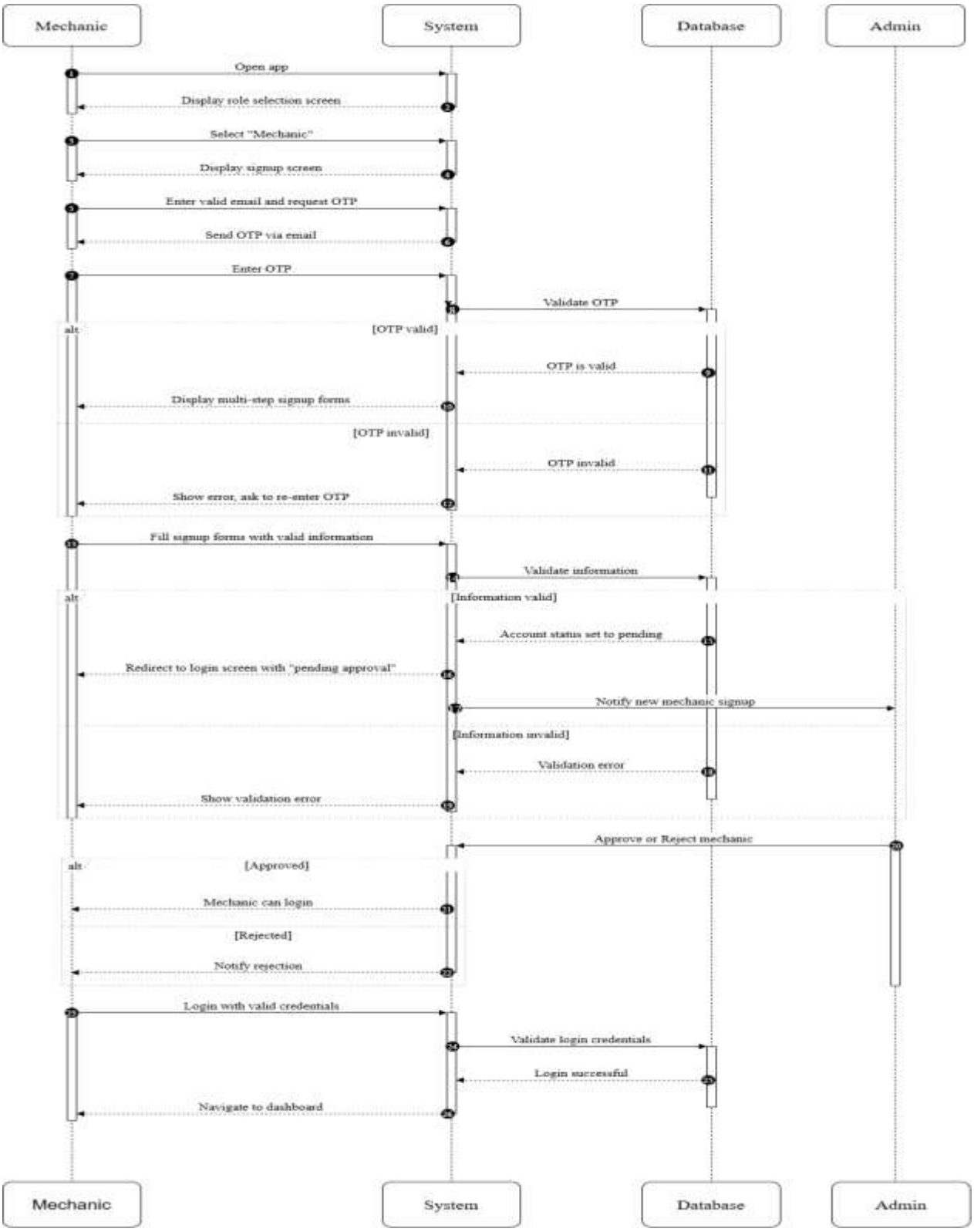


Figure 59: Mechanic SignUp and Login - Sequence Diagram

8.3.3 Add a Car

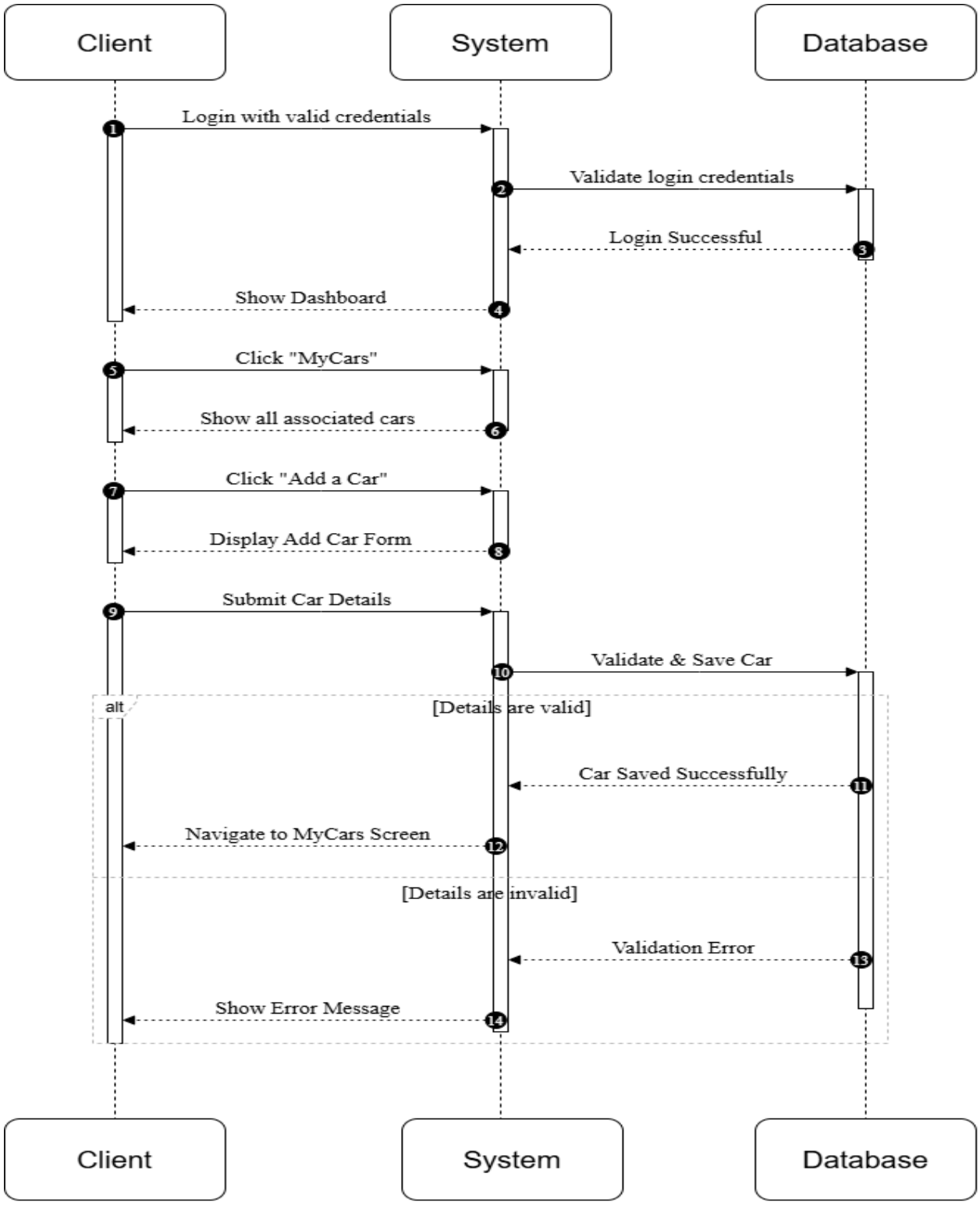


Figure 60: Add a Car - Sequence Diagram

8.3.4 View All Cars

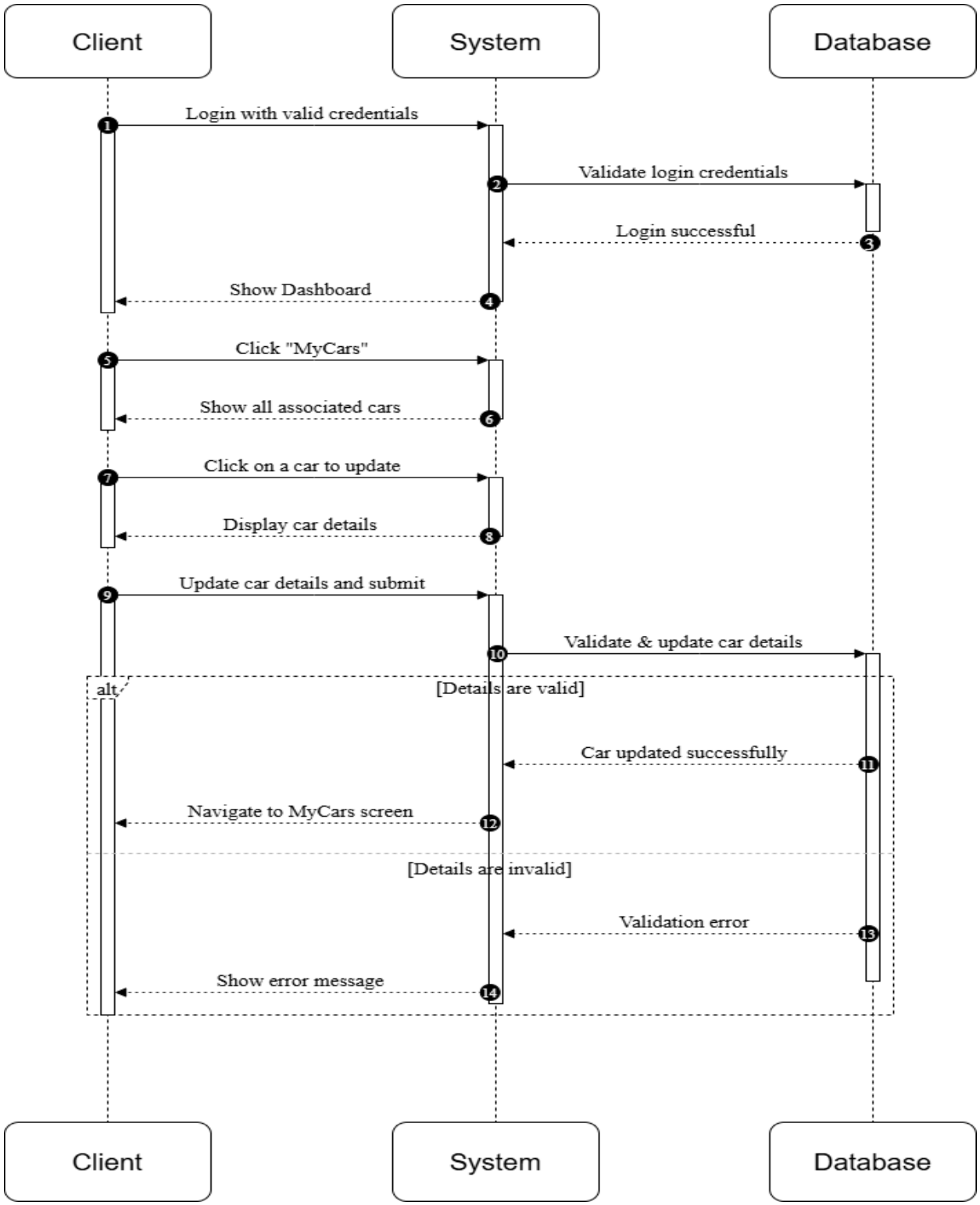


Figure 61: View All Cars – Sequence Diagram

8.3.5 Delete a Car

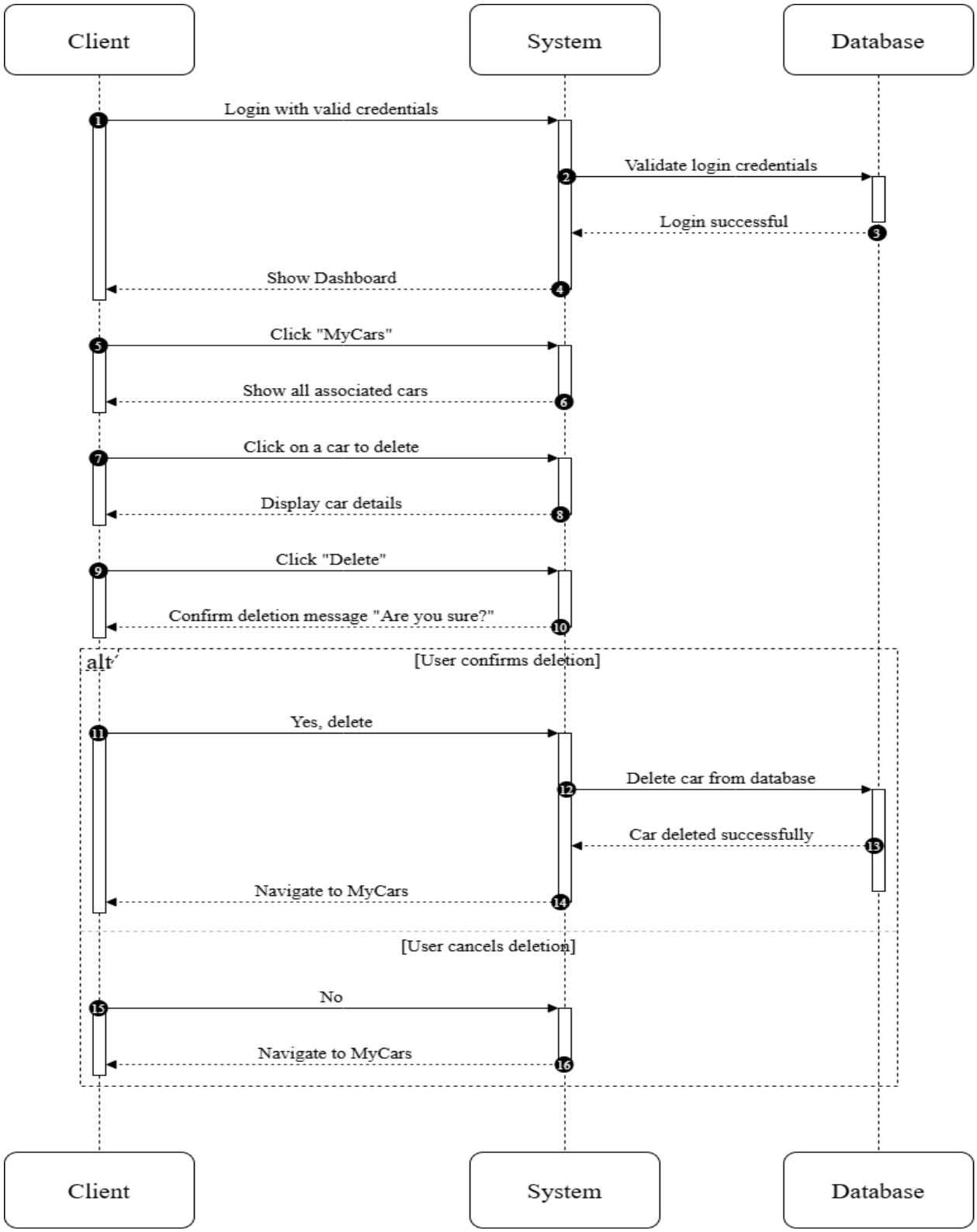


Figure 62: Delete a Car - Sequence Diagram

8.3.6 Forgot Password

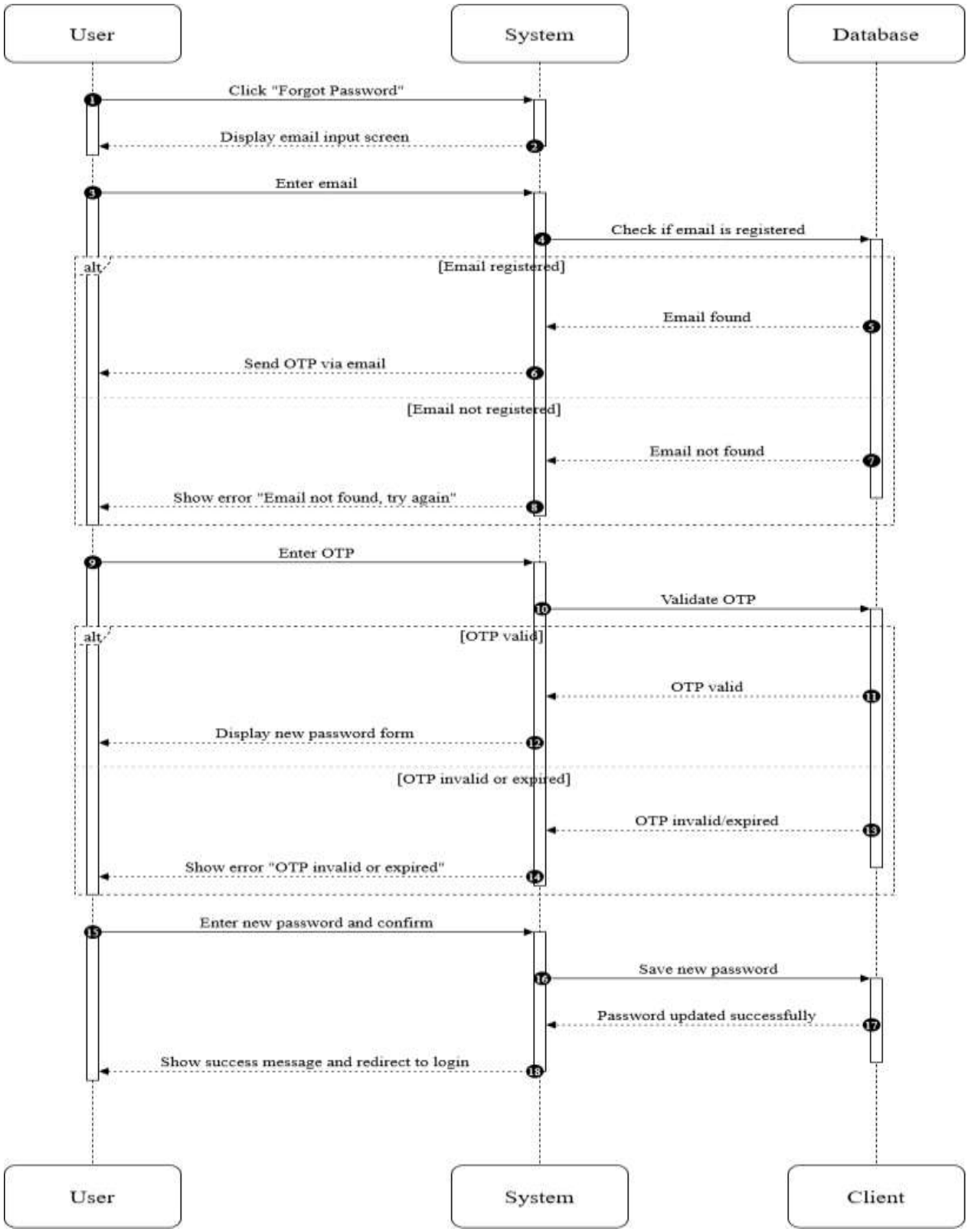


Figure 63: Forgot Password - Sequence Diagram

8.3.7 AI Chat

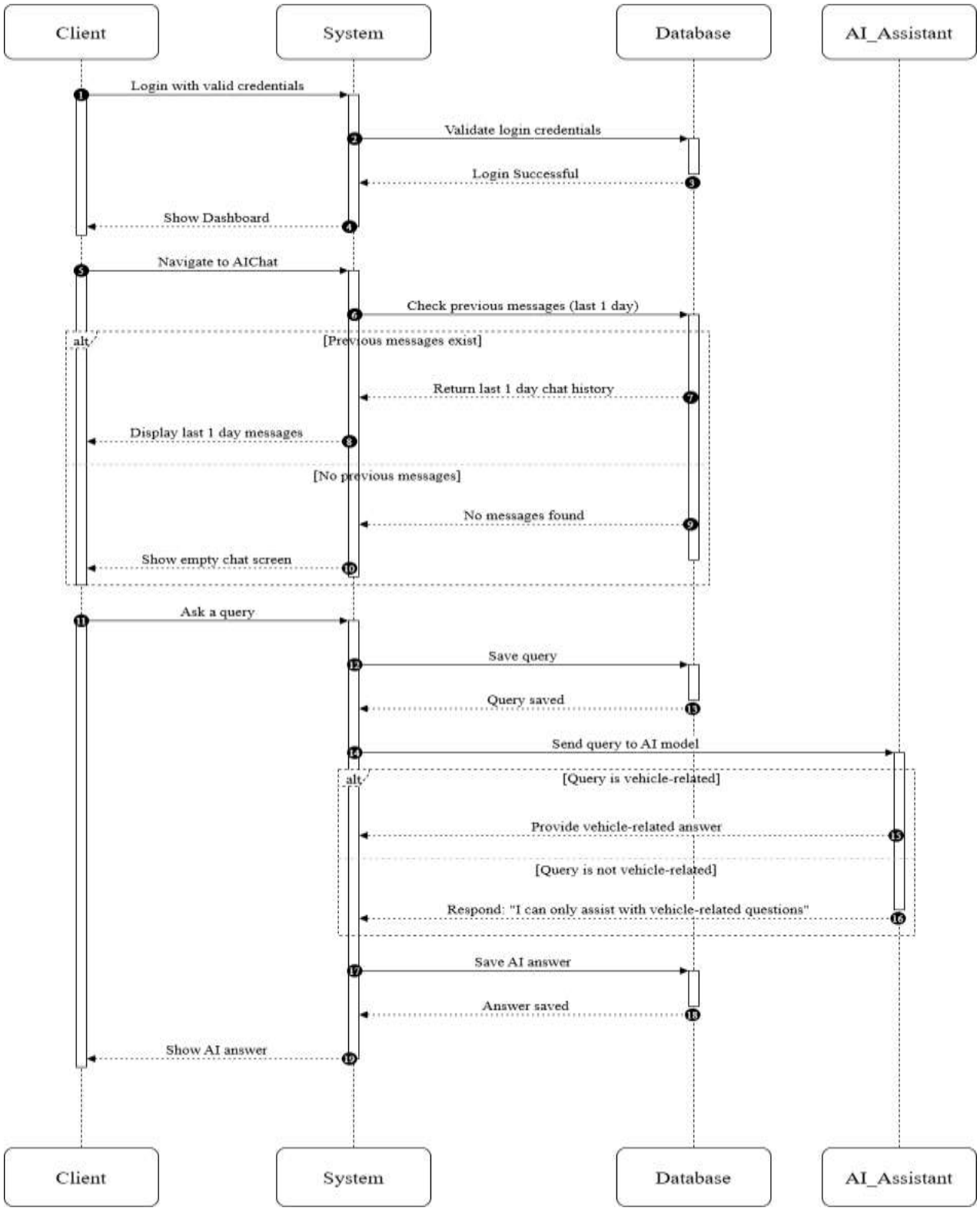


Figure 64: AI Chat - Sequence Diagram

8.3.8 Service Booking Flow

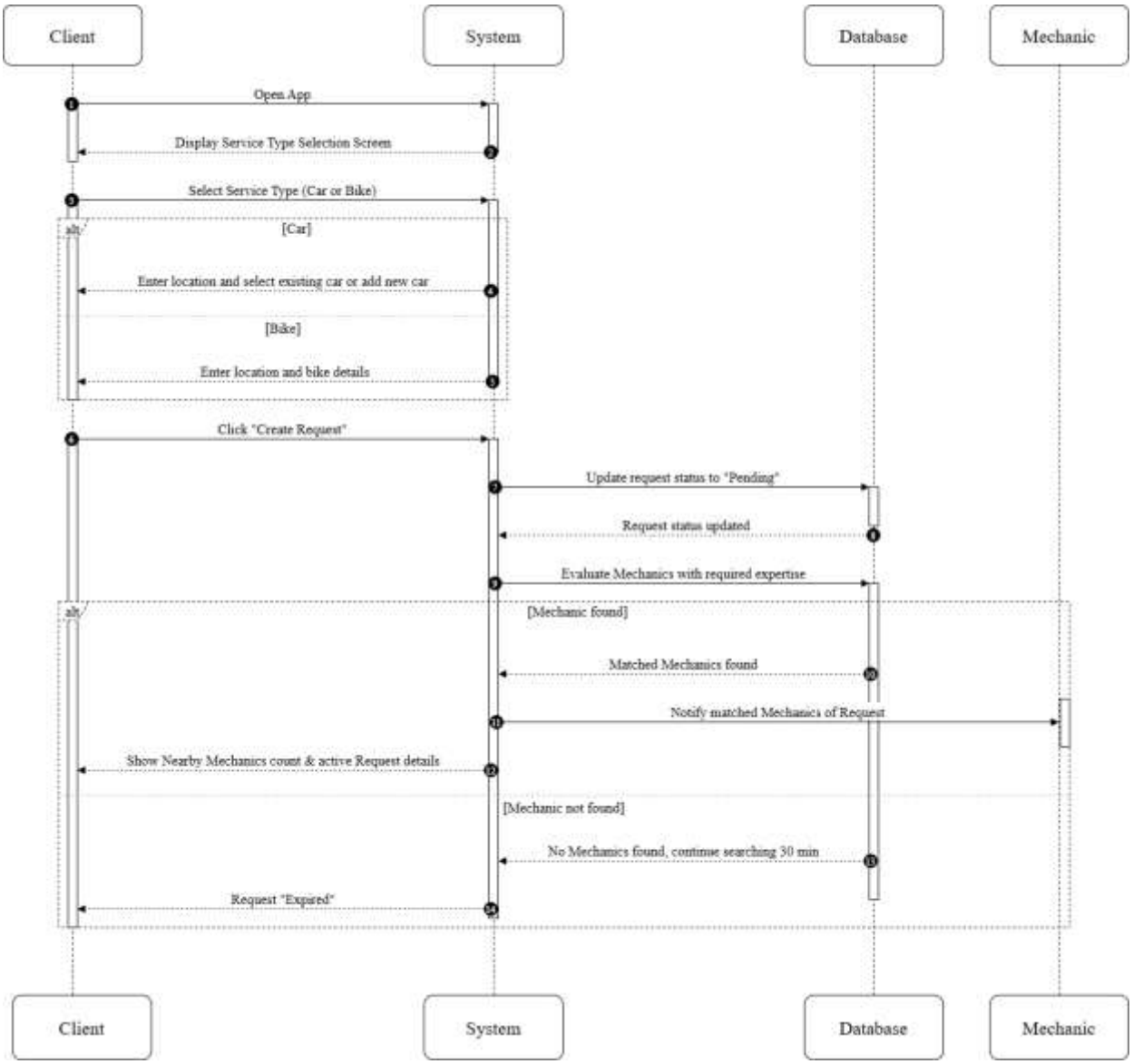


Figure 65: Service Booking Flow - Sequence Diagram

8.3.9 Chat Module

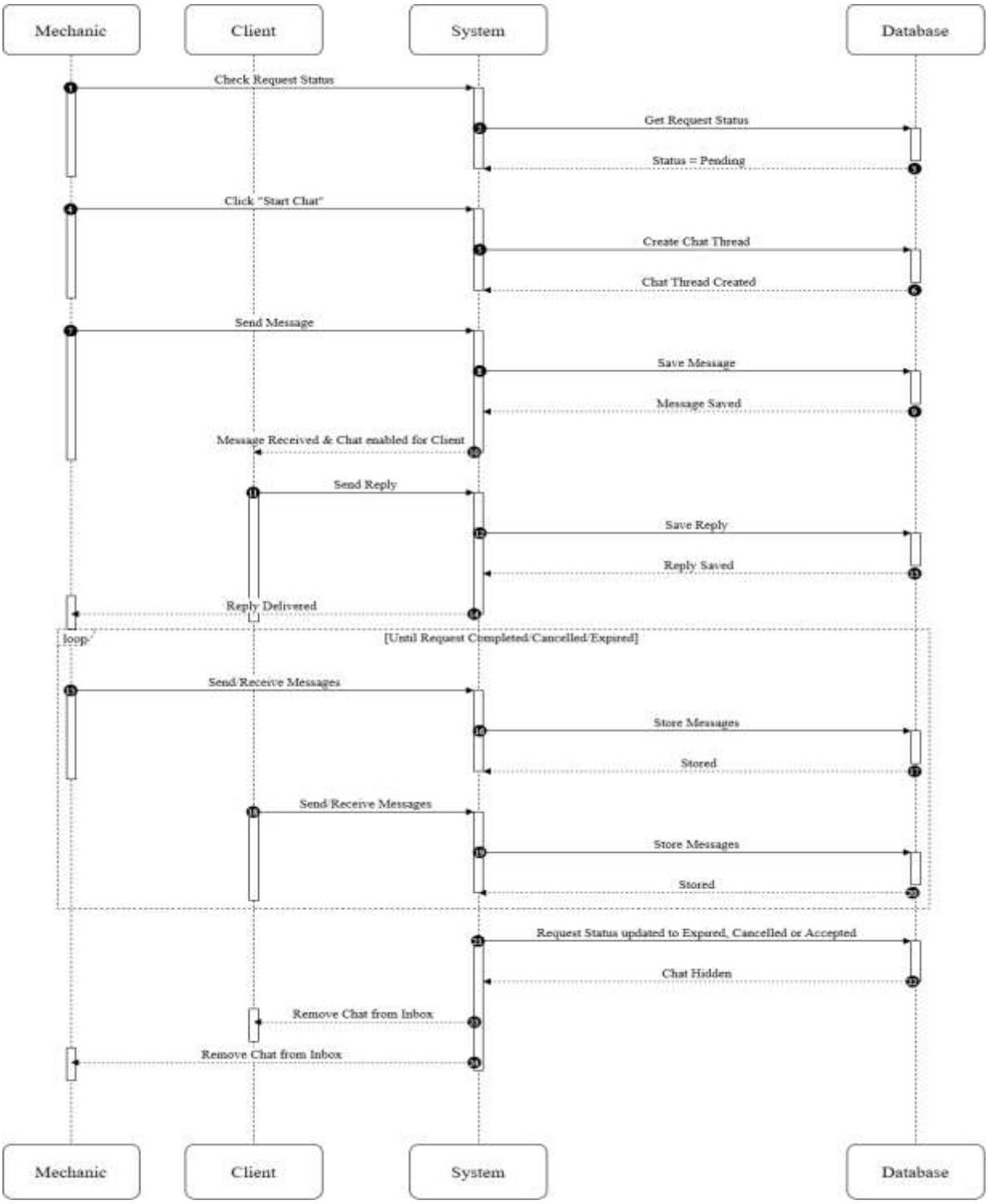


Figure 66: Chat Module - Sequence Diagram

8.3.10 Live Track Module

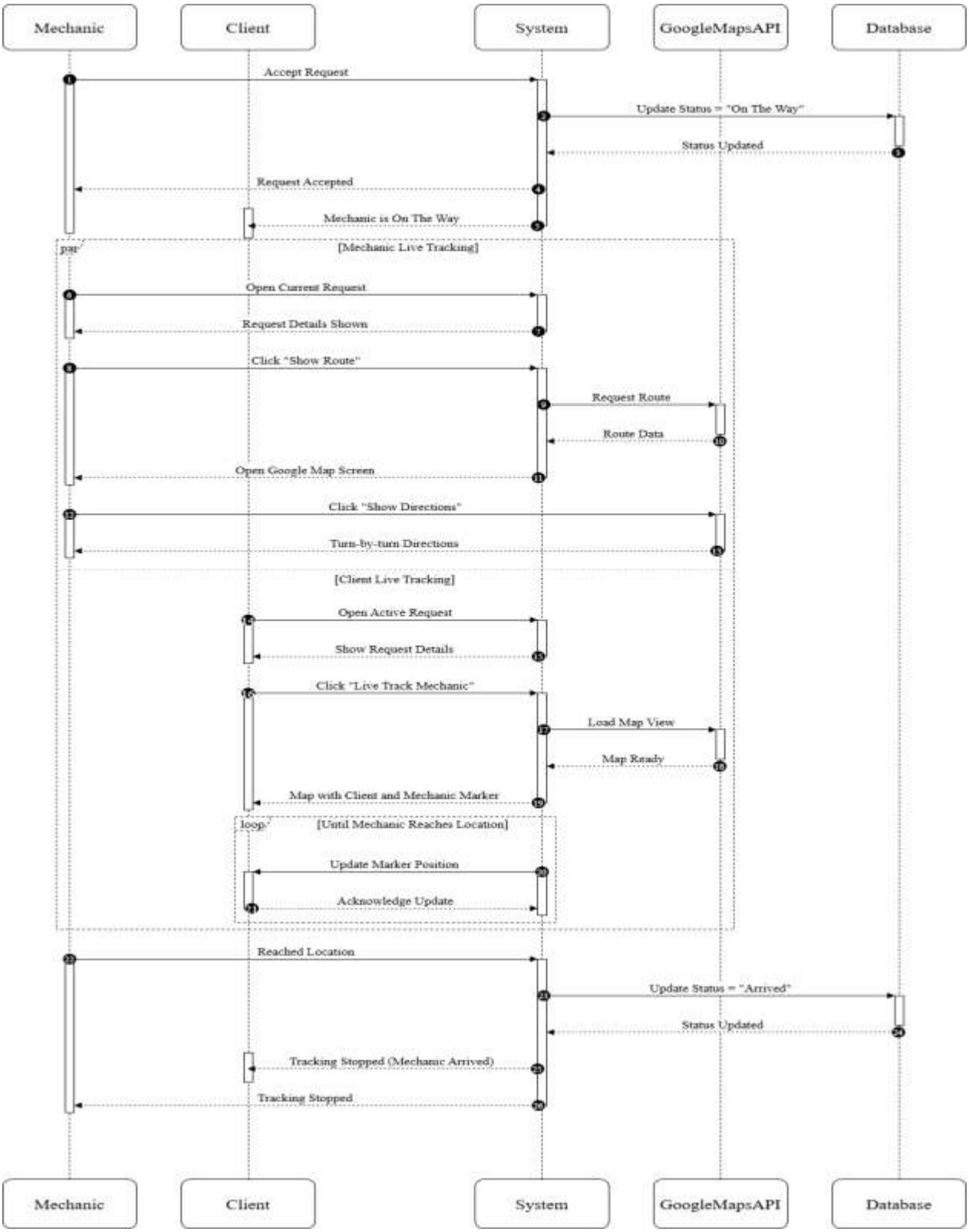


Figure 67: Live Track Module - Sequence Diagram

8.3.11 Payment Module

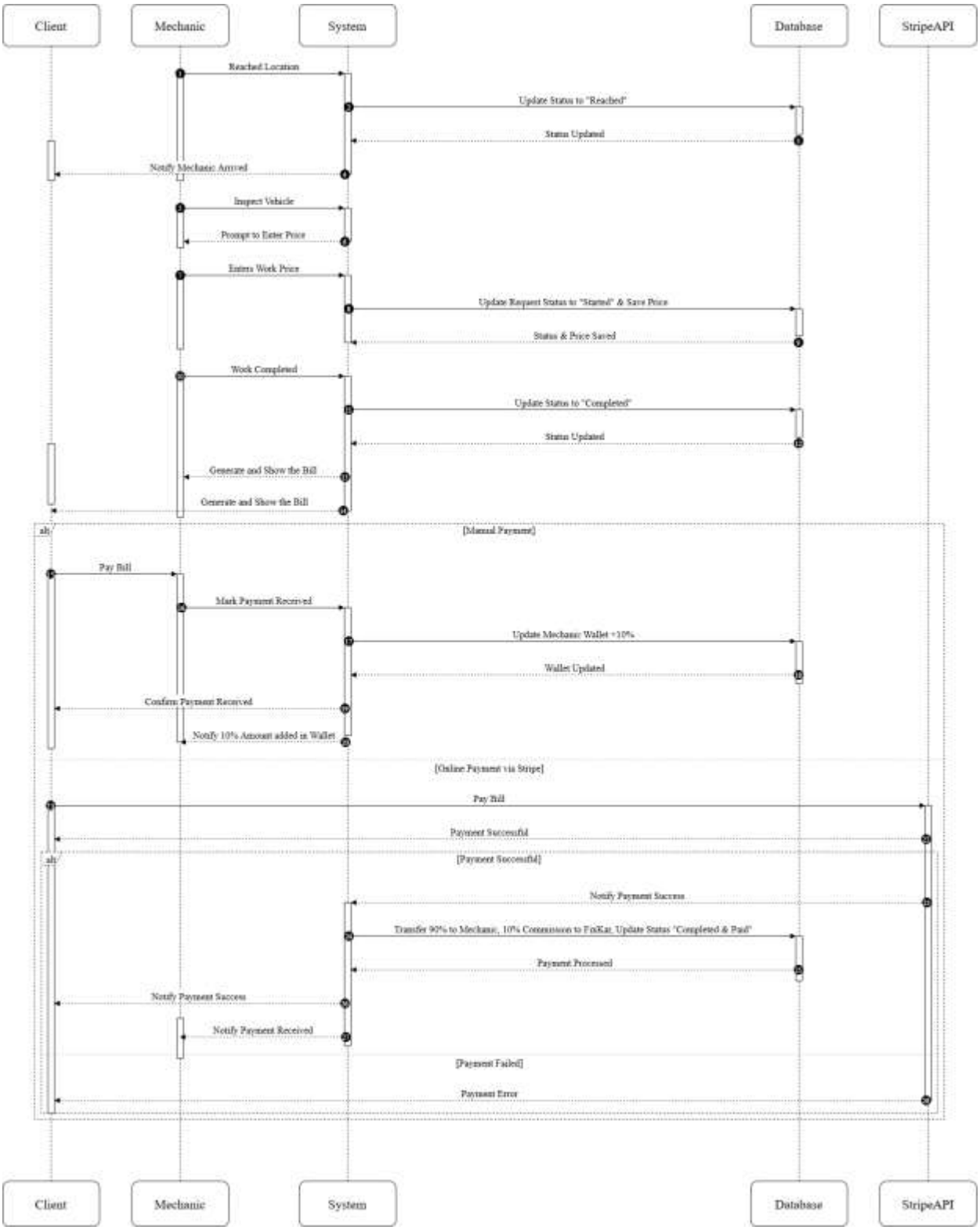


Figure 68: Payment Module - Sequence Diagram

8.3.12 Review and Rating Module

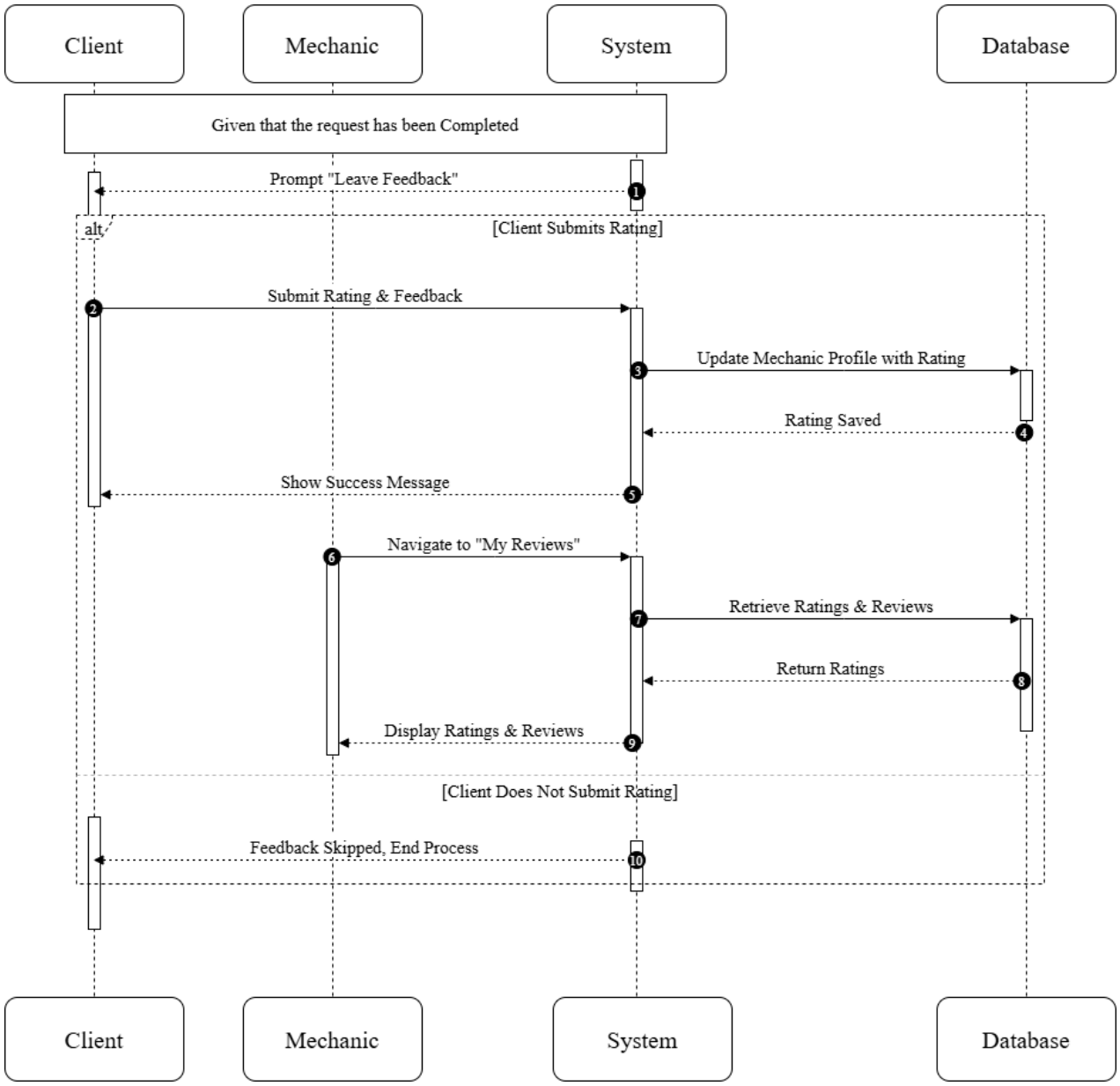


Figure 69: Review and Rating Module - Sequence Diagram

8.3.13 Multilingual Support Module

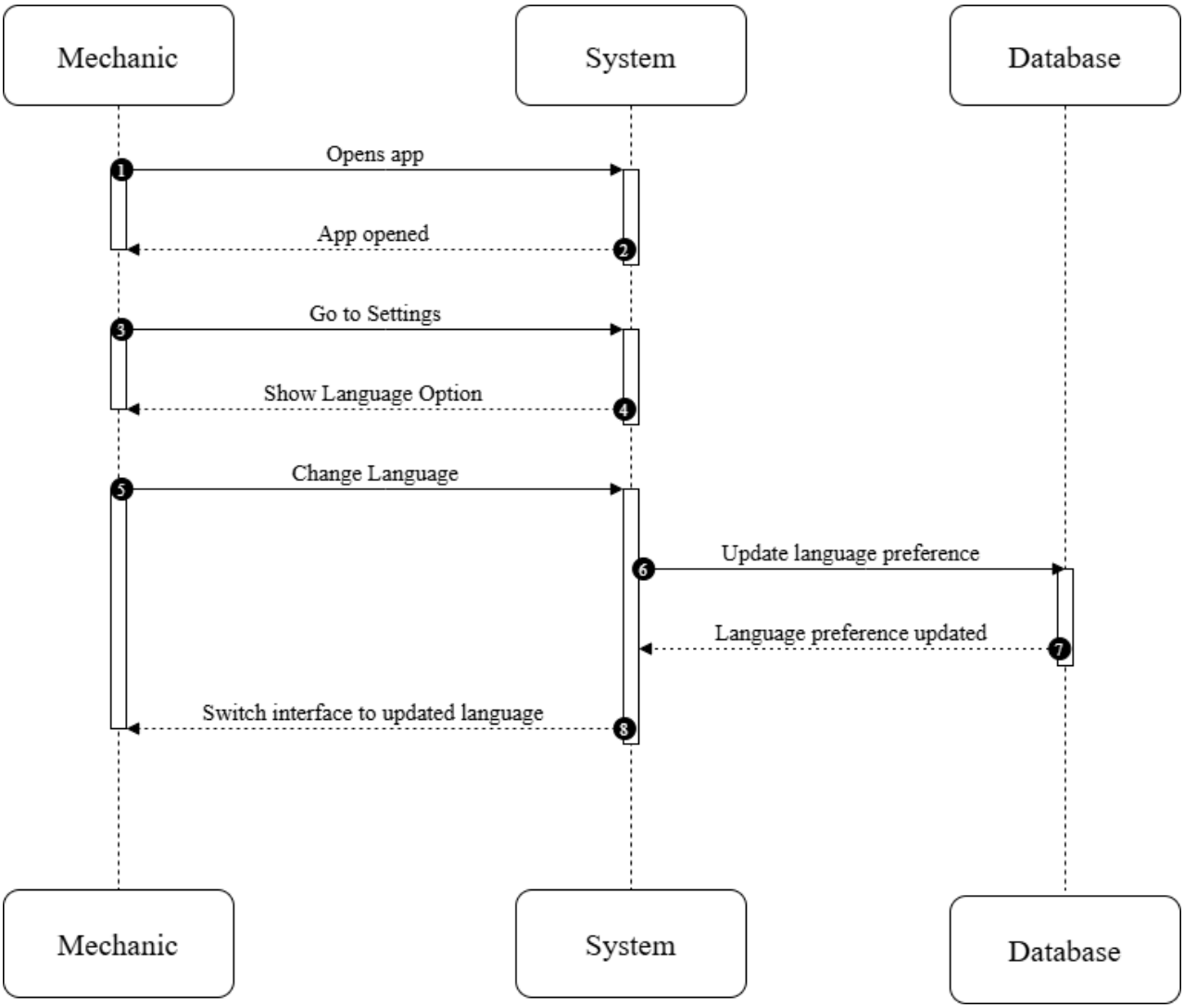


Figure 70: Multilingual Support Module - Sequence Diagram

8.3.14 Notification Management

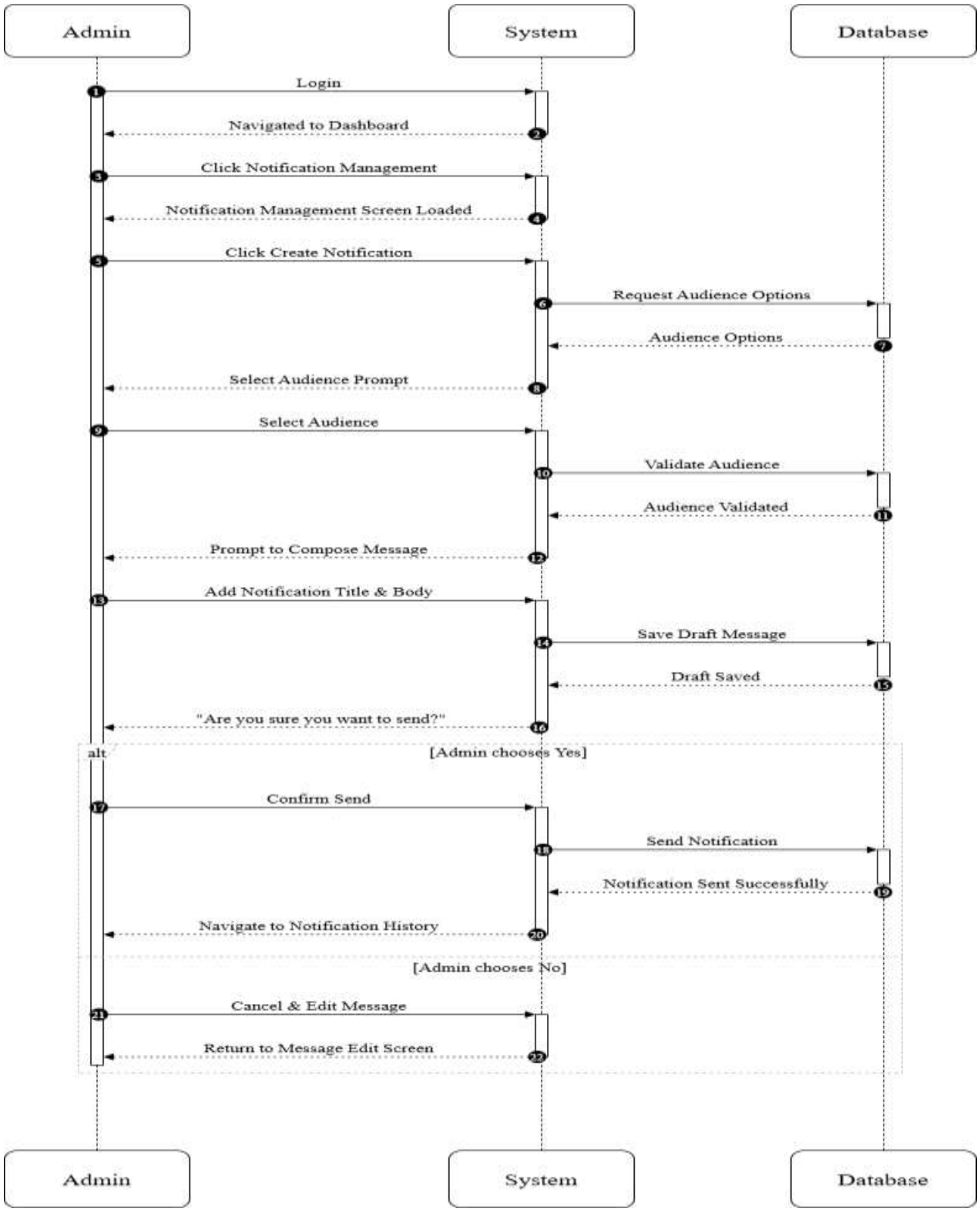


Figure 71: Notification Management - Sequence Diagram

8.3.15 Create New Admin

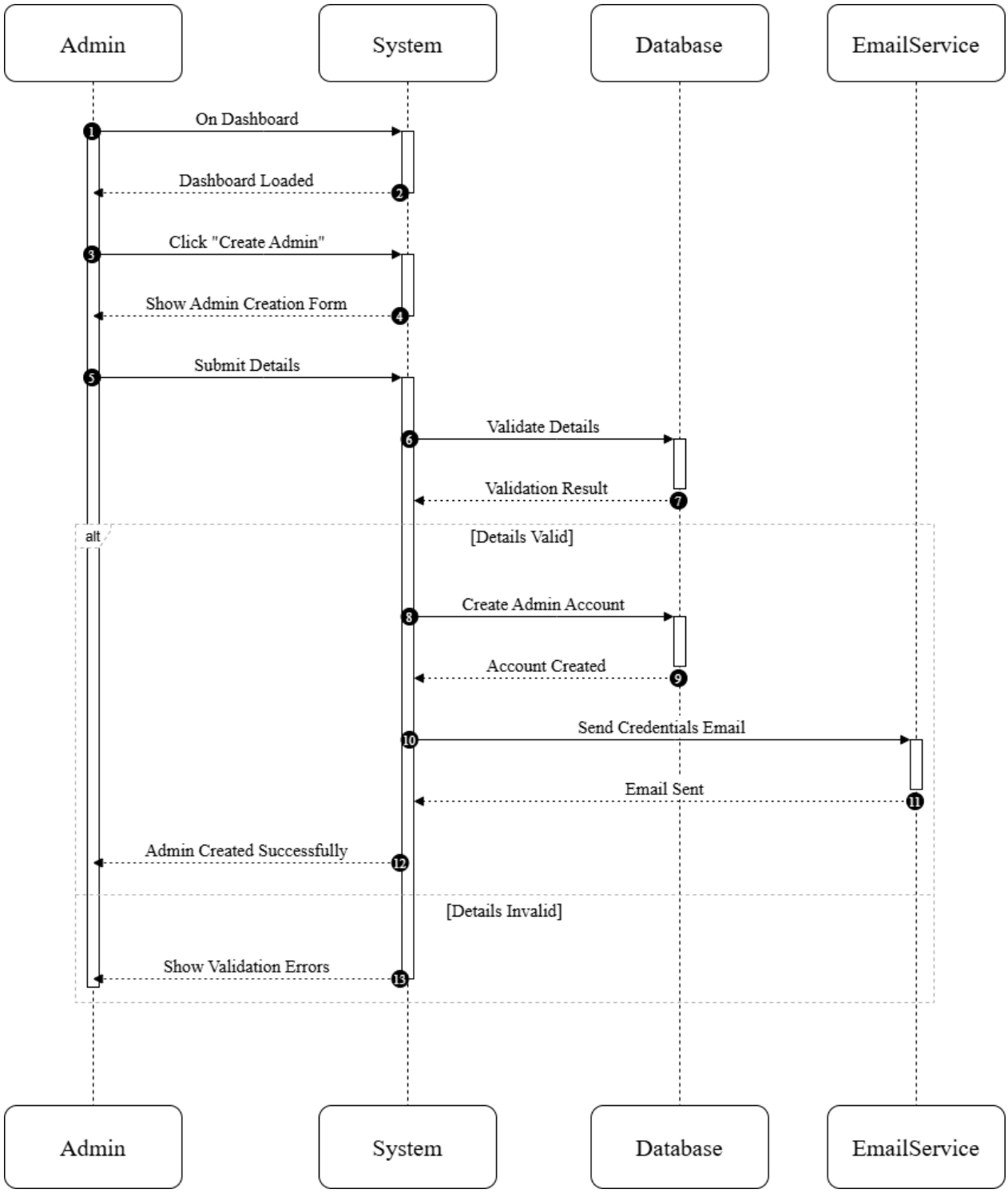


Figure 72: Create New Admin - Sequence Diagram

8.3.16 Payment Management

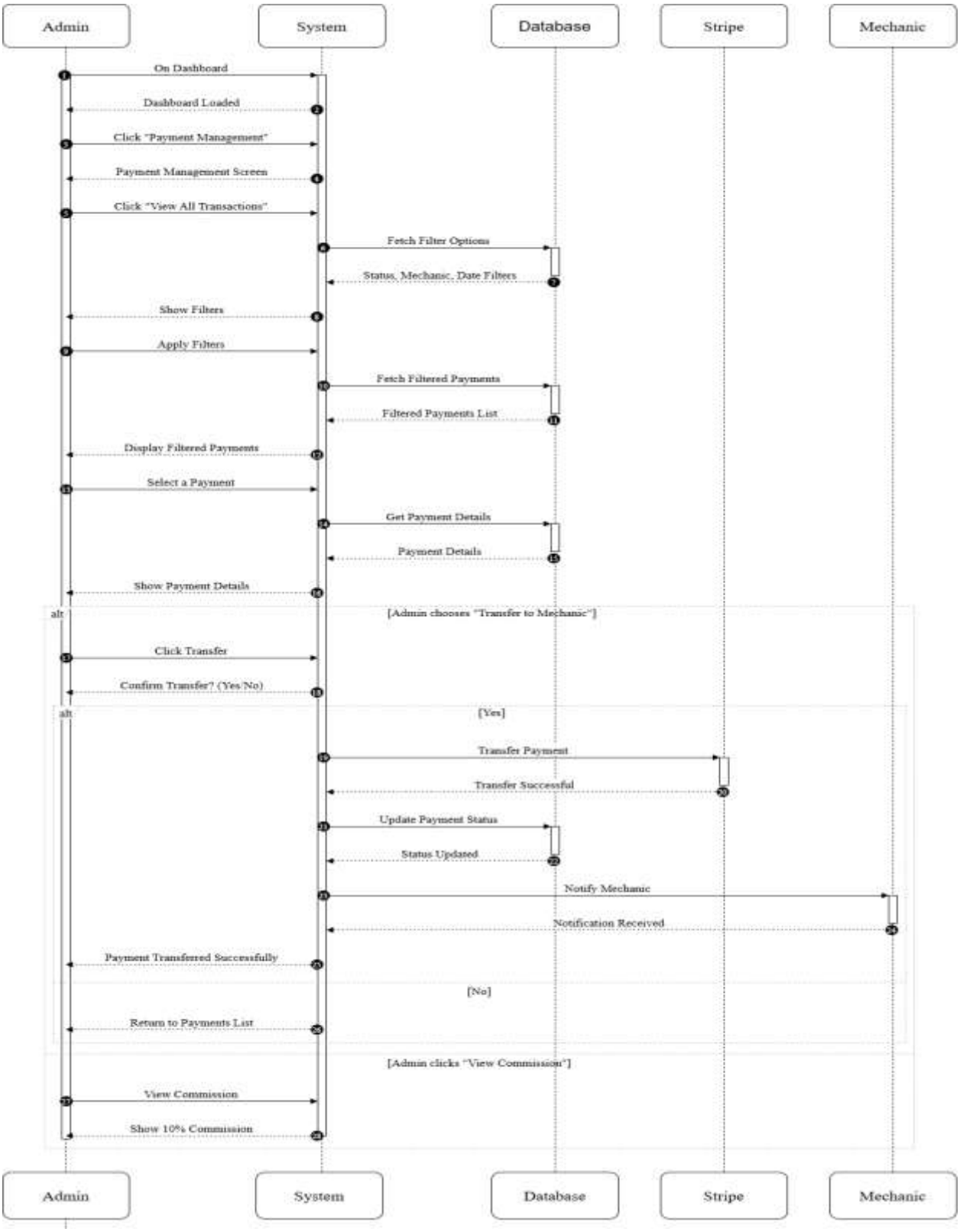


Figure 73: Payment Management - Sequence Diagram

8.3.17 User Management

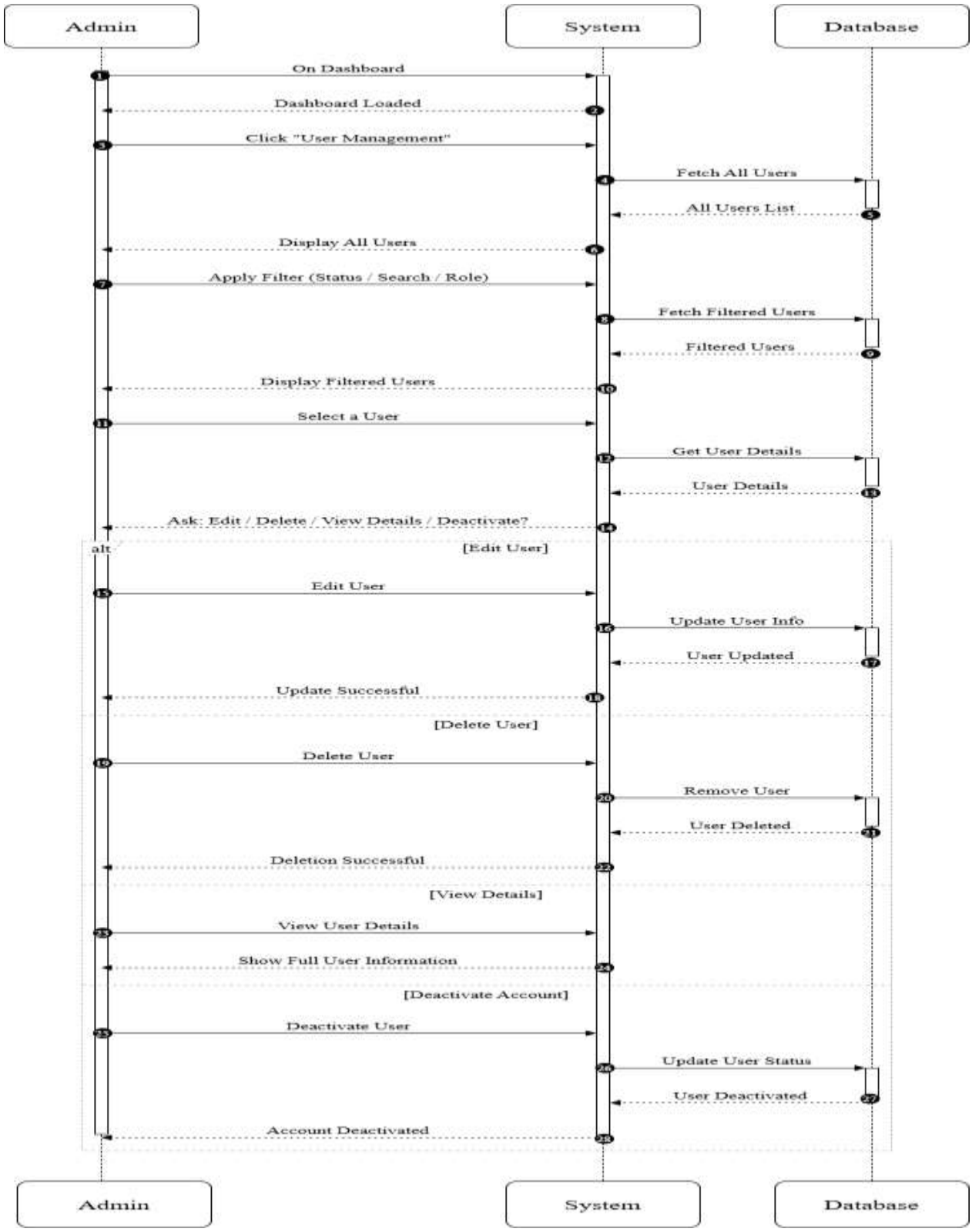


Figure 74: User Management - Sequence Diagram

8.3.18 Analytics Management

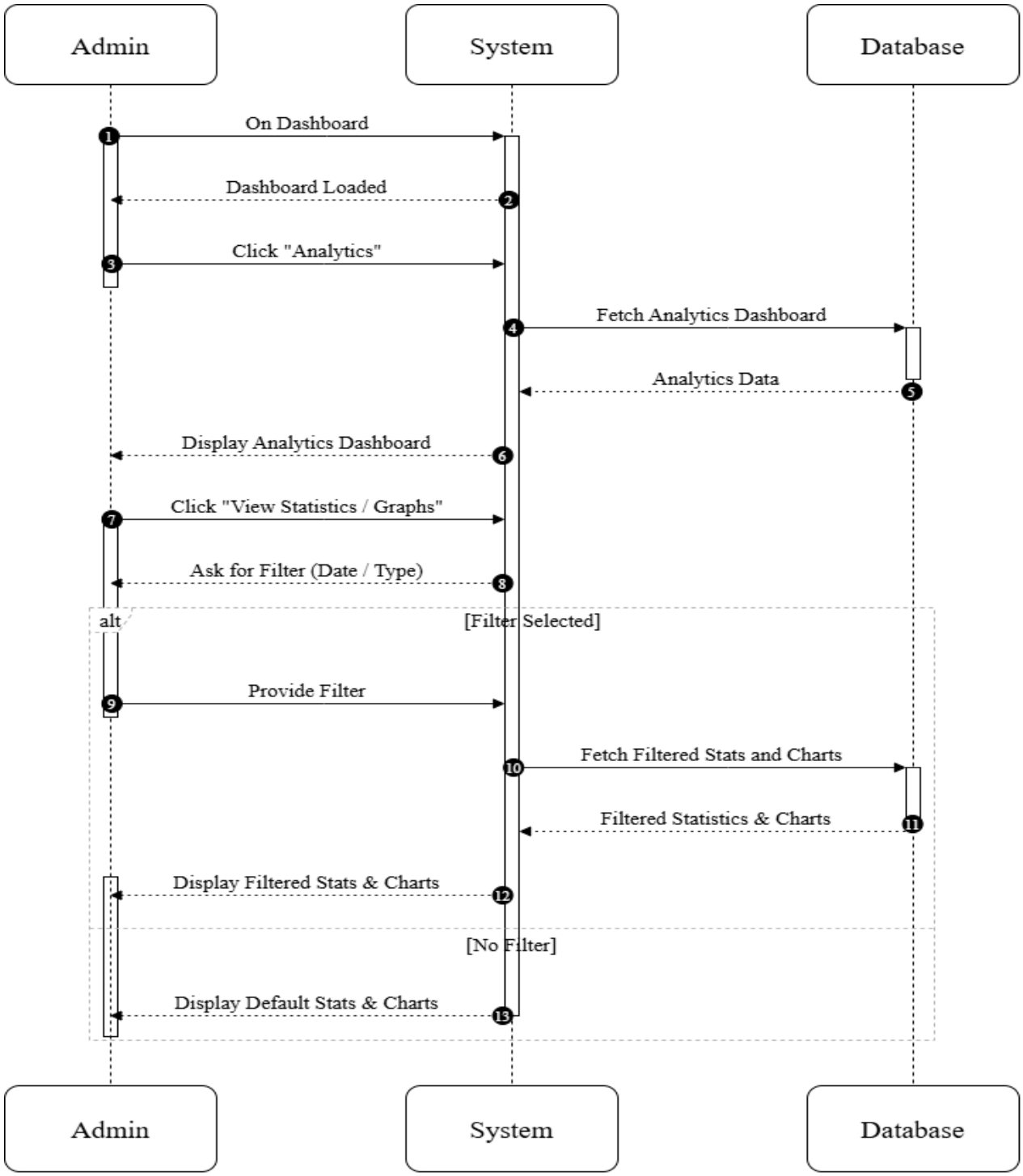


Figure 75: Analytics Management - Sequence Diagram

8.3.19 Approve/Reject Mechanics

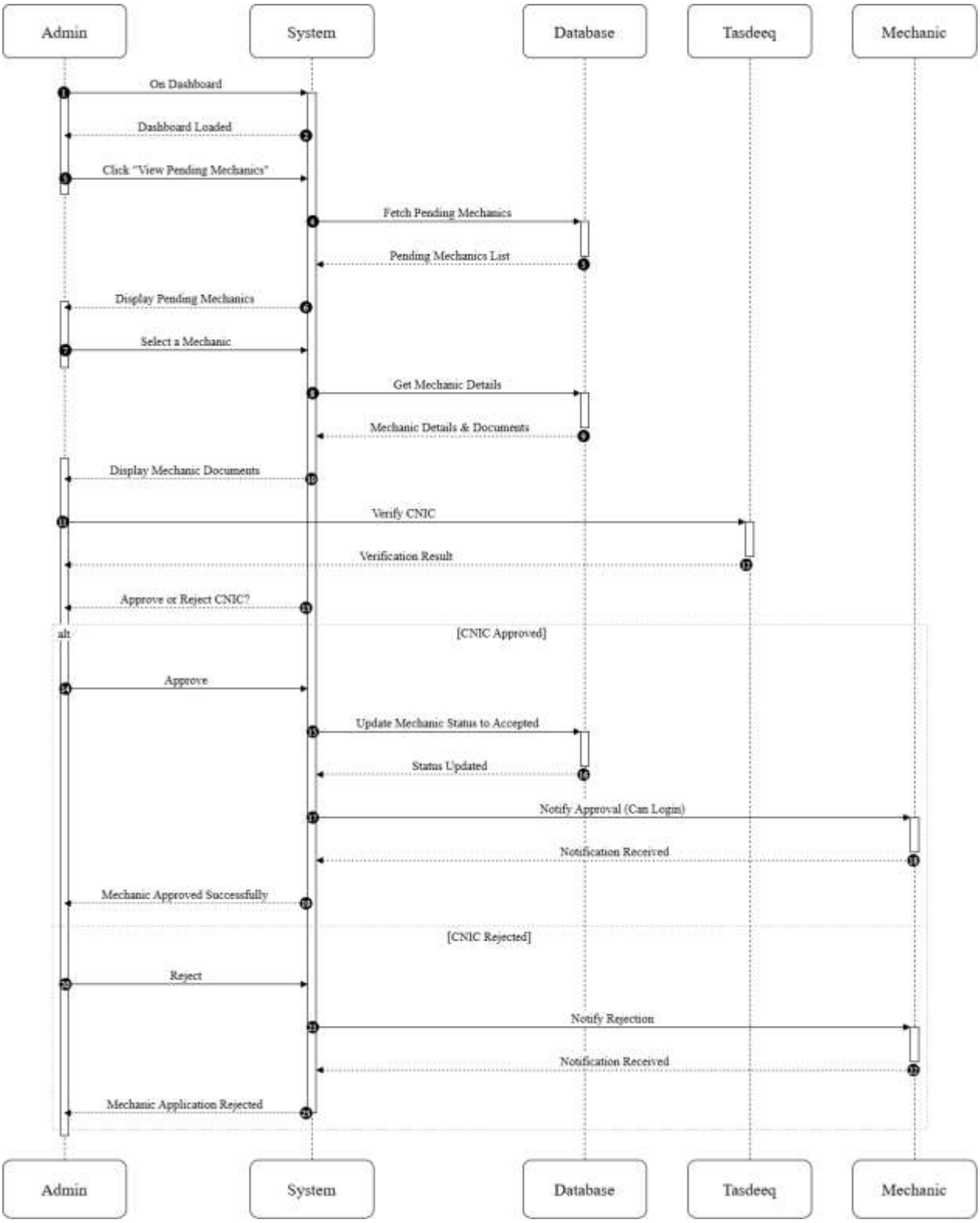


Figure 76: Approve/Reject Mechanics - Sequence Diagram

8.3.20 Booking Management

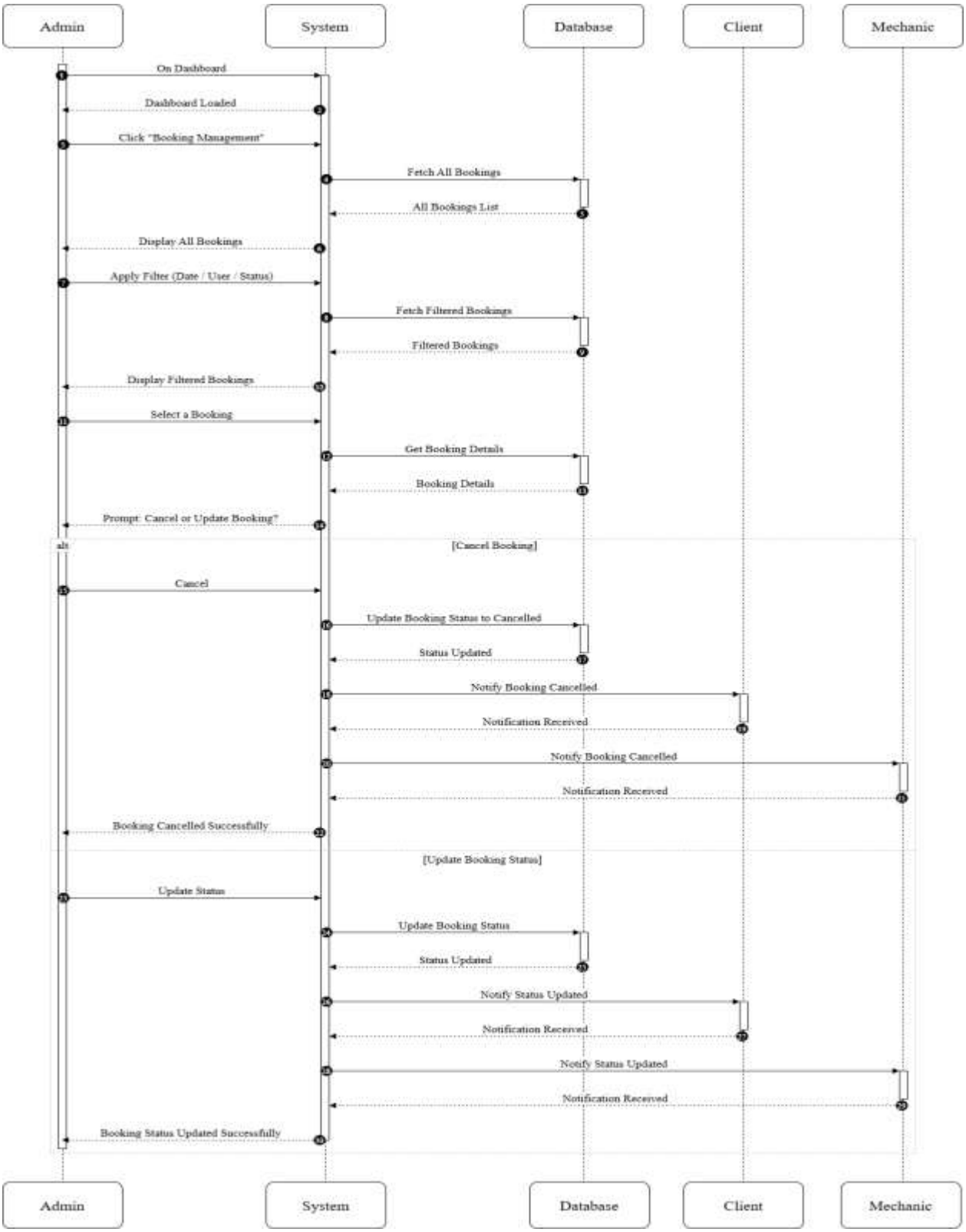


Figure 77: Booking Management - Sequence Diagram

8.4 Activity Diagrams

8.4.1 Client SignUp and Login

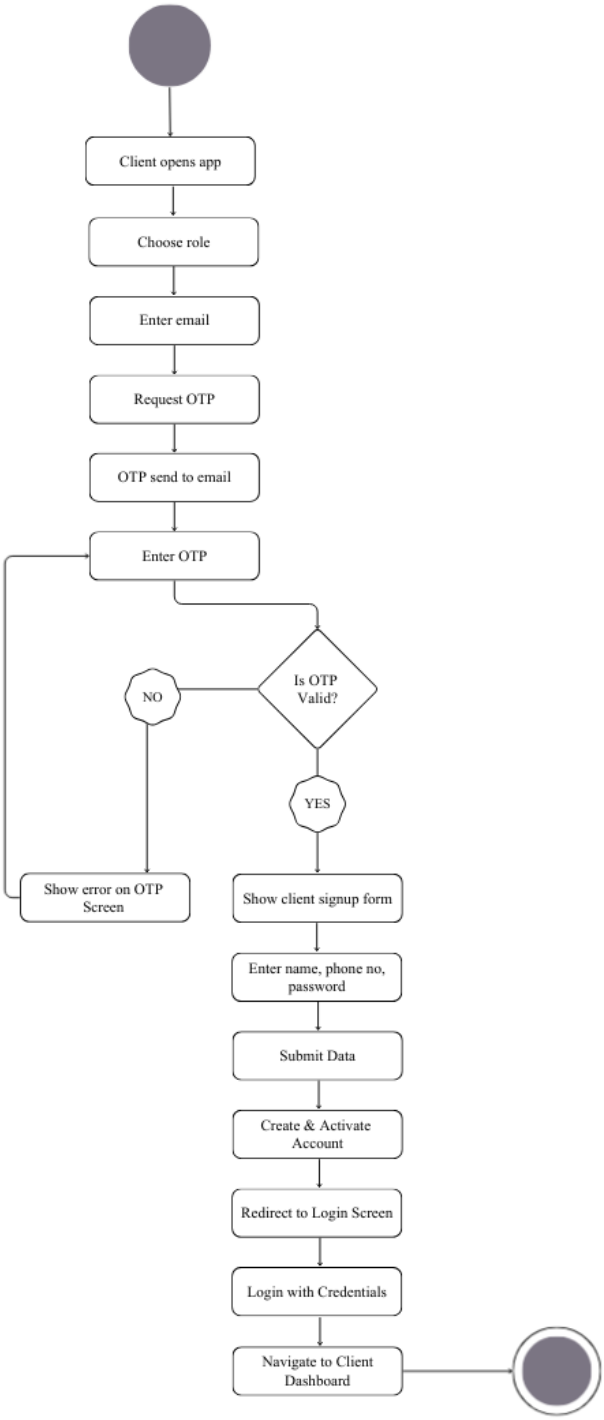


Figure 78: Client SignUp and Login - Activity Diagram

8.4.2 Mechanic SignUp and Login

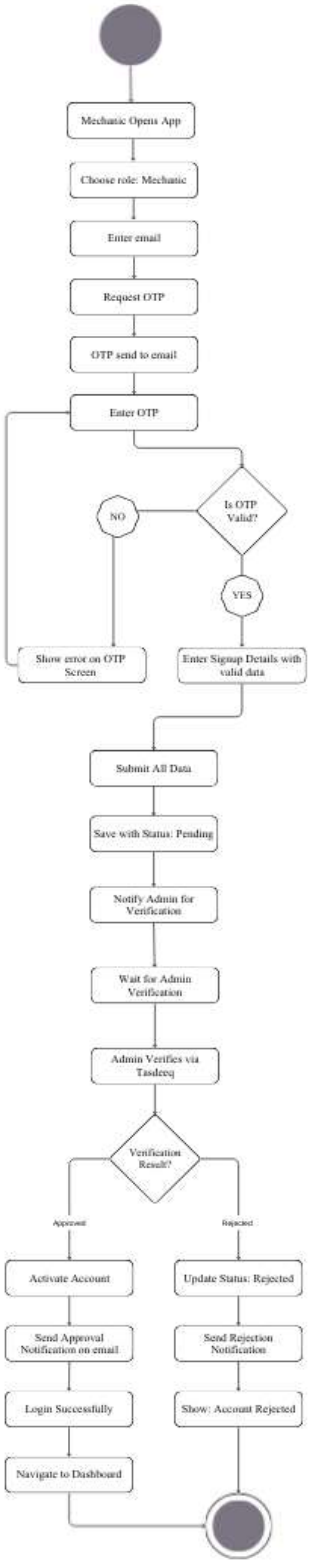


Figure 79: Mechanic SignUp and Login - Activity Diagram

8.4.3 Forgot Password

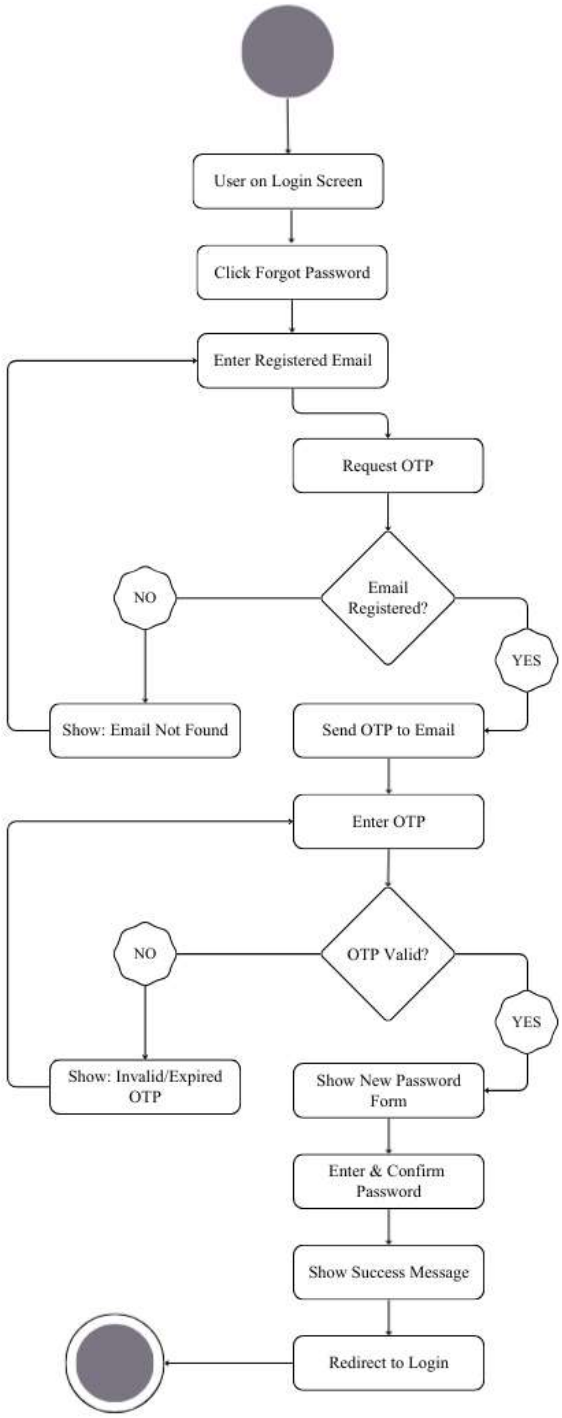


Figure 80: Forgot Password - Activity Diagram

8.4.4 Add a Car

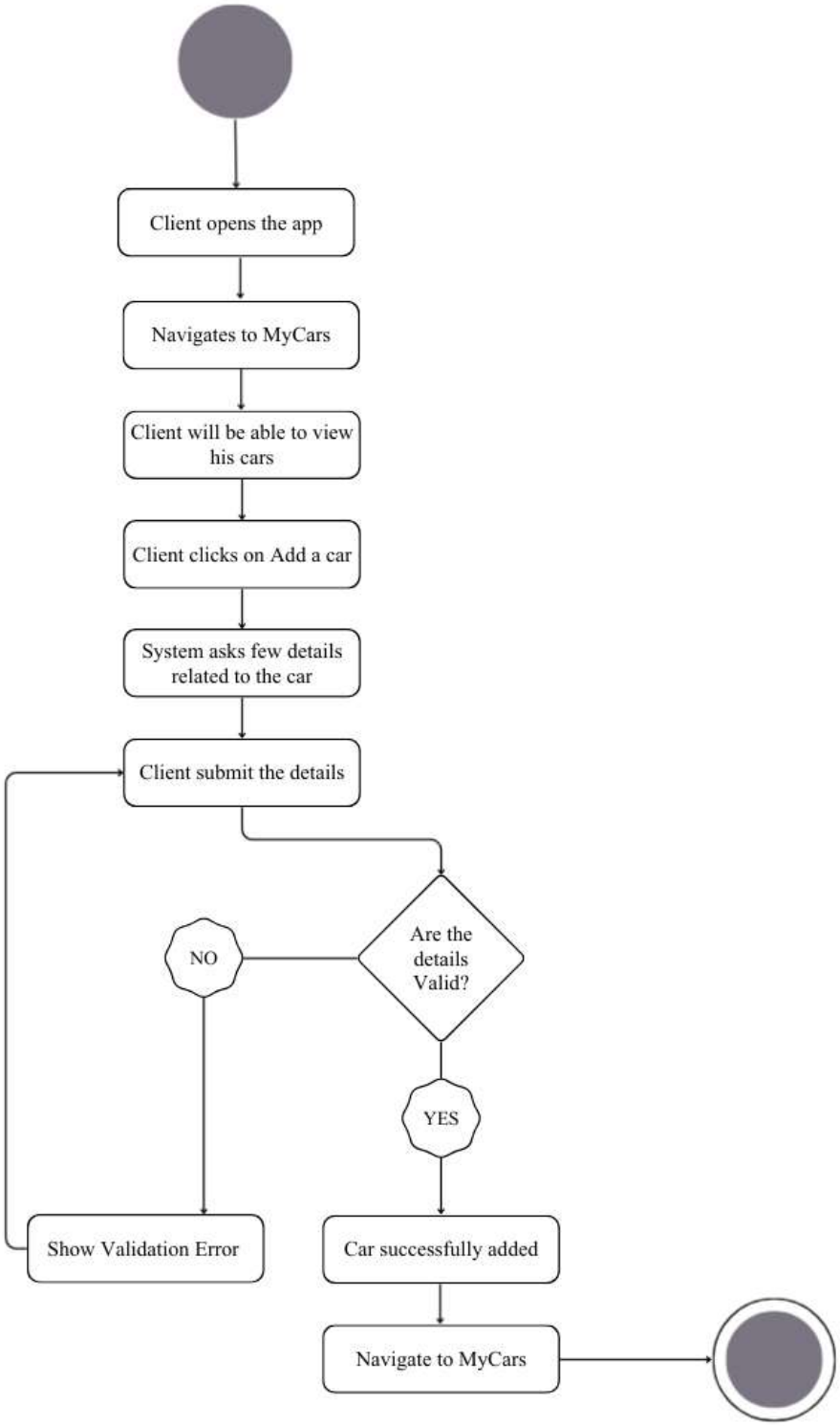


Figure 81: Add a Car - Activity Diagram

8.4.5 View All Cars

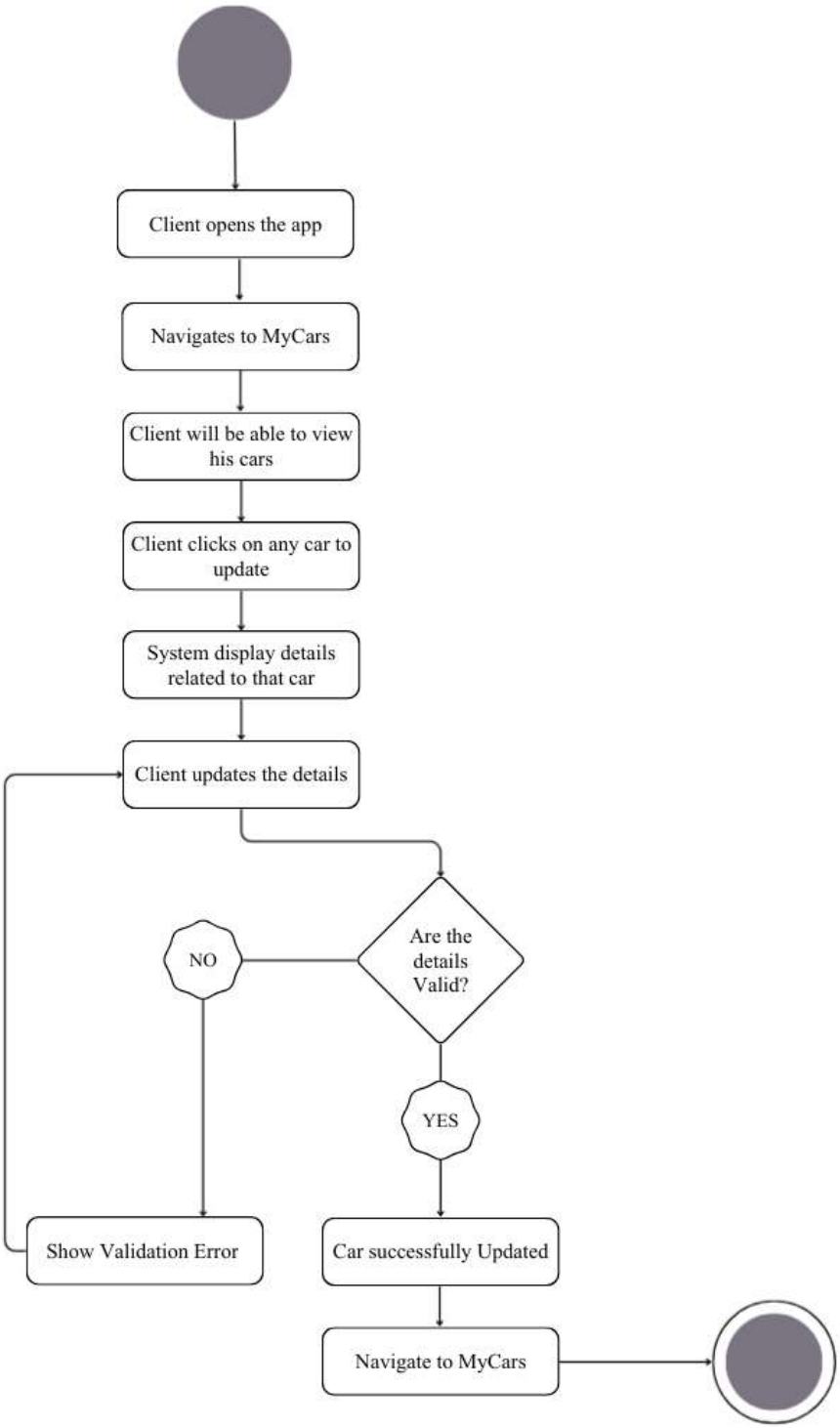


Figure 82: View All Cars - Activity Diagram

8.4.6 Delete a Car

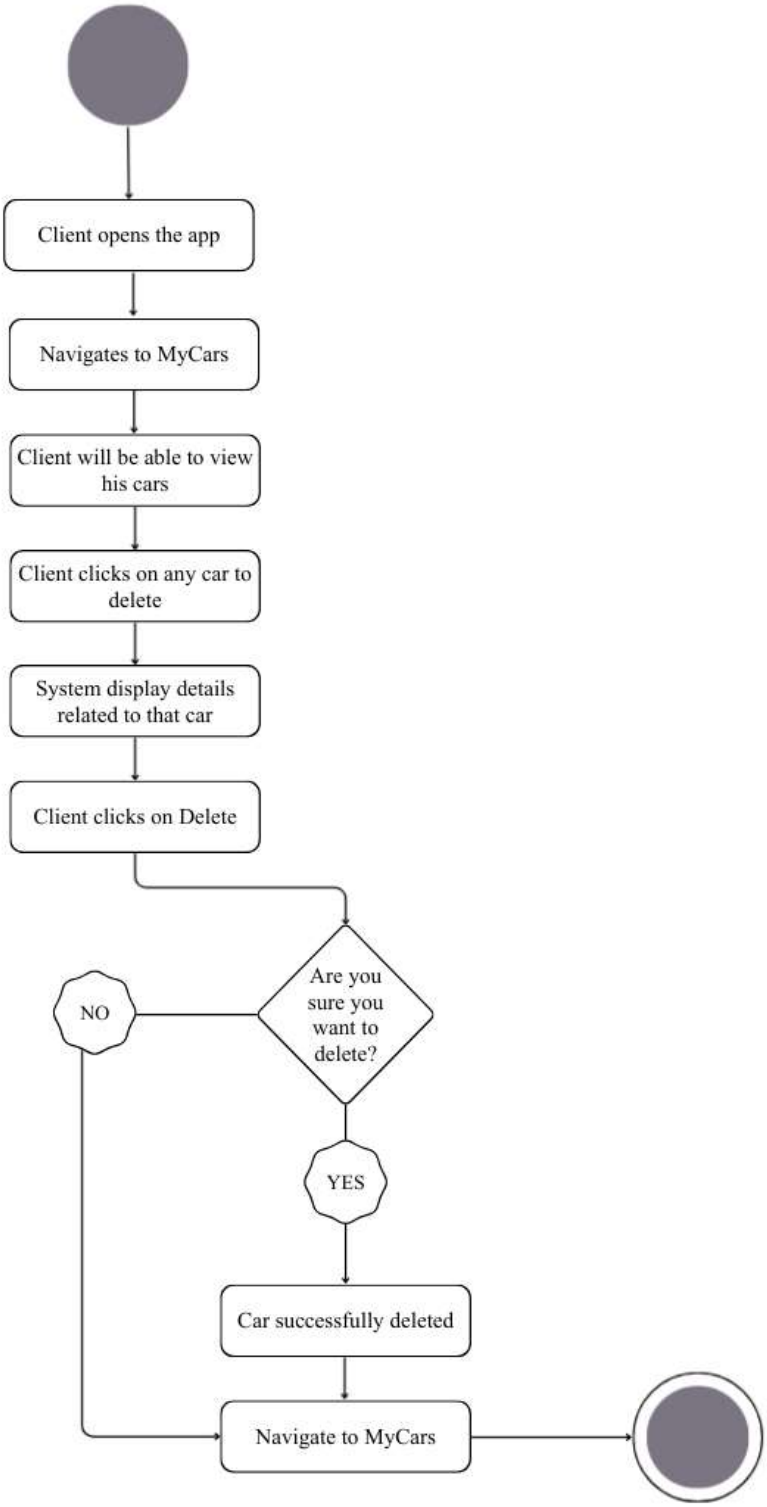


Figure 83: Delete a Car - Activity Diagram

8.4.7 AI Chat

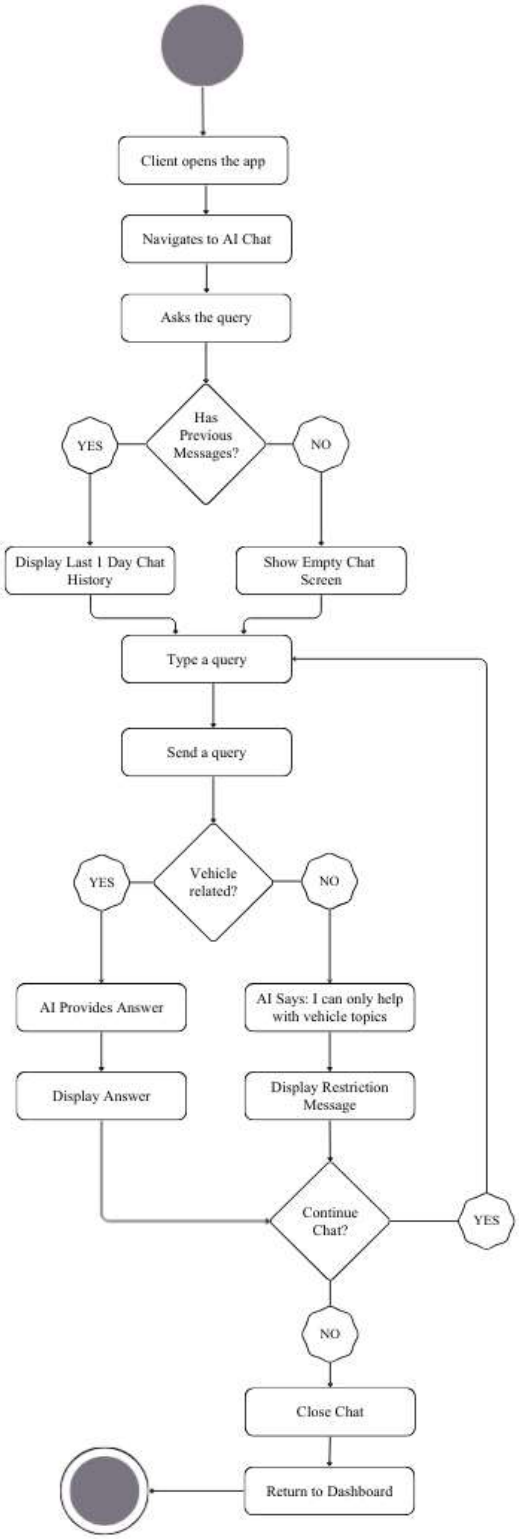


Figure 84: AI Chat - Activity Diagram

8.4.8 Service Booking Flow

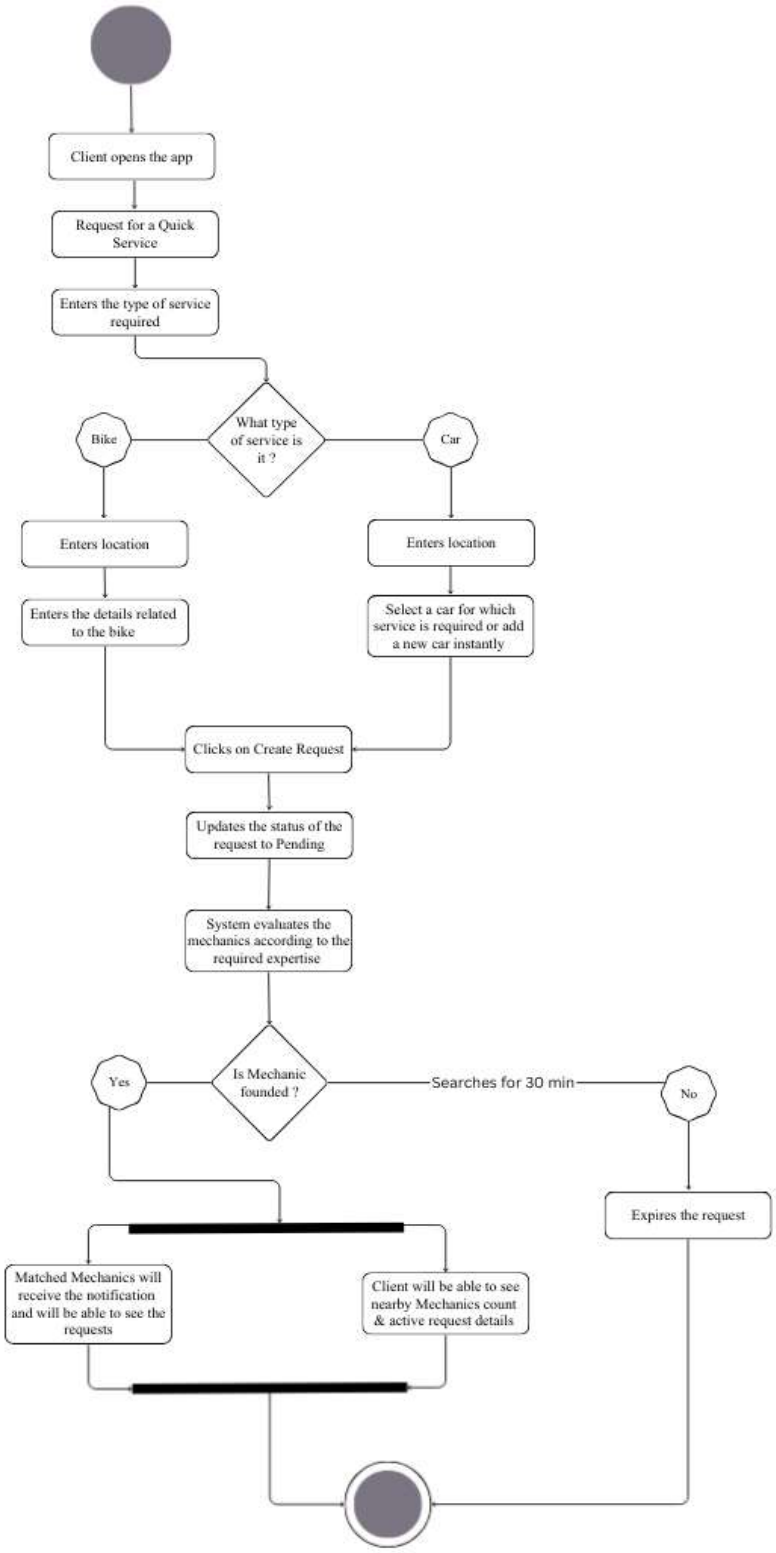


Figure 85: Service Booking Flow - Activity Diagram

8.4.9 Chat Module

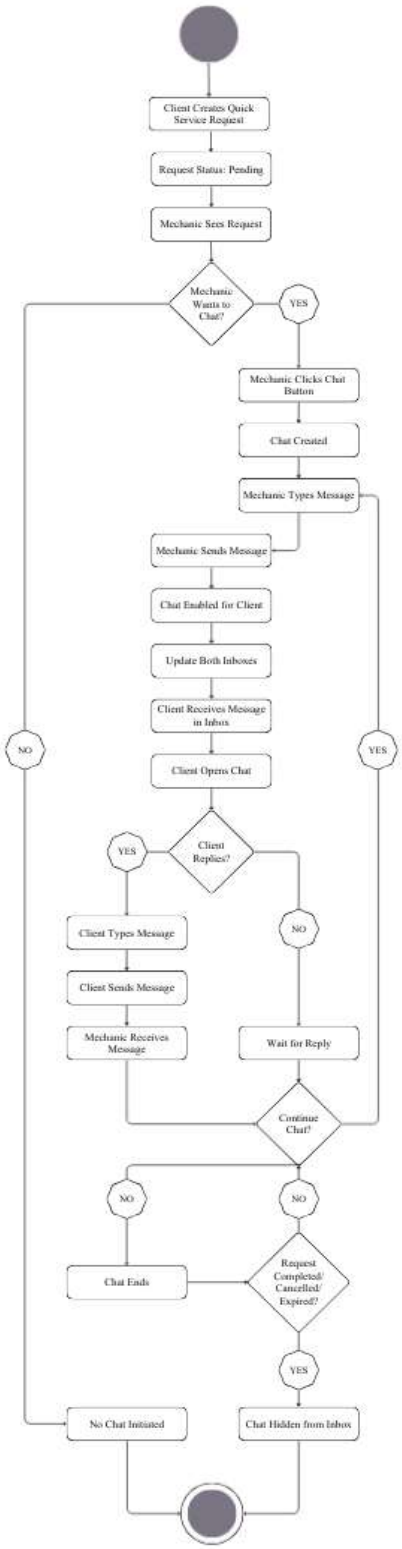


Figure 86: Chat Module - Activity Diagram

8.4.10 Payment Module

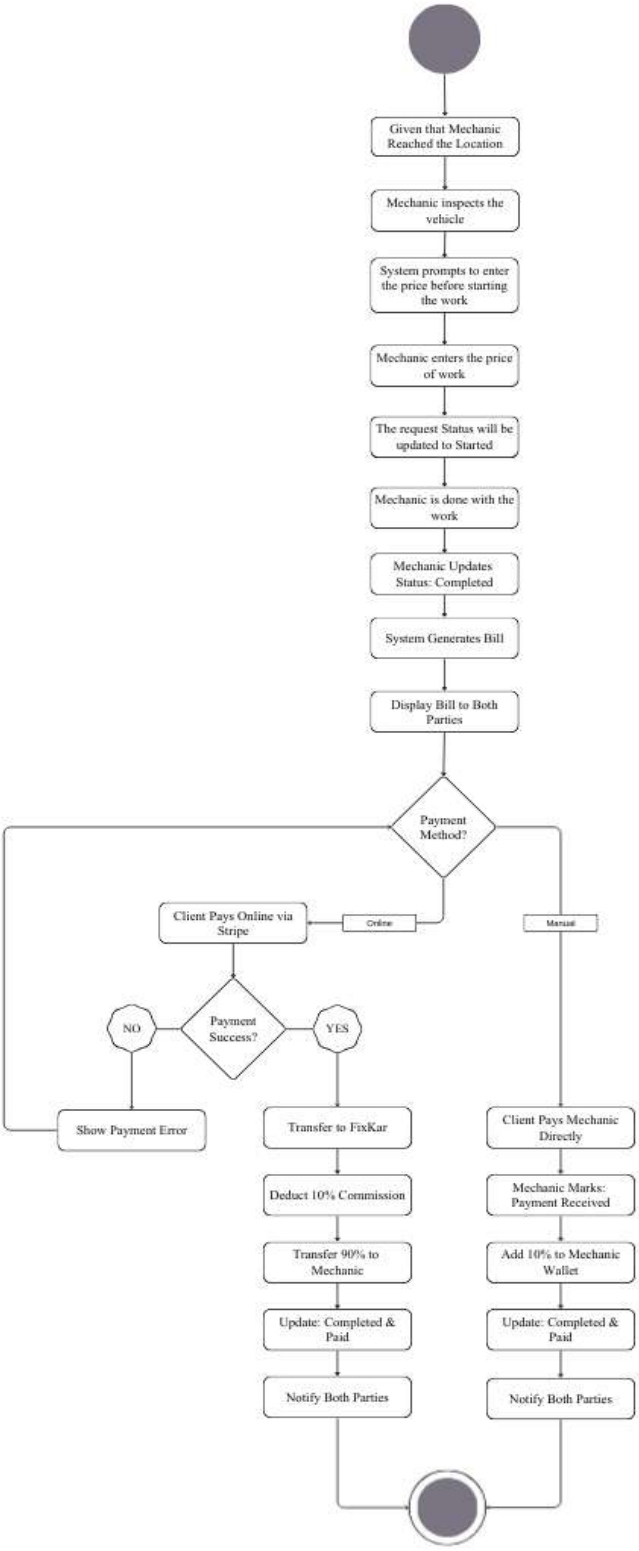


Figure 87: Payment Module - Activity Diagram

8.4.11 Live Tracking Module

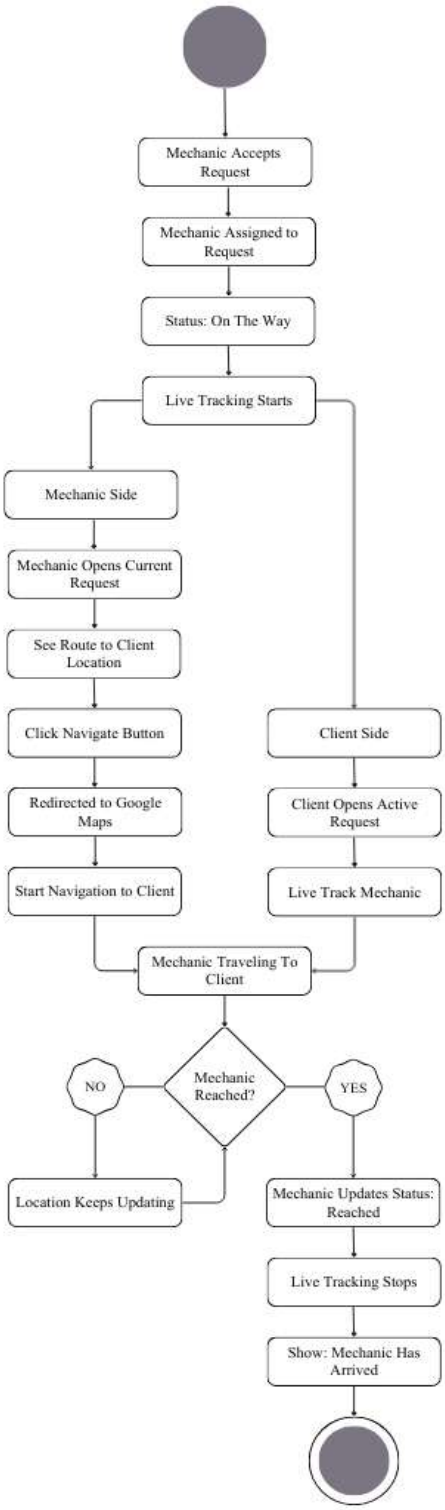


Figure 88: Live Tracking Module – Activity Diagram

8.4.12 Review and Rating Module

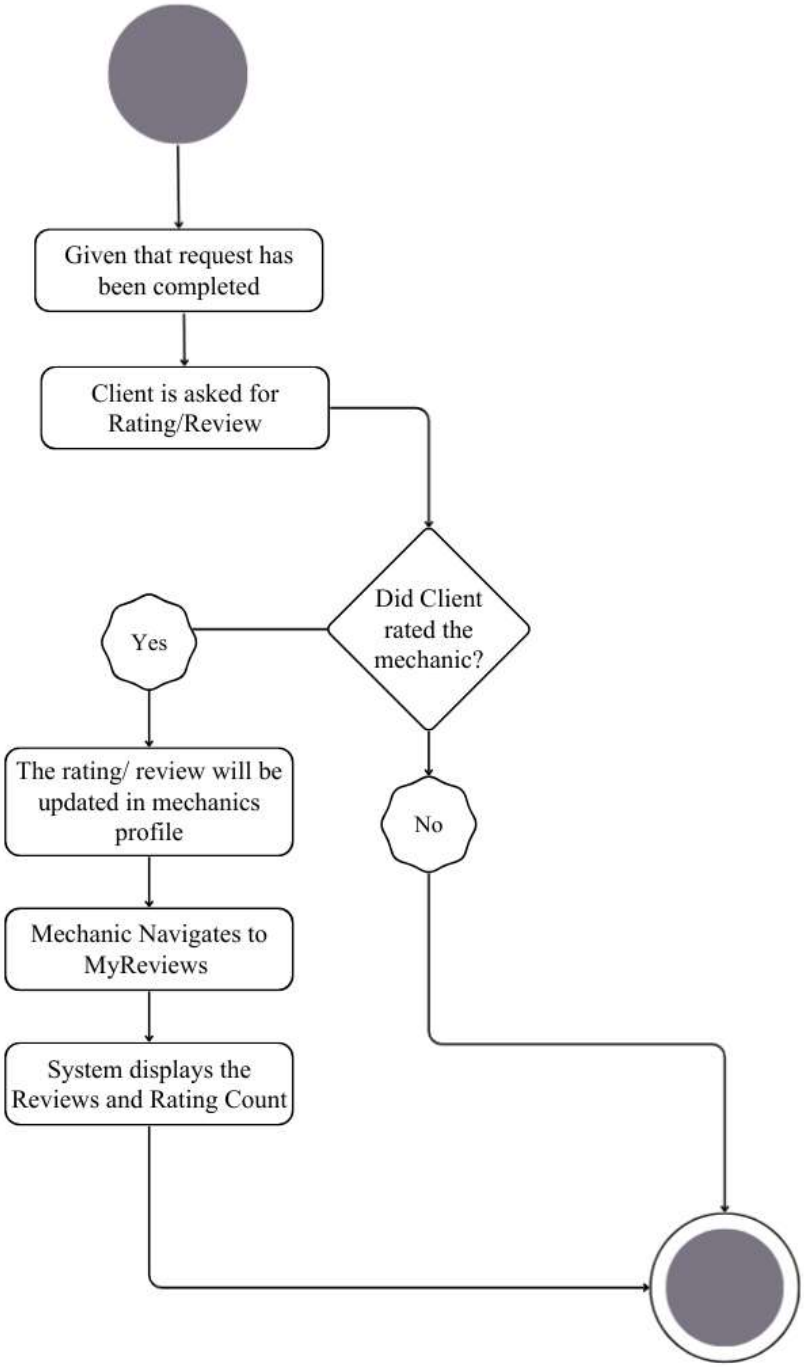


Figure 89: Review and Rating Module – Activity Diagram

8.4.13 Multilingual Support Module

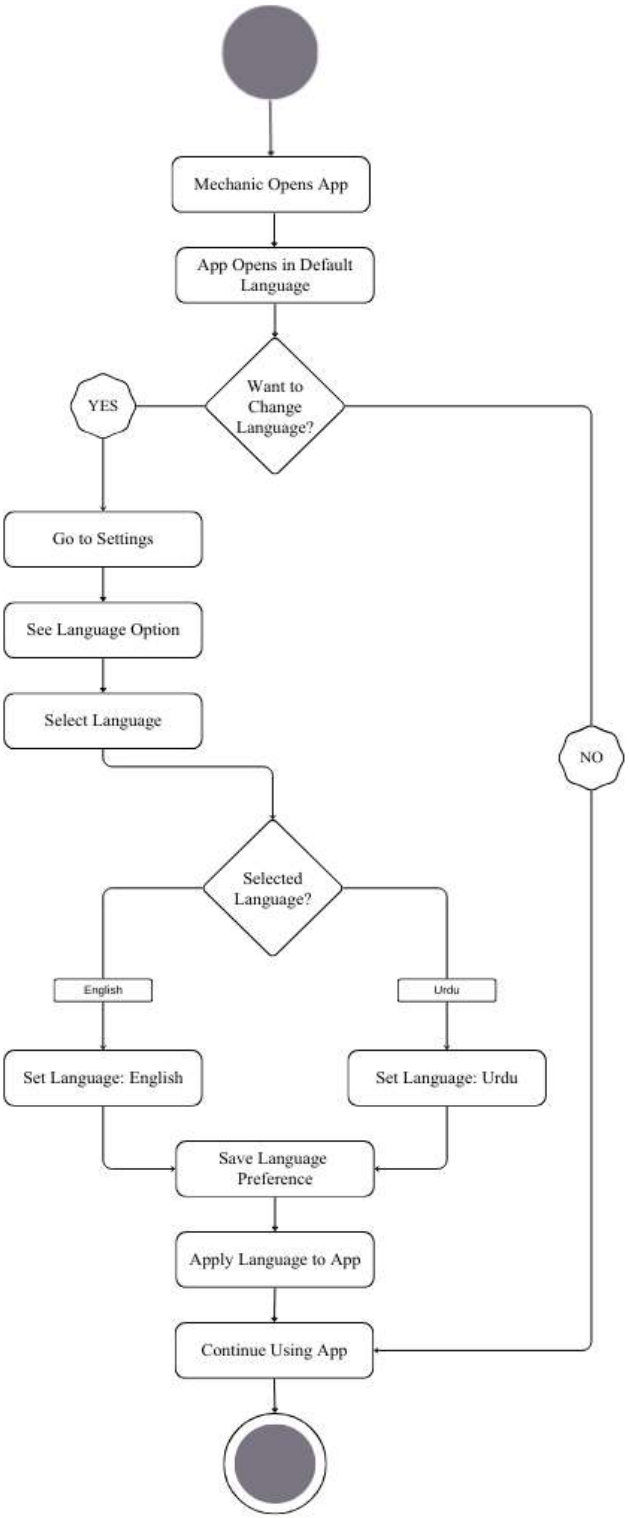


Figure 90: Multilingual Support Module - Activity Diagram

8.4.14 Notification Management

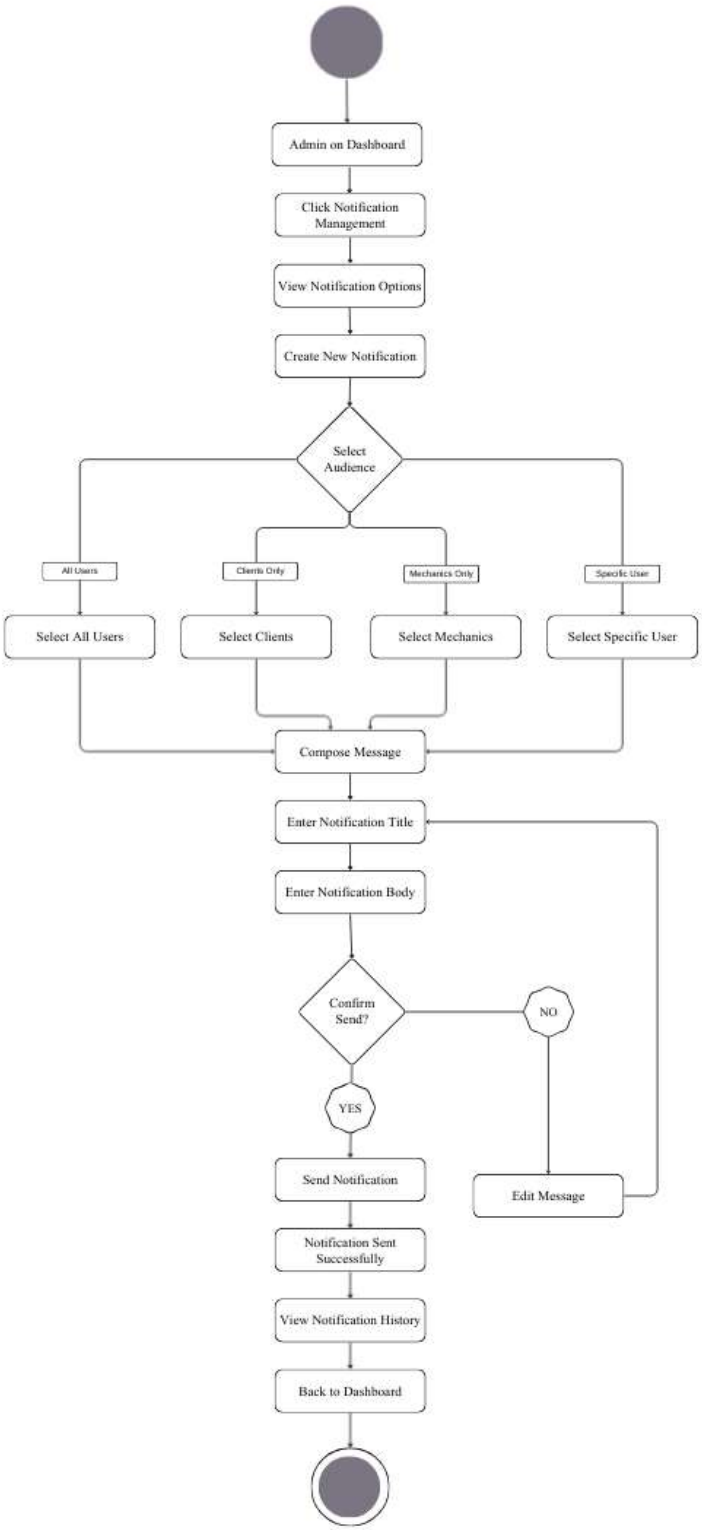


Figure 91: Notification Management - Activity Diagram

8.4.15 Create New Admin

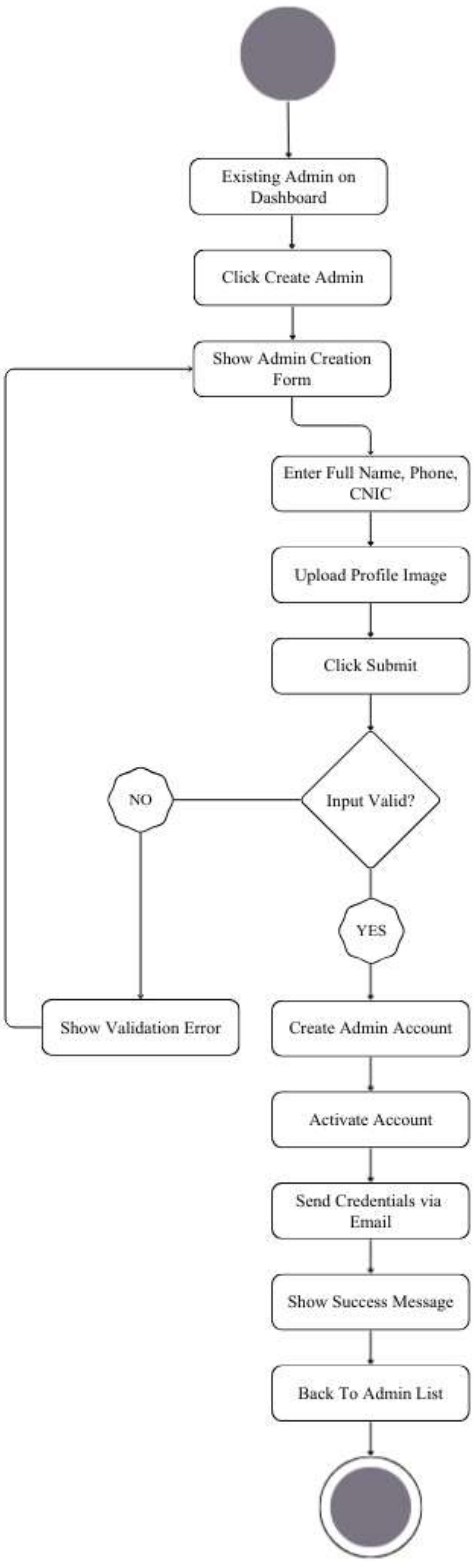


Figure 92: Create New Admin - Activity Diagram

8.4.16 Payment Management

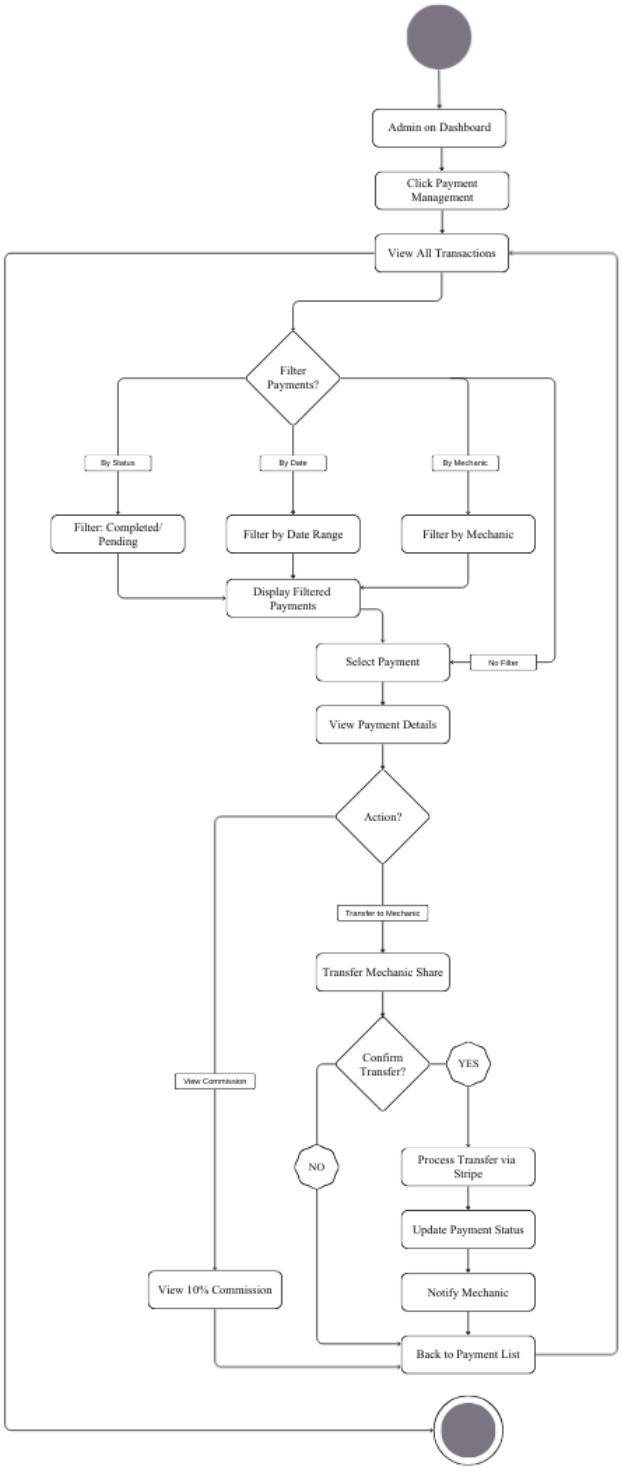


Figure 93: Payment Management - Activity Diagram

8.4.17 User Management

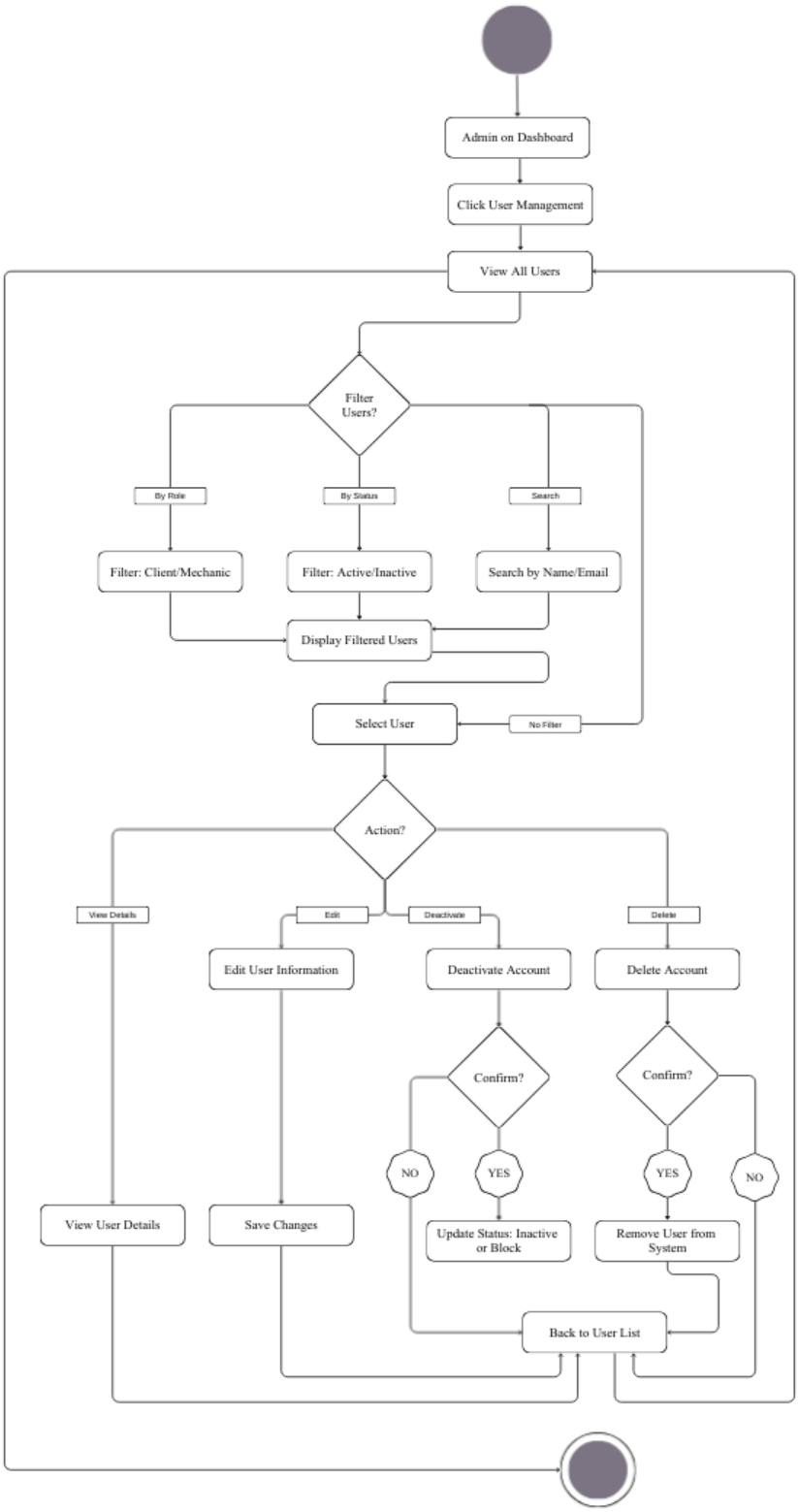


Figure 94: User Management - Activity Diagram

8.4.18 Analytics Management

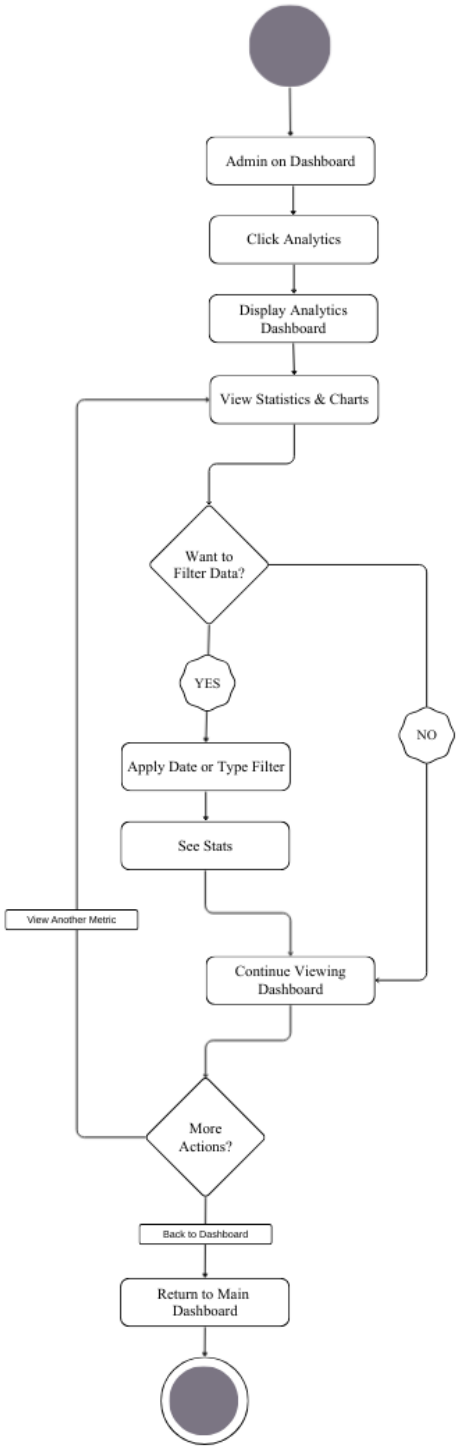


Figure 95: Analytics Management - Activity Diagram

8.4.19 Approve/ Reject Mechanics

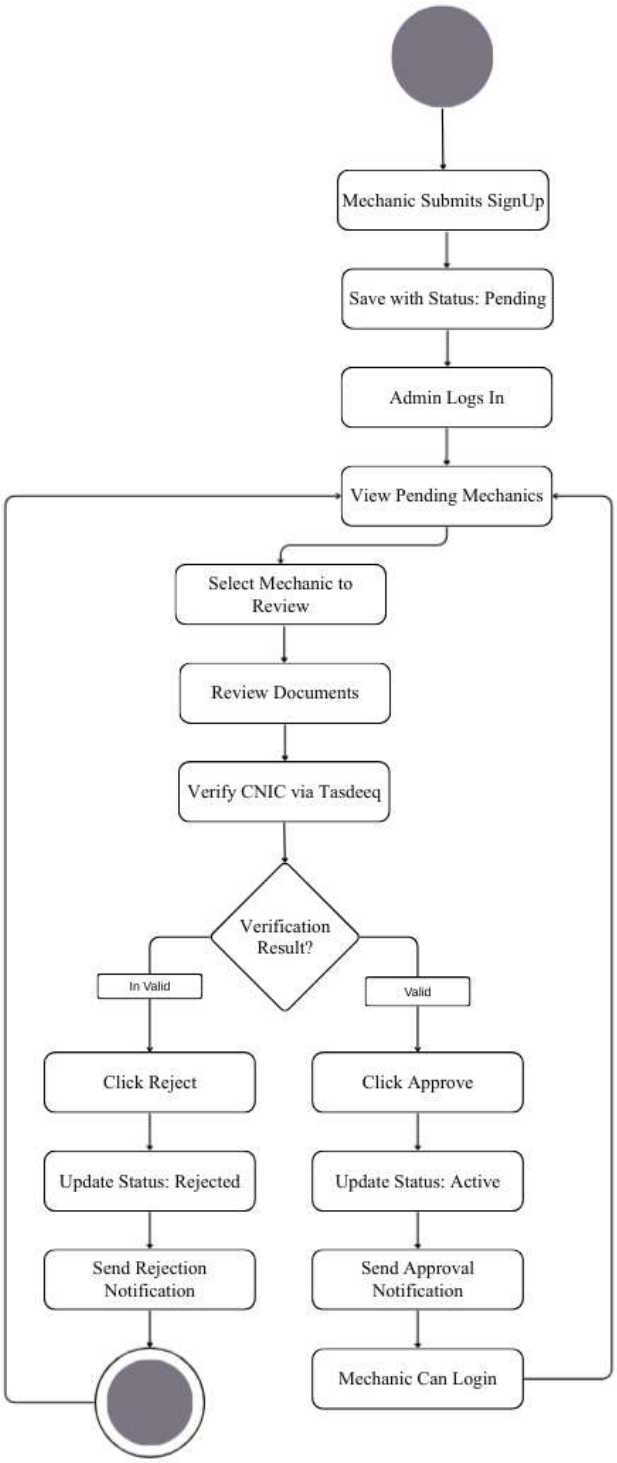


Figure 96: Approve/Reject Mechanics - Activity Diagram

8.4.20 Booking Management

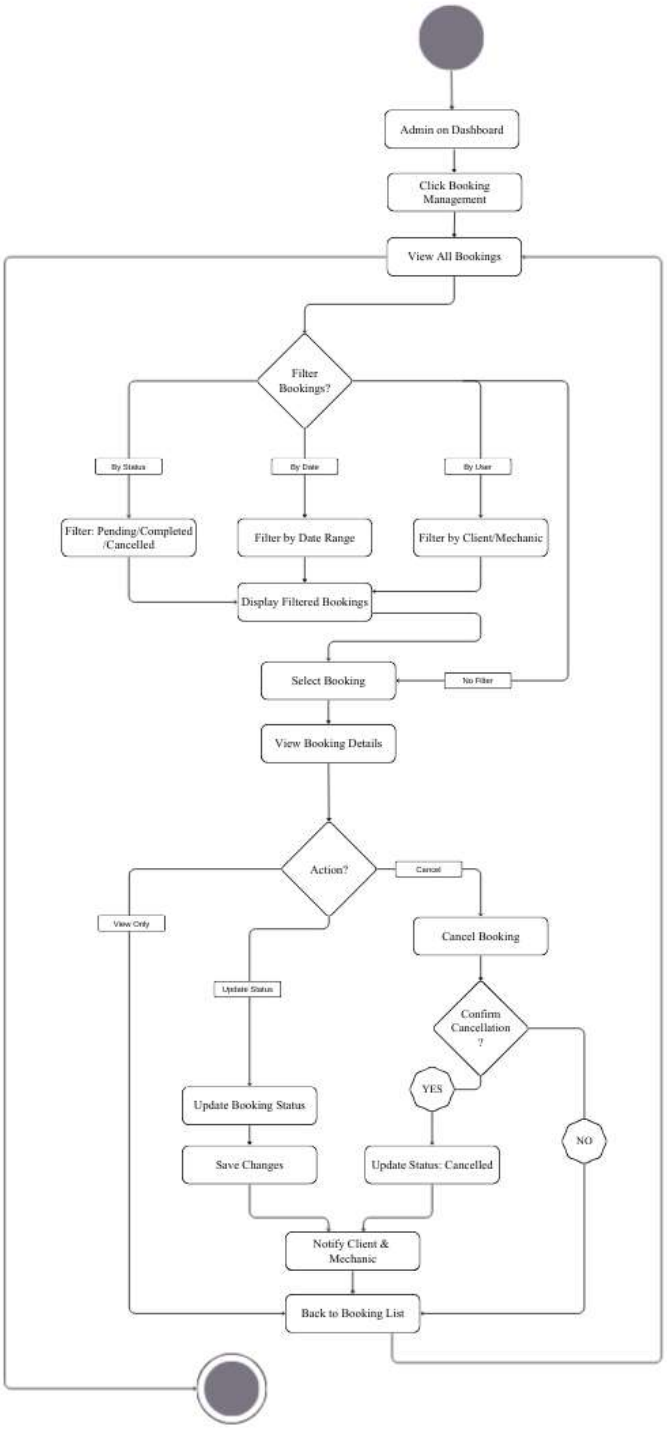


Figure 97: Booking Management - Activity Diagram

8.5 Use Case Diagrams

8.5.1 Admin Use Case Diagram

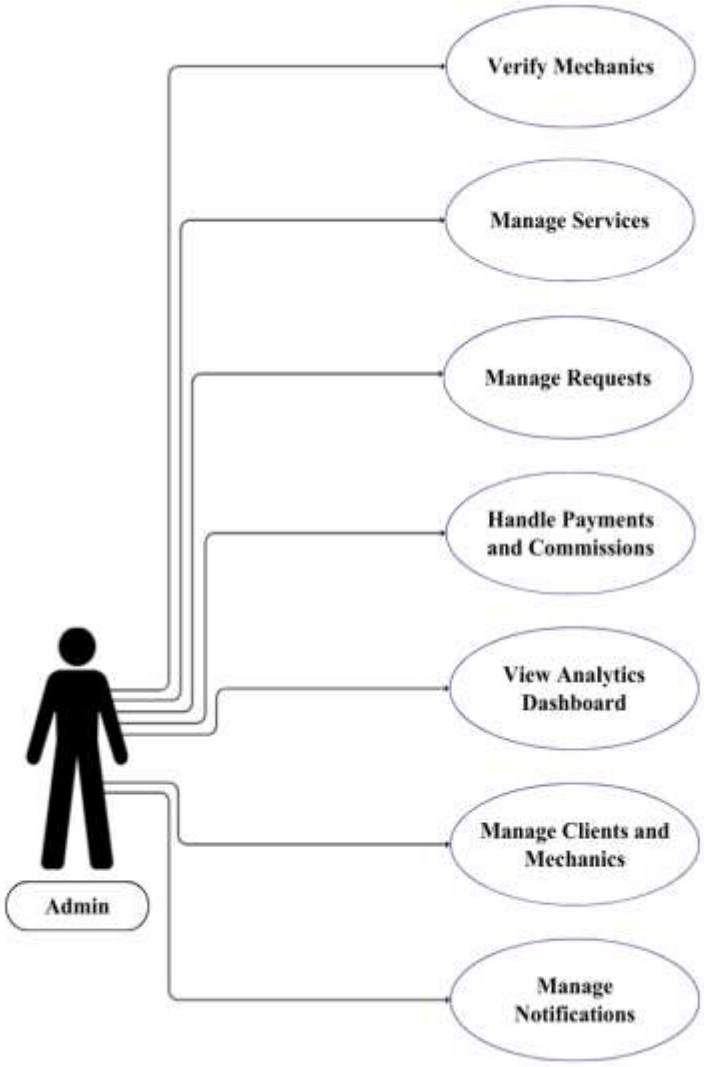


Figure 98: Admin – Use Case Diagram

8.5.2 Client Use Case Diagram

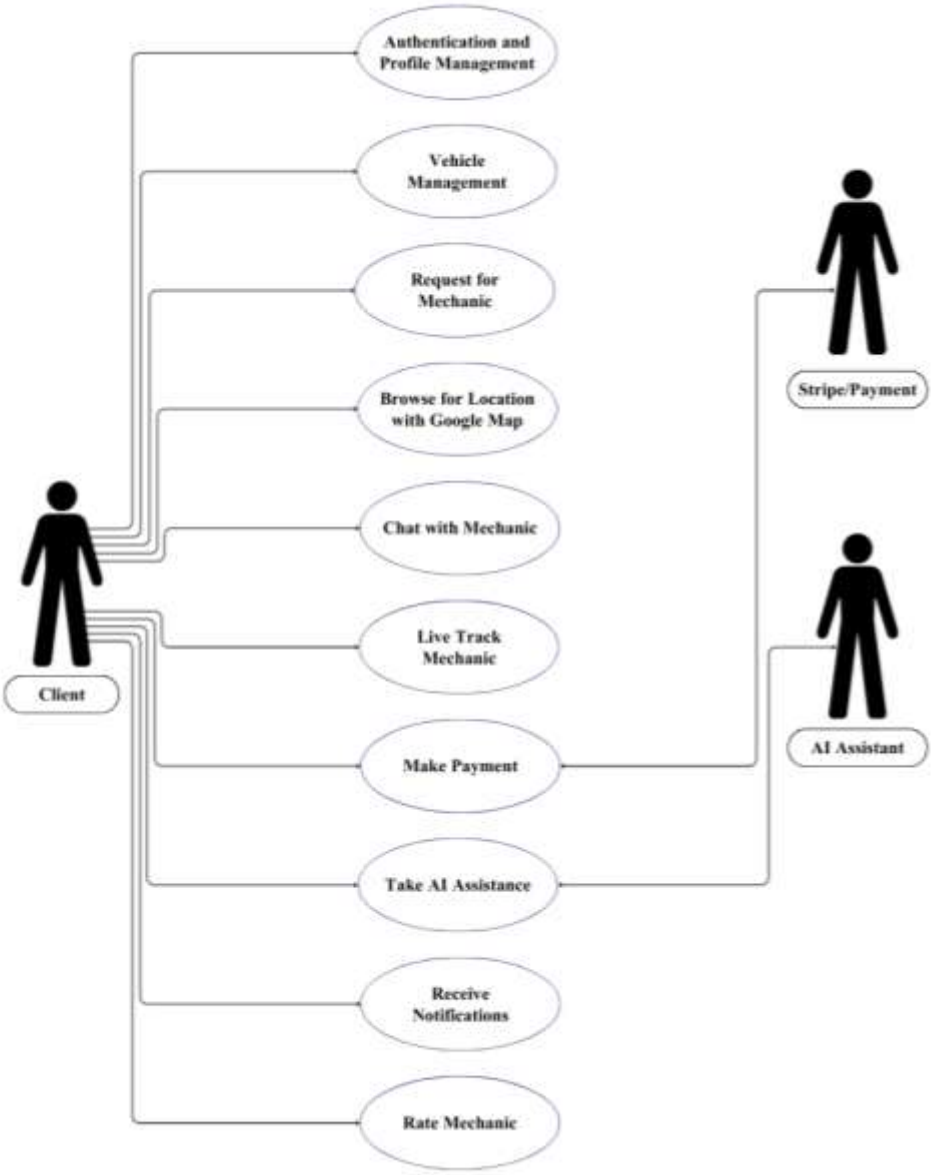


Figure 99: Client - Use Case Diagram

8.5.3 Mechanic Use Case Diagram

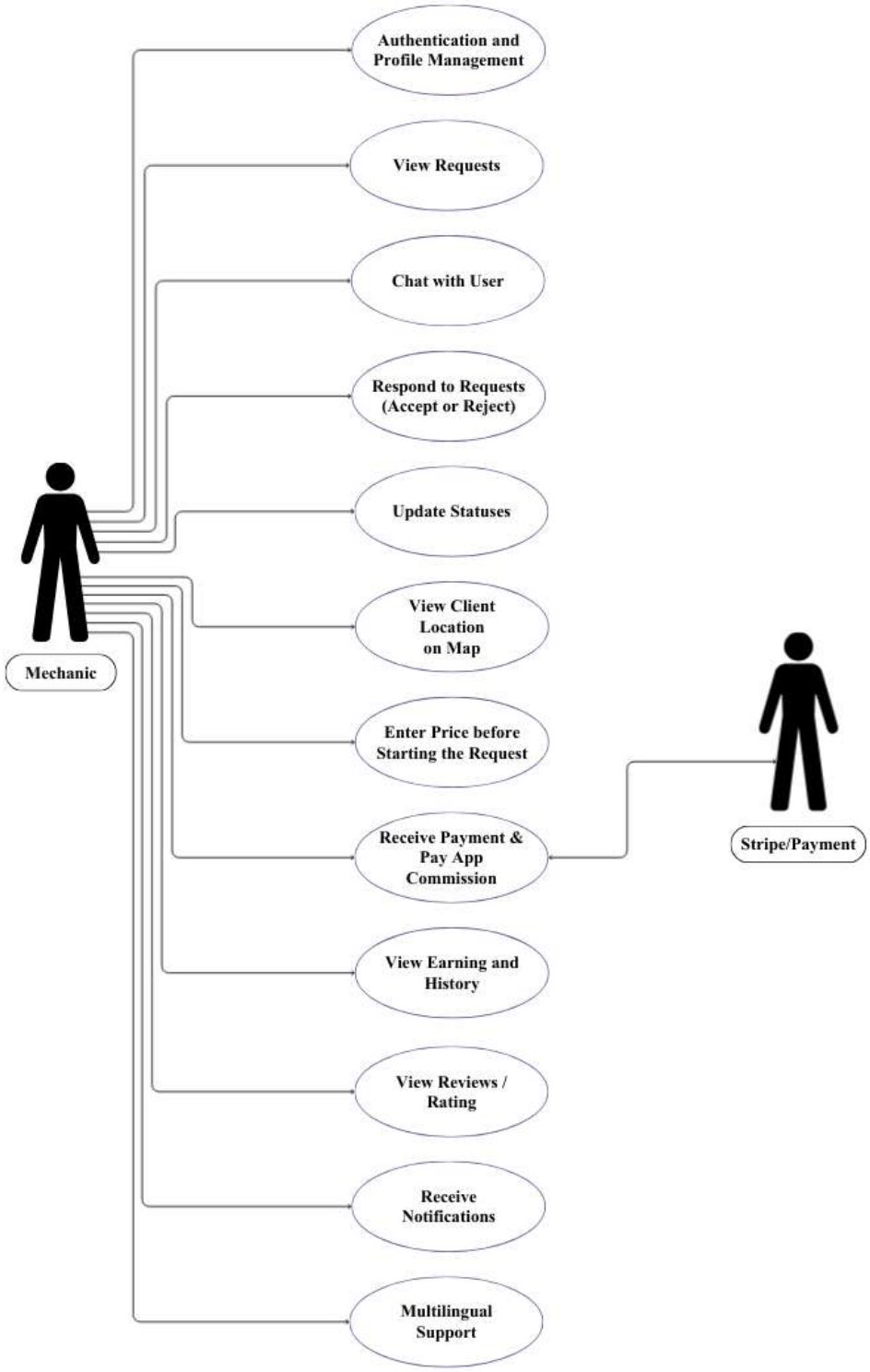


Figure 100: Mechanic - Use Case Diagram

8.6 Component Diagram

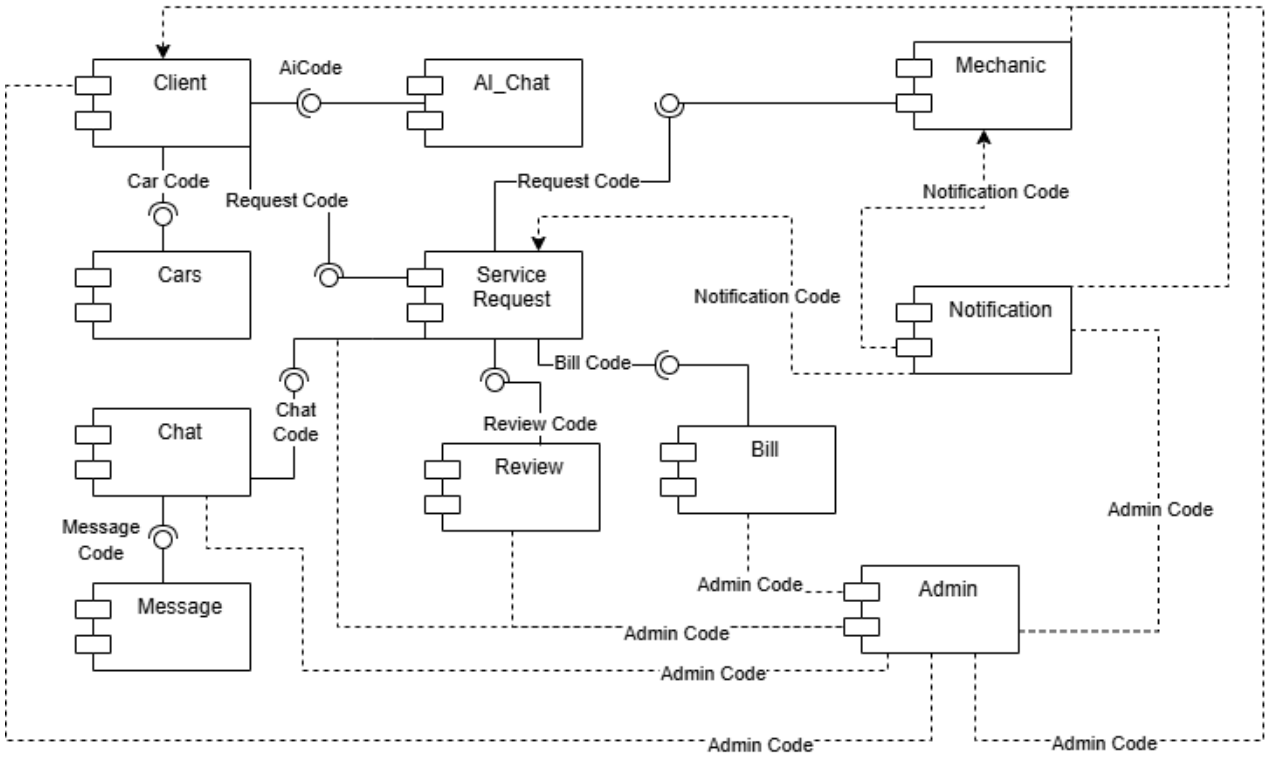


Figure 101: Component Diagram

Test Cases

1. User Account Management

1.1 SignUp

Test Case ID	P1-01				
Test Case Name	Sign Up	**Test Case Description**	Verifies the client signup functionality with OTP verification, covering steps from email validation, OTP generation, OTP verification, client information submission, and successful account creation.		
Created By	Muhammad Osama	**Version**	1.0	**Date**	17 Dec 2025

S. No	**Prerequisites:**
1	App must be functional and accessible
2	Client must have a valid email address or Email not previously registered.
3	Internet connection should be active

Test Scenario	Client account is successfully created after providing valid information and correct OTP.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Open client app	App opens successfully and shows Choose Role screen	Choose Role screen displayed	Pass
2	Select role "I am a Customer"	Customer role is selected and client flow starts	Customer role selected	Pass
3	Navigate to Signup tab	Signup form is displayed	Signup form displayed	Pass
4	Enter invalid email format	Validation error message is shown	Error message displayed	Fail
5	Enter already registered email	System displays the error message: "This email is already registered"	Error message displayed	Pass
6	Enter valid email address	Email is accepted	Email accepted	Pass

7	Click “Sign up” button	OTP is sent to entered email	OTP not send, email service wasn’t functional	Fail
7.1	Re-entered email and click “Sign up” button	OTP is sent to entered email	OTP sent	Pass
8	Modal appears with message “OTP sent to your email”	Modal should be displayed	Modal displayed	Pass
9	Navigate to Enter OTP screen	User is redirected to OTP screen	Navigated successfully	Pass
10	Enter invalid OTP	Error message “Invalid OTP” is displayed	Error message displayed	Pass
11	Enter expired OTP	Error message “OTP expired” is displayed	Error message displayed	Pass
12	OTP timer reaches 2 minutes	OTP expires and Resend button should display	“Resend OTP” button was displayed	Pass
13	Click “Resend OTP” button	New OTP is sent and timer restarts (2 minutes)	OTP resent	Pass
14	Enter valid OTP	OTP is accepted	OTP accepted	Pass
15	Click “Verify OTP” button	Show success toast OTP is verified successfully	Verified	Pass
16	Navigate to Client Info form	Client info form is displayed	Form displayed	Pass
17	Submit empty fields	Submit button should be disabled	Errors shown	Pass
18	Enter invalid phone number OR password less than 6 characters OR mismatched confirm password	Appropriate validation error message is shown for respective field	Errors shown	Pass
19	Enter valid name, phone, password & confirm password	All fields accepted	Data accepted	Pass

20	Click “Submit” button	Client account is created successfully	Account created	Pass
21	Success modal appears with message “Welcome aboard! Your account has been created successfully.”	Success modal should be displayed	Modal displayed	Pass
22	Navigate to Login screen	User is redirected to Login	Navigation stack issue	Fail
22.1	Navigate to Login screen	User is redirected to Login	Navigated to Login	Pass

Table 55: SignUp Test Case

1.2 Forgot Password

Test Case ID	P1-02				
Test Case Name	Forgot Password	**Test Case Description**	Verifies the complete forgot password functionality including sending OTP to email, verifying OTP, and changing password with proper validations.		
Created By	Muhammad Osama	**Version**	1.0	**Date**	20 Dec 2025

S. No	**Prerequisites:**
1	User must have a registered account
2	App must be functional and accessible
3	Valid email address required
4	Internet connection should be active

Test Scenario	User successfully resets password by receiving OTP, verifying it, and setting new password.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Navigate to Login screen	Login screen is displayed	Login screen displayed	Pass
2	Click “Forgot Password?” link	Send OTP screen opens	Send OTP screen opened	Pass
3	Verify back button is visible	Back button displayed in top-left	Back button visible	Pass

4	Verify “OTP Verification” heading and image	Heading and email icon displayed	Heading displayed	Pass
5	Verify subtitle text displayed	“A verification code will be sent...” message shown	Subtitle shown	Pass
6	Click "Get OTP" without email	Validation error “Email is Required” shown	Email Send	Fail
6.1	Click "Get OTP" without email	Validation error “Email is Required” shown	Error shown	Pass
7	Enter invalid email format (e.g., “test@”)	Validation error “Please enter a valid email address” shown	Error shown	Pass
8	Enter email that is not registered	Backend error “User not found” shown	Error shown	Pass
9	Enter valid registered email	Email is accepted	Email accepted	Pass
10	Click “Get OTP” button	Button shows “Sending...”, loader appears	Loading shown	Pass
11	Wait for OTP to be sent	Success toast “OTP sent successfully” displayed	Toast displayed	Pass
12	Verify navigation to OTP Verification screen	OTP screen opens with 6 input boxes	OTP screen opened	Pass
13	Verify email displayed on OTP screen	Entered email address shown below subtitle	Email displayed	Pass
14	Verify timer shows “Expires in 120 Seconds”	Timer starts counting down from 120	Timer working	Pass
15	Enter invalid OTP (6 digits)	Backend error “Invalid OTP” shown	Error shown	Pass

16	Enter valid OTP before timer expires	OTP is accepted	OTP accepted	Pass
17	Click “Verify” button	Loader appears, verification starts	Loading shown	Pass
18	Wait for OTP verification	Success toast “OTP verified successfully” displayed	Toast displayed	Pass
19	Verify navigation to Change Password screen	Change Password screen opens with two password fields	Screen opened	Pass
20	Verify “Change Password” heading displayed	Heading with subtitle shown	Heading displayed	Pass
21	Click “Change Password” without filling fields	Validation errors shown for both fields	Errors shown	Pass
22	Enter new password less than 6 characters	Validation error “Password must be at least 6 characters” shown	Password accepted	Pass
23	Enter valid new password (at least 6 characters)	Password is accepted	Errors shown	Pass
24	Enter different password in confirm field	Validation error “Passwords must match” shown	Errors shown	Pass
25	Enter matching passwords in both fields	Both passwords accepted	Passwords accepted	Pass
26	Click “Change Password” button	Button shows “Updating Password...”, loader appears	Loading shown	Pass

27	Wait for password update	Success toast "Password changed successfully" displayed	Toast displayed	Pass
28	Verify navigation back to Login screen	User redirected to Login screen	Navigated to Login	Pass
29	Login with old password	Error "Invalid email or Password" shown	Error shown	Pass
30	Login with new password	Login successful, user navigated to home	Login successful	Pass

Table 56: Forgot Password Test Case

1.3 Change Password

Test Case ID	P1-03				
Test Case Name	Change Password	**Test Case Description**	Verifies the change password functionality for logged-in users including old password verification, new password validation, and successful password update.		
Created By	Muhammad Osama	**Version**	1.0	**Date**	20 Dec 2025

S. No	**Prerequisites:**
1	User must be logged in
2	App must be functional and accessible
3	User must know current password
4	Internet connection should be active

Test Scenario	Logged-in user successfully changes password by providing correct old password and valid new password with proper validation.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Navigate to Change Password screen from profile	Change Password screen is displayed with three password fields	Screen displayed	Pass
2	Verify "Reset your password"	Heading with underline shown	Heading displayed	Pass

3	Try to click “Update” without filling any field	Update button is disabled	Button disabled	Pass
4	Enter new password less than 6 characters	Validation error "Password must be at least 6 characters" shown	Error shown	Pass
5	Enter valid new password but different confirm password	Validation error “Passwords must match” shown	Error shown	Pass
6	Enter matching passwords in new and confirm fields	Passwords accepted, Update button enabled	Passwords accepted	Pass
7	Enter incorrect old password and click “Update”	Backend error “Old password” is incorrect" shown	Error shown	Pass
8	Enter correct old password but same as new password	Backend error "New password cannot be the same as old password" shown	Error shown	Pass
9	Enter correct old password and different new password	All fields accepted	Fields accepted	Pass
10	Click “Update” with all valid data	Button shows “Updating...” loader appears	Loading shown	Pass
11	Wait for password update response	Success toast "Password changed successfully" displayed	Toast displayed	Pass
12	Verify form is reset and navigation occurs	All fields cleared, user navigated back	Fields cleared, navigated	Pass
13	Logout and login with old password	Error “Invalid email or password” shown	Error shown	Pass
14	Login with new password	Login successful, user navigated to home	Login successful	Pass
15	Test with no internet connection	Error toast displayed: “Network request failed” shown	Error shown	Pass

Table 57: Change Password Test Case

1.4 Login

Test Case ID	P1-04				
Test Case Name	Login	**Test Case Description**	Verifies the login functionality using valid and invalid email and password combinations, including field validation, authentication errors, successful login, and network error handling.		
Created By	Zainab Asif	**Version**	1.0	**Date**	20 Dec 2025

S. No	**Prerequisites:**
1	User must have an account registered
2	App must be functional and accessible
3	User must know registered email and password
4	Internet connection should be active

Test Scenario	User successfully logs in by providing a valid registered email and correct password, with proper validation and authentication.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Navigate to Login Screen	Login screen is displayed with Email and Password fields	Screen displayed	Pass
2	Verify "Login" heading is being displayed	Login heading shown correctly	Heading displayed	Pass
3	Click Login button without entering email and password	Login button remain disabled	Button disabled	Pass
4	Enter invalid email format (with @abc)	Validation error "Enter a valid email address" shown	Error shown	Pass
5	Enter valid email, but leave password empty	Validation error "Password is required" shown	Error shown	Pass
6	Enter unregistered email with valid password	Backend error "Invalid email or password" shown	Error shown	Pass
7	Enter valid email with wrong password	Backend error "Invalid email or password" shown	Error shown	Pass

8	Enter valid email and password	All fields accepted, Login button enabled	Login button enabled	Pass
9	Click Login button	Button shows loading indicator	Loading indicator shown	Pass
10	Wait for login response	Success message is shown in a toast	Toast displayed	Pass
11	Verify successful navigation	User navigated to dashboard screen	Navigation Successful	Pass
12	Test with no internet connection	Error toast displayed: "Network request failed" shown	Error shown	Pass

Table 58: Login Test Case

1.5 Admin Approval/Rejection

Test Case ID	P1-05				
Test Case Name	Approve / Reject Mechanic Registration Request	**Test Case Description**	Verifies the admin functionality to view pending mechanic registration requests and approve or reject them based on Tasdeeq verification status, including validation when verification is incomplete and successful notification after approval or rejection.		
Created By	Zainab Asif	**Version**	1.0	**Date**	20 Dec 2025

S. No	**Prerequisites:**
1	Admin must be logged in to the system
2	Admin dashboard must be functional and acessible
3	At least one mechanic registration request must be in pending state
4	Internet connection should be active

Test Scenario	Admin successfully views pending mechanic registration requests and approves or rejects a selected request only after Tasdeeq verification is completed.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Admin navigates to their dashboard.	Dashboard displays pending approval count	As Expected	Pass
2	Admin clicks on "Pending Requests"	Navigates to Pending Requests page	As Expected	Pass

3	Admin selects a pending request to approve/reject.	All the details related to mechanic are displayed	As Expected	Pass
4	Admin attempts to Approve/Reject a request without Tasdeeq verification check	Approve/Reject button is disabled	Approve/Reject button was enabled	Fail
4.1	Admin attempts to Approve/reject a request without Tasdeeq verification check	Approve/Reject button is disabled	Approve/Reject button is disabled	Pass
5	Admin attempts to Approve/reject a request after Tasdeeq verification check	System approves/rejects the account, and notifies the respective mechanic via email/ notification	As Expected	Pass

Table 59: Admin Approval/Rejection Test Case

1.6 Profile Updation (User)

Test Case ID	P1-06				
Test Case Name	Profile Updation (User)	**Test Case Description**	Verifies that a user can successfully view and update personal profile information (such as full name) from the profile section of the application, including validation of unchanged data, enabling/disabling of the save button, and real-time update reflection without refreshing the screen.		
Created By	Zainab Asif	**Version**	1.0	**Date**	20 Dec 2025

S. No	**Prerequisites:**
1	User must be logged in to the application
2	User profile must already exist
3	Profile screen must be functional and accessible
4	Internet connection should be active

Test Scenario	User successfully views profile information and updates personal details. The system validates unchanged data, enables saving only when changes are made, and reflects updated information immediately on the profile screen.

Step no.	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	User opens the app	App opens successfully	As Expected	Pass
2	Go to “Profile” screen	Quick view routes are shown	As Expected	Pass
3	Client is on “Profile” screen	Correct name and profile picture in the header	As Expected	Pass
4	User clicks on “Personal Information”	All related details of the user is shown correctly	As Expected	Pass
5	User clicks on any detail like “Full name”	Update Your Name screen opens	As Expected	Pass
6	User is on Update Your Name screen	Current value should be visible disabled	As Expected	Pass
7	User is on Update Your Name screen	Update value input box with current value is enabled	As Expected	Pass
8	User doesn’t update the name	Save button is disabled	As Expected	Pass
9	User updates the name	Ready to update tag is visible	Tag wasn’t visible	Fail
9.1	User updates the name	Ready to update tag is visible	As expected	Pass
10	User updates the name	Save button is enabled and user is navigated to “My Profile” screen	As Expected	Pass
11	User is on “My Profile” screen	Updated value should be visible without refreash	As Expected	Pass

Table 60: Profile Updation (User) Test Case

1.7 Profile Updation (Mechanic Vehicle Expertise)

Test Case ID	P1-07				
Test Case Name	Profile Updation (Mechanic Vehicle Expertise)	**Test Case Description**	Verifies that a mechanic can successfully update vehicle expertise information from the profile section by adding or removing vehicle types. The system ensures changes are reflected immediately and updated expertise is saved correctly.		
Created By	Zainab Asif	**Version**	1.0	**Date**	20 Dec 2025

S. No	**Prerequisites:**
1	Mechanic must be logged in to the application
2	Mechanic profile must already exist
3	Profile screen must be functional and accessible
4	Internet connection should be active

Test Scenario	Mechanic successfully views vehicle expertise and updates it by removing or re-adding vehicle types. The system updates the expertise list in real time.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Mechanic is on "Personal Information" screen	Details are visible	As Expected	Pass
2	Mechanic goes on vehicle expertise section	The vehicles should be listed as per his choice previously with (-) button beside each	As Expected	Pass
3	Mechanic clicks (-) button	The vehicle will be removed from vehicle expertise	As Expected	Pass
4	Mechanic clicks (-) button	The vehicle will be visible with (+) button just below it	It was visible on refresh	Fail
4.1	Mechanic clicks (-) button	The vehicle will be visible with (+) button just below it	As Expected	Pass
5	Mechanic clicks (+) button	Vehicle is added back to vehicle expertise	As Expected	Pass

Table 61: Profile Updation (Mechanic Vehicle expertise) Test Case

2. Vehicle Management

2.1 View My Cars

Test Case ID	P2-01				
Test Case Name	View My Cars List	**Test Case Description**	Verifies the My Cars screen where client can view all their registered vehicles with details.		
Created By	Muhammad Osama	**Version**	1.0	**Date**	18 Dec 2025

S. No	**Prerequisites:**
1	User must be registered as Client
2	App must be functional and accessible
3	Internet connection should be active

Test Scenario	Client successfully views all registered cars with complete details, images, and action buttons.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Navigate to My Cars screen from profile section	My Cars screen is displayed	Navigated to the screen	Pass
2	Verify header "My Cars" with vehicle count	Header displays total number of vehicles	Count displayed correctly	Pass
3	Verify empty state when no cars exist	Lottie animation with "No Cars Yet" message shown	Empty state displayed	Pass
4	Verify "Add Your First Car" button in empty state	Car image or placeholder icon is shown	Image displayed	Pass
5	When cars exist, verify each car card displays image/placeholder	Number plate is displayed on bottom-left of image	Number plate displayed in wrong format	Fail
5.1	When cars exist, verify each car card displays image	Number plate is displayed on bottom-left of image	Number plate shown	Pass

6	Verify car card shows number plate badge	Car details are displayed correctly	As Expected	Pass
7	Verify car card shows make, model, and year	Specifications with icons are displayed	As Expected	Pass
8	Verify car card shows type, transmission, and color	Engine type badges are displayed	As Expected	Pass
9	Verify car card shows engine type badges and mileage if available	Engine type badges and mileage are displayed	As Expected	Pass
10	Verify Edit and Delete buttons at bottom of each card	Both action buttons are visible with icons	As Expected	Pass
11	Pull down to refresh the list	Pull-to-refresh works and reloads car data	As Expected	Pass
12	Tap on any car card	User navigates to Car Details screen	As Expected	Pass
13	Verify "+" button in header when cars exist	Add car button is visible in header	As Expected	Pass
14	Test with no internet connection	Lottie animation with "No Cars Yet" message shown	As Expected	Pass

Table 62: View My Cars Test Case

2.2 Add a New Car

Test Case ID	P2-02				
Test Case Name	Add New Car	**Test Case Description**	Verifies adding a new car with all details, images, and validations.		
Created By	Muhammad Osama	**Version**	1.0	**Date**	18 Dec 2025

S. No	**Prerequisites:**
1	User must be registered as Client
2	App must be functional and accessible
3	Internet connection should be active
4	Camera and gallery permissions granted

Test Scenario	Client successfully adds a new car with valid information and images after passing all validations.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Click "Add Car" button from My Cars screen	Add Car screen is displayed with all form sections	Add Car screen displayed	Pass
2	Verify all sections visible (Photos, Basic Info, CarType, Transmission, Engine Type)	All sections with proper icons and labels are displayed	All sections visible	Pass
3	Verify image counter shows "0 of 3 Images"	Counter displays correct initial count	Counter displayed	Pass
4	Click "Add Photo" button	Image picker modal opens with camera and gallery options	Modal opened	Pass
5	Select image from gallery or camera	Image is added and displayed with remove button	Images added	Pass
6	Add second and third images	Images are added, counter updates to "3 of 3 Images"	Images added	Pass
7	Try to add 4th image when limit reached	"Add Photo" button should be hidden. Error toast "You can only upload up to 3 images"	Button hided	Pass
8	Click remove button on any image	Image is removed and counter updates accordingly	Image removed	Pass
9	Click "Save Car" without filling any field	Validation errors displayed for all required fields	Error shown	Pass
10	Enter make and model fields with valid text	Text is accepted without errors	Text accepted	Pass
11	Enter invalid year (letters, 3 digits, or year before 1980)	Validation error "Enter valid year" is shown	Error shown	Pass
12	Enter valid 4-digit year between 1980 and current year + 1	Year is accepted	Year accepted	Pass

13	Enter color field	Color is accepted	Color accepted	Pass
14	Enter number plate in lowercase (e.g: “abc-1234”)	Number plate automatically converts to uppercase “ABC-1234”	Converted to uppercase	Pass
15	Enter invalid number plate format (e.g: “A-12”)	Validation error “Please enter a valid number” is shown	Error shown	Pass
16	Enter already registered number plate	Backend error “Number plate already exists” is shown	Error shown	Pass
17	Enter valid mileage in km	Mileage is accepted with numeric keyboard	Mileage accepted	Pass
18	Select one car type option (Hatchback, Sedan, SUV, Crossover, Pickup, Van)	Selected option is highlighted, others deselected	Option doesn’t highlights	Fail
18.1	Select one car type option (Hatchback, Sedan, SUV, Crossover, Pickup, Van)	Selected option is highlighted, others deselected	As Expected	Pass
19	Select one transmission option (Automatic or Manual)	Selected option is highlighted, other deselected	Option selected	Pass
20	Try to save without selecting engine type	Validation error "Select at least one engine type" is shown	Error shown	Pass
21	Select one or multiple engine types (Petrol, Diesel, CNG, Hybrid, Electric)	Selected options show checkmarks	Options selected	Pass
22	Enter additional notes (optional field)	Text is accepted up to 500 characters	Notes accepted	Pass
23	Fill all required fields with valid data	All fields accept data without validation errors	All data accepted	Pass
24	Click "Save Car" button	Button text changes to “Saving...” and loader appears	Loading state shown	Pass

25	Wait for backend response	Car is created successfully in database	Car created	Pass
26	Verify success toast message	"Car added successfully" toast is displayed	Toast displayed	Pass
27	Verify automatic navigation	User is redirected back to My Cars screen	Navigated back	Pass
28	Verify new car in list	Newly added car appears in My Cars list with all details	Car visible in list	Pass
29	Verify vehicle count updated	Header count increased by 1	Count updated	Pass
30	Test with no internet connection	Error toast "Network Request Failed"	Error shown	Pass

Table 63: Add New Car Test Case

2.3 Edit Car

Test Case ID	P2-03				
Test Case Name	Edit Car Details	**Test Case Description**	Verifies editing existing car information including text fields, images, and all validations with proper data pre-population and update confirmation.		
Created By	Muhammad Osama	**Version**	1.0	**Date**	18 Dec 2025

S. No	**Prerequisites:**
1	User must be registered as Client
2	At least one car must be added
3	Camera and gallery permissions granted
4	Internet connection should be active

Test Scenario	Client successfully updates car information with modified data and images, with all changes reflected in the system.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Click Edit button from car card or car details screen	Edit Car screen opens	Edit screen opened	Pass
2	Verify all existing data is in form fields	All fields display current car information	Data pre-filled	Pass

3	Verify existing images displayed without "New" badge	Existing images shown with remove option only	Existing images shown	Pass
4	Verify image counter shows correct count (e.g: "2 of 3 Images")	Counter displays current number of images	Counter correct	Pass
5	Click remove button on existing image	Confirmation modal "Delete Image" appears	Modal displayed	Pass
6	Click Cancel on delete image confirmation	Modal closes, image remains in list	Modal closed	Pass
7	Click remove and confirm deletion	Image is removed from list counter updates	Image Still there	Fail
7.1	Click remove and confirm deletion	Image is removed from list counter updates	Image Removed	Pass
8	Click "Add Photo" button	Image picker modal opens	Modal opened	Pass
9	Select new image from gallery or camera	New image added with "New" badge in corner	New image added	Pass
10	Verify that the "Add Photo" button is hidden when the maximum limit of 3 images is reached.	The "Add Photo" button should be hidden	Button hided	Pass
11	Remove newly added image	New image is removed without confirmation modal	New image removed	Pass
12	Update make field with new text	Text updates successfully	Text updated	Pass
13	Update model field with new text	Text updates successfully	Text updated	Pass
14	Clear year field and try to save	Validation error "Year is required" is shown	Error shown	Pass
15	Update invalid number plate format (e.g: "A-12")	Validation error "Please enter a valid number" is shown	Error shown	Pass

16	Update year with invalid value (e.g: “1970”)	Validation error “Enter valid year” is shown	Error shown	Pass
17	Update year with valid 4-digit year	Year is accepted	Year updated	Pass
18	Update color field	Color is updated	Color updated	Pass
19	Change number plate to already registered plate of another car	Backend error “Number plate already exists” is shown	Error shown	Pass
20	Update number plate with valid unique value	Number plate is updated (auto-converted to uppercase)	Plate updated	Pass
21	Update mileage field with new value	Mileage is updated	Mileage updated	Pass
22	Change car type selection to different option	New car type is highlighted, previous deselected	Car type changed	Pass
23	Change transmission selection	New transmission type is highlighted	Changed	Pass
24	Update engine type selections (add or remove)	Engine types are updated with checkmarks	Engine types updated	Pass
25	Try to deselect all engine types and save	Validation error “Select at least one engine type” is shown	Error shown	Pass
26	Update additional notes	Notes are updated within 500 character limit	Notes updated	Pass
27	Clear a required field and try to update	Validation error shown for that field	Error shown	Pass
28	Make multiple changes and click ”Update Car”	Button shows “Updating...” and loader appears	Loading shown	Pass
29	Wait for backend to process deleted images and updates	Images deleted from cloudinary and updates saved successfully	Updates processed	Pass
30	Verify success toast message	“Car updated Successfully” toast is displayed	Toast displayed	Pass

31	Verify automatic navigation back	User redirected to My Cars or Car Details screen (Back Screen)	Navigated back	Pass
32	Verify changes reflected in car list	Updated information visible in car card	Changes visible	Pass
33	Open car details to verify all updates	All updated fields display new values correctly	All updates reflected	Pass
34	Test edit car with no internet connection	Error toast "Network Request Failed"	Error shown	Pass

Table 64: Edit Car Test Case

2.4 View Car Details

Test Case ID	P2-04				
Test Case Name	View Car Specific Details	**Test Case Description**	Verifies viewing complete details of a selected car with all information and images		
Created By	Muhammad Osama	**Version**	1.0	**Date**	18 Dec 2025

S. No	**Prerequisites:**
1	User must be registered as Client
2	At least one car must be added
3	Internet connection should be active

Test Scenario	Client successfully views all registered cars with complete details, images, and action buttons.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Tap on any car card from My Cars list	Car Details screen opens	Details screen opened	Pass
2	Verify car images are displayed at top	Images shown and are horizontally scrollable	Images displayed	Pass
3	If multiple images, swipe through images	Images scroll horizontally with page indicators	Scrolling works	Pass
4	Verify image indicators show current image	Active indicator is highlighted	Indicators working	Pass

	position			
5	Verify back button in top-left corner	Back button is visible and functional	Button displayed	Pass
6	Verify edit button in top-right corner	Edit icon button is visible and functional	Button displayed	Pass
7	Verify delete button in top-right corner	Delete icon button is visible and functional	Button displayed	Pass
8	Verify number plate badge overlays on image	Number plate is displayed on bottom-left of image	Badge displayed	Pass
9	Verify car make and model displayed prominently	Make and model shown in large text	Displayed correctly	Pass
10	Verify year badge is shown	Year displayed with calendar icon	Year badge shown	Pass
11	Verify quick stats section (Type, Transmission, Mileage)	Three stats displayed with icons and dividers	Stats displayed	Pass
12	Verify engine type section	Engine badges shown with appropriate icons	Engine types shown	Pass
13	Verify vehicle details section with all info	All details displayed in grid format with icons	Details displayed	Pass
14	Verify additional notes section if available	Notes displayed in text box	Notes displayed	Pass
15	Scroll to bottom to view action buttons	Edit and Delete buttons visible at bottom	Buttons visible	Pass
16	Click back button from top left corner	User navigates back to My Cars list	Navigated back	Pass
17	When no images, verify placeholder icon is shown	Car icon placeholder displayed	Placeholder shown	Pass

Table 65: View Car Details Test Case

2.5 Delete Car

Test Case ID	P2-05				
Test Case Name	Delete Car	**Test Case Description**	Verifies deleting a car with confirmation modal, proper cleanup of images from storage, and updated UI reflecting the deletion..		
Created By	Muhammad Osama	**Version**	1.0	**Date**	18 Dec w2025

S. No	**Prerequisites:**
1	User must be registered as Client
2	At least one car must be added
3	Internet connection should be active

Test Scenario	Client successfully deletes a car after confirmation and sees updated list.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Navigate to My Cars screen with existing cars	My Cars list is displayed with car cards	List displayed	Pass
2	Click Delete button at bottom of any car card or from car details	Confirmation modal appears	Modal displayed	Pass
3	Verify modal title displays “Delete Car”	Title is displayed correctly	Title shown	Pass
4	Verify deletion with a confirmation modal.	“Are you sure you want to delete this car? This action cannot be undone.” is shown	Message shown	Pass
5	Verify modal has two buttons: Cancel and Delete	Both buttons are visible with proper labels	Button displayed	Pass
6	Click Cancel button	Modal closes without deleting car	Modal Closed, but car deleted	Fail
6.1	Click Cancel button	Modal closes without deleting car	As Expected	Pass
7	Verify car still exists in list	Car card remains visible in My Cars	Car still visible	Pass

8	Click Delete button again to reopen modal	Confirmation modal appears again	Modal reopened	Pass
9	Click Delete button in modal to confirm	Loader appears, deletion process starts	Deletion started	Pass
10	Wait for backend to delete car and images	Car deleted from database, images removed from Cloudinary	Car deleted	Pass
11	Verify success toast message	"Car deleted successfully" toast is displayed	Toast displayed	Pass
12	Verify car removed from My Cars list	Deleted car no longer appears in the list	Car removed from list	Pass
13	Verify vehicle count updated in header	Count decreased by 1 (e.g., "3 Vehicles" to "2 Vehicles")	Count updated	Pass
14	Verify that when the last remaining car is deleted, the empty state is displayed.	"No Cars Yet" message with Lottie animation and "Add Your First Car" button shown	Empty state displayed	Pass
15	Test delete with no internet connection	Error toast "Network Request Failed"	Error shown	Pass

Table 66: Delete Car Test Case

3. AI Chat

Test Case ID	P3-01				
Test Case Name	AI Chat Functionality	**Test Case Description**	Verifies the AI Chat functionality where client can interact with FixKar AI assistant from home screen via quick access or sidebar, send messages, receive AI responses, view chat history, and handle various scenarios including empty states, typing indicators, and error handling.		
Created By	Muhammad Osama	**Version**	1.0	**Date**	17 Dec 2025

S. No	Prerequisites:
1	User must be registered as Client
2	User must be logged in
3	Internet connection should be active
4	Backend AI service must be operational
5	User must have valid authentication token

Test Scenario	Client successfully interacts with FixKar AI and receives AI-generated responses.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Client opens AI Support from Sidebar menu	User is redirected to AI Chat screen	Navigated to AI Chat	Pass
2	Client opens AI Support from Home quick access	User is redirected to AI Chat screen	Navigated to AI Chat	Pass
3	Check initial screen state when no chat history exists	Empty state with animation and "No chats available" message	Empty state displayed	Pass
4	Verify send button is disabled when input is empty	Send button appears grayed out	Text entered, button active	Pass
5	Type a message in input field	Text is entered and send button becomes active	Text entered, button active	Pass
6	Click send button	Message is sent and appears on right side bubble	Message sent	Pass
7	Verify typing indicator appears	Typing dots animation is displayed while AI processes	Typing indicator shown	Pass
8	Wait for AI response and verify the received response	AI response appears on left side bubble with robot avatar	Response received and verified	Pass
9	Send button disabled while AI typing	User is redirected to OTP screen	Navigated successfully	Pass
10	Verify timestamp is displayed below	Time is shown for both user and AI	Timestamps displayed	Pass

	messages	messages		
11	AI replies in same language as user	Language consistency maintained	Verified	Pass
12	Send vehicle-related query	AI provides relevant automotive advice	Relevant answer provided	Pass
13	Send non-vehicle query (e.g., "What's the weather?")	AI politely declines, mentions vehicle-only assistance	Appropriate decline message	Pass
14	Scroll up and verify scroll to bottom button appears	Button with down arrow icon is displayed	Button displayed	Pass
15	Verify chat maintains conversation context within last 30 minutes	AI responses consider previous conversation history	Context maintained	Pass
16	Navigate away and return to AI Chat screen	Previous chat history within 24 hours is loaded and displayed	History loaded	Pass
17	Send empty message (only spaces)	Message is not sent, send button remains disabled	Message not sent	Pass
18	Verify AI response supports markdown formatting	Bold, lists, and formatting render correctly	Markdown rendered	Pass
19	Network/API error occurs	Empty state with animation and “No chats available” message	Empty state displayed	Pass

Table 67: AI Chat Test Case

4. Service Request Handling

4.1 Request For Service

Test Case ID	P4-01				
Test Case Name	Request for service	**Test Case Description**	This test case verifies that a logged-in client can successfully create and submit a service request by selecting a vehicle type, choosing a location, adding or selecting a vehicle, and entering valid request details. Upon submission, the system should navigate to the Active Request screen and display the request information correctly.		
Created By	Zainab Asif	**Version**	1.0	**Date**	17 Dec 2025

S. No	**Prerequisites:**
1	App must be functional and accessible
2	Client must have login to the app.
3	Internet connection should be active

Test Scenario	Client can create and submit a service request successfully.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Open client app	App opens successfully	App opens successfully	Pass
2	Select "Create Request"	Navigates to "Vehicle Type Screen"	Navigates to "Vehicle Type Screen"	Pass
3	Select "Car" or "Bike" as the vehicle type	Allow to press continue button	Allow to press continue button	Pass
4	Client press the Continue button	Navigates to "Request Details Screen"	Navigates to "Request Details Screen"	Pass
5	Select location from map	Opens the modal to search for the location	Opens the modal to search for the location	Pass

6	Search the required place	Give the matched names below using Places API	Places API not working	Fail
6.1	Search the required place	Give the matched names below using Places API	Give the matched names below using Places API	Pass
7	Selects the place	Address is shown in address bar	Address is shown in address bar	Pass
8	Select Use Current location	Toast is shown and address is shown in address bar	Toast is shown and address is shown in address bar	Pass
9	Add a new Car or bike instantly	If it's a car navigates to add a new car, if it's a bike shows the form on the same screen	Expected output achieved	Pass
10	Fills the valid details for new car	Shows the toast of car added and navigates back to "Request detail screen"	Shows the toast of car added and navigates back to "Request detail screen"	Pass
11	Select already existing Car	Tick mark is shown just beside the car	Tick mark is shown just beside the car	Pass
12	Fill the valid details for the request	Request service button is enabled	Request service button is enabled	Pass
13	Clicks on Request Service button	Navigates to "Active Request" screen System displays details of the request and near by mechanics count.	Navigates to "Active Request" screen	Pass

Table 68: Request For Service Test Case

4.2 View Nearby Requests

Test Case ID	P4-02				
Test Case Name	View nearby Requests	**Test Case Description**	This test case verifies that a mechanic can go online, view nearby service requests within a 5 km radius, accept a request, and that the client is updated in real time. It also ensures the system handles location permission denial and prevents the mechanic from accepting multiple requests at the same time.		
Created By	Zainab Asif	**Version**	1.0	**Date**	17 Dec 2025

S. No	**Prerequisites:**
1	App must be functional and accessible
2	Client/ Mechanic must have login to the app.
3	Request must not be cancelled

Test Scenario	Mechanic can go online and see nearby requests in real time.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Open mechanic app	App opens successfully	App opens successfully	Pass
2	Mechanic is offline	Displays “something went wrong” on “New Request” screen	Displays “something went wrong” on “New Request” screen	Pass
3	Mechanic toggle to get online	System asks for location permission	System asks for location permission	Pass
4	Mechanic denies for the permission	System doesn’t allow to get mechanic online	Online successful due to miss condition	Fail
4.1	Mechanic denies for the permission	System doesn’t allow to get mechanic online	System doesn’t allow to get mechanic online	Pass

5	Mechanic accepts the location permission to "Allow all the time"	Status changed to online successfully	Status changed to online successfully	Pass
6	Mechanic is already on "New Request" screen	System displays pending nearby request with 5km in real time	System displays pending nearby request with 5km in real time	Pass
7	Mechanic clicks on "New Requests"	System displays pending nearby request with 5km	System displays pending nearby request with 5km	Pass
8	Mechanic is on "New Requests" screen	App shows the nearby request counts	App shows the nearby request counts	Pass
9	Mechanic is on "New Requests" screen	App shows card for request details with timer running for request to expire	App shows card for request details with timer running for request to expire	Pass
10	Mechanic clicks on "Accept Request"	System shows a confirmation modal	System shows a confirmation modal	Pass
11	Mechanic Confirms acception	Navigates to "Active Request" screen	Navigates to "Active Request" screen	Pass
12	Client is on "Active Request" screen	Client UI will be updated in real time from pending to acceptance with minimal mechanic details	Socket wasn't working, worked on refresh	Fail
12.1	Client is on "Active Request" screen	Client UI will be updated in real time from pending to acceptance with minimal mechanic details	Expected result achieved	Pass

13	Mechanics tries to accept another parallel request	System displays an error toast that complete your active request first	System displays an error toast that complete your active request first	Pass

Table 69: View Nearby Requests Test Case

4.3 Cancel the Request

Test Case ID	P4-03				
Test Case Name	Cancel the request	**Test Case Description**	This test case verifies that a client can cancel an active service request, including showing a confirmation modal, updating the request status, navigating back to the dashboard, and notifying the mechanic of the cancellation.		
Created By	Zainab Asif	**Version**	1.0	**Date**	17 Dec 2025

S. No	**Prerequisites:**
1	App must be functional and accessible
2	Client must have login to the app.
3	Request must not be completed

Test Scenario	Client can cancel an active request and the system updates status and informs the mechanic accordingly.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Open client app	App opens successfully	App opens successfully	Pass
2	Select "Active Request"	Shows the details for request and mechanic	Shows the details for request and mechanic	Pass
3	Client selects on "Cancel Request"	Shows a modal for confirmation	Shows a modal for confirmation	Pass
4	Confirms the cancellation	Request status changed to cancel	Toast was shown but request status	Fail

			didn't change, due to API not connected	
4.1	Confirms the cancellation	Request status changed to cancel	Request status changed to cancel	Pass
5	Confirms the cancellation	Navigates to dashboard	Error message displayed	Pass
6	Confirms the cancellation	Mechanic is informed of cancellation	Mechanic is informed of cancellation	Pass

Table 70: Cancel Request Test Case

5. Real Time Chat

Test Case ID	P5-01				
Test Case Name	Real time Chat	**Test Case Description**	This test case verifies real-time chat functionality between the client and mechanic, including message delivery, inbox updates, typing indicators, chat availability based on request status, and proper removal of chat after request completion.		
Created By	Zainab Asif	**Version**	1.0	**Date**	17 Dec 2025

S. No	**Prerequisites:**
1	App must be functional and accessible
2	Client/ Mechanic must have login to the app.
3	Mechanicmust be online

Test Scenario	Client and mechanic can exchange messages in real time with proper indicators.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Open mechanic app	App opens successfully	App opens successfully	Pass
2	Mechanic is on "New Request" screen	Shows the nearby requests	Shows the nearby requests	Pass

3	Select “Chat” button	Opens the Chat screen, with name of the client at the top	Name wasn’t displayed due to wrong variable	Fail
3.1	Select “Chat” button	Opens the Chat screen, with name of the client at the top	Opens the Chat screen, with name of the client at the top	Pass
4	Sends a message	Client receives the message inside inbox in real time	Client receives the message inside inbox in real time	Pass
5	Client Opens the inbox	New tag with new messages count will be shown	New tag with new messages count will be shown	Pass
6	Client open the chat	Inbox count is updated	Inbox count wasn’t being updated, due to logic issues	Fail
6.1	Client open the chat	Inbox count is updated	Inbox count is updated	Pass
7	Client starts typing to reply	Mechanic will be able to see typing dots	Mechanic will be able to see typing dots	Pass
8	Client stops typing	Typing dots removed within 2 seconds	Typing dots removed within 2 seconds	Pass
9	Client sends the message	Mechanic will receive the message with the name of the client and picture displayed in real time	Mechanic will receive the message in real time	Pass
10	Before request is accepted , Client opens the “Active Request” screen	Only Inbox button is shown, no Chat option is available until the mechanic accepts the request or initiates the chat.	Expected output achieved	Pass

11	Mechanic or Client is on the "Active Request" screen	Expected behavior above will be correctly depicted	Expected output achieved	Pass
12	Request is completed	Chat is disappeared from the inbox	Chat is disappeared from inbox	Pass

Table 71: Real Time Chat Test Case

6. Live Tracking

6.1 View Location

Test Case ID	P6-01				
Test Case Name	View Location	**Test Case Description**	This test case verifies that a mechanic can view the client's location during an active request, navigate to the destination using Google Maps, and receive correct directions, travel time, and arrival status updates in real time.		
Created By	Zainab Asif	**Version**	1.0	**Date**	17 Dec 2025

S. No	**Prerequisites:**
1	App must be functional and accessible
2	Mechanic must have login to the app.
3	Internet connection should be active

Test Scenario	Mechanic can view client location, get directions, and confirm arrival.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Open mechanic app	App opens successfully	App opens successfully	Pass
2	Mechanic is on "Active request" screen and selects "on the way" button	Mechanic is able to see the view location icon in real time on map	Mechanic is able to see the view location icon in real time on map	Pass
3	Mechanic is on "Active request" screen and	The button text changes to Reached	The button text changes to Reached	Pass

	selects "on the way" button			
4	Select "View Location" icon on the map	Navigates to google map	Navigates to google map	Pass
5	Mechanic clicks on directions	Direction and travel time are shown correctly	Direction and travel time are shown correctly	Pass
6	When mechanic reached the location	You have reached tag is displayed	You have reached tag is displayed	Pass

Table 72: View Location Test Case

6.2 Live Track Mechanic

Test Case ID	P6-02				
Test Case Name	Live Track Mechanic	**Test Case Description**	This test case verifies that the client can track the mechanic in real time during an active request, including marker updates, distance polylines, arrival status, and proper handling when the mechanic is in foreground, background, or offline.		
Created By	Zainab Asif	**Version**	1.0	**Date**	17 Dec 2025

S. No	**Prerequisites:**
1	App must be functional and accessible
2	Client/ Mechanic must have login to the app.
3	Internet connection should be active

Test Scenario	Client can live track the mechanic with real-time location updates and arrival notifications.			
Step no.	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Open client app	App opens successfully	App opens successfully	Pass
2	Mechanic clicks on "On the Way"	Displays a button of "Live Tracking" in real time	Displays a button of "Live Tracking" in real time	Pass
3	Client clicks on "Live Track" button	Navigates to "Live Tracking" screen	Navigates to "Live Tracking"	Pass

4	Client is on "Live Tracking screen"	Markers for both client and mechanic is shown with polyline for distance in between	Markers for both client and mechanic is shown with polyline for distance in between	Fail
5	Mechanic moves few meters with app in foreground state	App updates the mechanic location after every 3 sec in the database and triggers socket	Expected result achieved	Pass
6	Mechanic moves few meters with app in background state	App updates the mechanic location after every 3 sec in the database and triggers socket	Expected result achieved	
7	Mechanic moves few meters with app in kill state	App updates the mechanic location after every 3 sec in the database and triggers socket	Expected result achieved	
8	App listens the socket	Marker is smoothly updated on map with polyline measuring distance again	Marker was updating but polyline was not updating	Fail
8.1	App listens the socket	Polyline started updating with adding dependency array for distance	Polyline started updating with adding dependency array for distance	Pass
9	Mechanic moves few meters	The marker is updated with showing the previous route it had been moving with	The marker is updated with showing the previous route it had been moving with	Pass
10	Mechanic moves few meters	The distance shown is changed in real time with google map API functional	The distance shown is changed in real time with google map API functional	Pass
11	Mechanic reached the location	The tag for "Mechanic has Reached" is displayed on the screen	Tag wasn't being displayed on time	Fail

11.1	Mechanic reached the location	The tag for “Mechanic has Reached” is displayed on the screen	Expected result achieved	Pass
12	Mechanic reached the location	The socket is not listened now although distance is shown correctly	The socket is not listened now although distance is shown correctly	Pass
13	Mechanic or client internet is turned off	Markers won’t be updated	Markers won’t be updated	Pass
13	Mechanic tries to be offline while active request is going on	System displays a warning toast, “You can’t get offline while active request is going on”.	System displays a warning toast, “You can’t get offline while active request is going on”.	Pass

Table 73: Live Track Mechanic Test Case